

ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG QUỐC TẾ



BÁO CÁO CUỐI KÌ

CƠ SỞ DỮ LIỆU

ĐỀ TÀI 4: DATAFRONTIERS REVIEW QUARTERLY

Giảng viên hướng dẫn:

TS. Vũ Đức Minh

Mã lớp Học phần:

ISV201604

Nhóm thực hiện:

Nhóm 5

Thành viên:

Phạm Ngọc Mai – 24070321

Nguyễn Thị Hồng Nhung -
24070562

Hà Nội, tháng 7 năm 2026

MỤC LỤC

DANH MỤC BẢNG	7
DANH MỤC HÌNH ẢNH.....	9
LỜI CẢM ƠN.....	10
LỜI MỞ ĐẦU	11
CHƯƠNG 1: TÓM TẮT BÀI TOÁN, QUY TẮC NGHIỆP VỤ VÀ CHANGE LOG	12
1.1. Tóm tắt bài toán và phạm vi triển khai	12
1.2. Các quy tắc nghiệp vụ (Business Rules - BRxx)	12
1.3. Bảng Change Log	14
1.4. Tác động của các thay đổi đến thiết kế tổng thể.....	17
1.5. Danh sách thực thể (Entities) và mối quan hệ chính	17
1.6. Cấu trúc file nguồn nộp kèm (Tuân thủ quy ước)	18
1.7. Lý do cần thiết kế cơ sở dữ liệu và kết luận chương.....	19
CHƯƠNG 2: ERD, MÔ HÌNH QUAN HỆ VÀ CHUẨN HÓA.....	20
2.1. Mô hình Thực thể – Liên kết (ERD).....	20
2.2. Lược đồ quan hệ (Relational Mapping).....	20
2.3. Chuẩn hóa cơ sở dữ liệu (Normalization).....	22
2.3.1. Phân tích chi tiết.....	23
2.3.2. Tổng kết	24
2.4. Kết luận chương.....	24
2.5. Sơ đồ Crow's Foot ERD tổng thể.....	26
CHƯƠNG 3: Triển khai Schema và Dữ liệu.....	27
3.1. Triển khai cấu trúc dữ liệu	27
3.1.1. Nguyên tắc thiết kế schema.....	27
3.1.2. Danh sách bảng và số lượng	27
3.2. Từ điển dữ liệu (Data Dictionary)	28
3.2.1. Từ điển dữ liệu bảng PERSON	28
3.2.2. Từ điển dữ liệu bảng MANUSCRIPT	29
3.2.3. Từ điển dữ liệu bảng ISSUE	30
3.2.4. Từ điển dữ liệu bảng REVIEW_SCORE	31
3.3. Ràng buộc cấp bảng nổi bật.....	32

3.3.1. Ràng buộc UNIQUE	32
3.3.2. Ràng buộc FOREIGN KEY với hành vi có chủ đích	32
3.3.3. Ràng buộc CHECK logic	33
3.3.4. Ràng buộc DEFAULT và ENUM.....	34
3.4. Dữ liệu mẫu (Seed Data).....	34
3.4.1. Chiến lược nạp dữ liệu	35
3.4.2. Tập dữ liệu mẫu.....	35
3.4.3. Phân bố trạng thái bài nộp.....	35
3.4.4. Mục tiêu dữ liệu mẫu	36
3.5. Kết luận chương.....	37
CHƯƠNG 4: BÁO CÁO VÀ KHAI THÁC DỮ LIỆU (SQL QUERY PACK)	38
4.1. Q01: Filter + Order trên một bảng.....	38
4.2. Q02: INNER JOIN ít nhất 3 bảng.....	39
4.3. Q03: LEFT JOIN / Tìm dữ liệu thiếu	40
4.4. Q04: GROUP BY + HAVING	41
4.5. Q05: Subquery / EXISTS	43
4.6. Q06: CTE (Common Table Expression)	44
4.7. Q07: Report theo thời gian / Date Functions	45
4.8. Q08: Query dùng View	47
4.9. Kết luận chương.....	48
CHƯƠNG 5: VIEWS VÀ STORED PROGRAMS.....	50
5.1. Views	50
5.1.2. View 2: vw_issue_table_of_content	52
5.1.3. View 3: vw_reviewer_performance	53
5.2. Stored Programs.....	55
5.2.1. Function: fn_get_avg_score	55
5.2.2. Procedure: sp_assign_reviewer.....	56
5.2.3. Procedure: sp_publish_issue	61
5.3. Tóm tắt các Stored Programs.....	64
5.4. Kết luận chương.....	64
CHƯƠNG 6: TRIGGERS VÀ EVENTS	66
6.1. Tổng quan Triggers.....	66

6.1.1. Trigger 1: Trg_Prevent_Page_Overlap_Insert.....	66
6.1.3. Trigger 3: trg_check_publication_date	69
6.1.4. Trigger 4: trg_audit_manuscript_status	70
6.1.5. Trigger 5: trg_check_review_scores_insert	71
6.1.6. Trigger 6: trg_check_review_scores_delete	72
6.1.7. Trigger 7: trg_check_review_response_date	73
6.2. Tóm tắt các Triggers	74
6.3. Event Scheduler	75
6.3.1. Event: evt_cleanup_old_audit_logs	75
6.4. Kiểm tra và giám sát	76
6.4.1. Danh sách Triggers	77
6.4.2. Danh sách Events	77
6.4.3. Trạng thái Event Scheduler	77
6.5. Kết luận chương.....	77
CHƯƠNG 7: INDEX VÀ EXECUTION PLAN	79
7.1. Mục tiêu và phạm vi	79
7.2. Index hiện tại (SHOW INDEX).....	79
7.3. Phân tích hiệu năng trước khi tạo Index	80
7.3.1. Query 1: Lọc theo status và received_date (Q01 mở rộng)	80
7.3.2. Query 2: JOIN giữa REVIEWER, PERSON, REVIEW_ASSIGNMENT (Q04)	80
7.4. Index được tạo	82
7.4.1. Index 1: idx_status_received.....	82
7.4.2. Index 2: idx_manuscript_reviewer.....	82
7.5. Phân tích hiệu năng sau khi tạo Index	82
7.5.1. Query 1 (sau khi tạo index).....	83
7.5.2. Query 2 (sau khi tạo index).....	84
7.6. So sánh trước và sau khi tạo Index	85
7.7. Danh sách các Index	86
7.8. Giải thích chi tiết các cột trong EXPLAIN.....	86
7.9. Trade-off và khuyến nghị	87
7.9.1. Lợi ích	87
7.9.2. Chi phí (Write Overhead).....	87

7.9.3. Khuyến nghị	88
7.9.4. Xóa Index (nếu cần)	88
7.10. Kết luận chương.....	88
CHƯƠNG 8: ADMINISTRATION VÀ BACKUP	89
8.1. Mục tiêu và phạm vi	89
8.2. Phân quyền người dùng (Role-Based Access Control)	89
8.2.1. Bảng mô tả chi tiết quyền hạn.....	90
8.2.2. Script tạo Role và User	91
8.2.3. Kiểm tra quyền hạn	93
8.3. Sao lưu và phục hồi (Backup & Restore Runbook).....	94
8.3.1. Nguyên tắc sao lưu.....	94
8.3.2. Sao lưu (Backup).....	94
8.3.3. Phục hồi (Restore).....	95
8.4. Bảo mật và lưu ý	96
8.5. Tóm tắt các Role và User.....	96
8.6. Kết luận chương.....	97
CHƯƠNG 9: KIỂM THỬ (TESTING).....	98
9.1. Mục tiêu và phạm vi kiểm thử	98
9.2. Positive Tests (Dữ liệu hợp lệ)	98
9.2.1. Test P01: Gán reviewer cho bài (sp_assign_reviewer).....	98
9.2.2. Test P02: Xuất bản số báo (sp_publish_issue).....	100
9.2.3. Test P03: Function fn_get_avg_score	101
9.2.4. Test P04: View vw_manuscript_review_summary	101
9.2.5. Test P05: Trigger Audit	102
9.3. Negative Tests (Dữ liệu không hợp lệ).....	104
9.3.1. Test N01: Vi phạm UNIQUE email (BR02).....	104
9.3.2. Test N02: Vi phạm CHECK score (BR09).....	105
9.3.3. Test N03: Trigger chống dè trang (BR12)	106
9.3.4. Test N04: Vi phạm logic ngày xuất bản (BR14)	107
9.3.5. Test N05: Vi phạm CHECK comments length.....	108
9.3.6. Test N06: Gán trùng reviewer (Business Rule)	110
9.3.7. Test N07: Tác giả tự phản biện (Business Rule)	111

9.4. Tóm tắt các Test Cases	112
9.5. Kết quả kiểm thử.....	112
9.6. Kết luận chương.....	113
CHƯƠNG 10: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	115
10.1. Tổng kết kết quả đạt được	115
10.1.1. Phân tích và thiết kế	115
10.1.2. Triển khai cơ sở dữ liệu	115
10.1.3. Khai thác và báo cáo	115
10.1.4. Bảo vệ và tối ưu hóa.....	116
10.1.5. Quản trị và kiểm thử	116
10.2. Đánh giá hệ thống	116
10.2.1. Ưu điểm.....	116
10.3. Hướng phát triển và mở rộng.....	117
10.3.1. Mở rộng chức năng nghiệp vụ	117
10.3.2. Ứng dụng trí tuệ nhân tạo (AI).....	118
10.3.3. Phân tích và báo cáo nâng cao	118
10.3.4. Bảo mật và phân quyền nâng cao.....	118
10.3.5. Tối ưu hóa hạ tầng.....	118
10.3.6. Tích hợp hệ thống khác	119
10.4. Kết luận chung	119
TÀI LIỆU THAM KHẢO	120

DANH MỤC BẢNG

Bảng 1: Quy tắc nghiệp vụ	14
Bảng 2: CHANGE LOG.....	17
Bảng 3: Danh sách thực thể	18
Bảng 4: Danh sách thực thể	20
Bảng 5: Hệ thống đạt chuẩn 3nf	24
Bảng 6: Nguyên tắc thiết kế schema	27
Bảng 7: Nghiệp vụ chính và phụ trợ.....	28
Bảng 8: Từ điển dữ liệu bảng Person	29
Bảng 9: Từ điển dữ liệu bảng Manuscript.....	30
Bảng 10: Từ điển dữ liệu bảng ISSUE	31
Bảng 11: Từ điển dữ liệu Review_Score.....	31
Bảng 12: Ràng buộc Unique.....	32
Bảng 13: Ràng buộc Foreign Key với hành vi có chủ đích.....	33
Bảng 14: Ràng buộc Check Logic.....	34
Bảng 15: Ràng buộc DEFAULT và ENUM.....	34
Bảng 16: Dữ liệu mẫu.....	35
Bảng 17: Phân bố trạng thái bài nộp	36
Bảng 18: Mục tiêu dữ liệu mẫu	36
Bảng 19: Mô tả chi tiết truy vấn Q01	38
Bảng 20: Mô tả chi tiết truy vấn Q02	39
Bảng 21: Mô tả chi tiết truy vấn Q03	41
Bảng 22: Mô tả chi tiết truy vấn Q04	42
Bảng 23 Mô tả chi tiết truy vấn Q05	43
Bảng 24: Mô tả chi tiết truy vấn Q06	44
Bảng 25: Mô tả chi tiết truy vấn Q07	46
Bảng 26: Mô tả chi tiết truy vấn Q08	47
Bảng 27: Tổng kết 8 truy vấn	48
Bảng 28: View 1: vw_manuscript_review_summary	50
Bảng 29: View 2: vw_issue_table_of_contents	52
Bảng 30: View 3: vw_reviewer_performance.....	53
Bảng 31: Function: fn_get_avg_score.....	55
Bảng 32: Procedure: sp_assign_reviewer.....	57
Bảng 33: Procedure: sp_publish_issue	62
Bảng 34: Tóm tắt các Stored Programs	64
Bảng 35: Trigger 1: Trg_Prevent_Page_Overlap_Insert.....	66
Bảng 36: Trigger 2: Trg_Prevent_Page_Overlap_Update	68
Bảng 37: Trigger 3: trg_check_publication_date	69
Bảng 38: Trigger 4: trg_audit_manuscript_status	70
Bảng 39: Trigger 5: trg_check_review_scores_insert.....	71

Bảng 40: Trigger 6: trg_check_review_scores_delete	72
Bảng 41: Trigger 7: trg_check_review_response_date	73
Bảng 42: Tóm tắt các Triggers	75
Bảng 43: nt: evt_cleanup_old_audit_logs	76
Bảng 44: Các Triggers và Event Scheduler.....	78
Bảng 45: Mục tiêu	79
Bảng 46: Kết quả EXPLAIN của truy vấn Q01 trước khi tối ưu	80
Bảng 47: Kết quả EXPLAIN của truy vấn Q04 trước khi tối ưu	81
Bảng 48: Kết quả EXPLAIN của truy vấn Q01 (sau khi tối ưu bằng Index).....	83
Bảng 49: Kết quả EXPLAIN của truy vấn Q01 (sau khi tối ưu bằng Index).....	84
Bảng 50: Tổng hợp so sánh hiệu năng trước và sau khi tạo Index.....	85
Bảng 51: Danh sách các Index	86
Bảng 52: Bảng từ điển giải thích các cột trong kết quả EXPLAIN	87
Bảng 53: Khuyến nghị	88
Bảng 54: Mục tiêu quản trị và bảo mật cơ sở dữ liệu.....	89
Bảng 55: Phân quyền truy cập theo vai trò (RBAC)	90
Bảng 56: Mô tả chi tiết quyền hạn.....	91
Bảng 57: Nguyên tắc sao lưu.....	94
Bảng 58: Giải thích tham số Backup	95
Bảng 59: Bảo mật và lưu ý	96
Bảng 60: Tóm tắt các Role và User.....	96
Bảng 61: Mục tiêu kiểm thử	98
Bảng 62: Mô tả chi tiết Test case P01	99
Bảng 63: Mô tả chi tiết Test case P02	100
Bảng 64: Mô tả chi tiết Test case P03	101
Bảng 65: Mô tả chi tiết Test case P04	102
Bảng 66: Mô tả chi tiết Test case P05	103
Bảng 67: Mô tả chi tiết Test case N01	104
Bảng 68: Mô tả chi tiết Test case N02	105
Bảng 69: Mô tả chi tiết Test case N03	106
Bảng 70: Mô tả chi tiết Test case N04	107
Bảng 71: Mô tả chi tiết Test case N05	109
Bảng 72: Mô tả chi tiết Test case N06	110
Bảng 73: Mô tả chi tiết Test case N07	111
Bảng 74: Tóm tắt các Test Cases	112
Bảng 75: Kết quả kiểm thử.....	113
Bảng 76: Ưu điểm của hệ thống	117
Bảng 78: Hạn chế của hệ thống.....	117

DANH MỤC HÌNH ẢNH

HÌNH 1: Sơ đồ Crow's Foot ERD tổng thể	26
Hình 2: Kết quả thực tế Q01: Filter + Order	39
Hình 3: Kết quả thực tế Q02: INNER JOIN.....	40
Hình 4: Kết quả thực tế Q03: LEFT JOIN / Tìm dữ liệu thiếu	41
Hình 5 Kết quả thực tế:Q04: GROUP BY + HAVING	43
Hình 6: Kết quả thực tế Q05: Subquery / EXISTS	44
Hình 7: Kết quả thực tế Q06: CTE	45
Hình 8: Kết quả thực tế REPORT theo thời gian / DATE FUNCTIONS.....	47
Hình 9: Kết quả thực tế QUERY dùng VIEW	48
Hình 10: Kết quả thực thi test query fn_get_avg_score	56
Hình 11: Kết quả thực thi sp_assign_reviewer.....	61
Hình 12: Kết quả thực thi sp_publish_issue.....	64
Hình 13: Kết quả thực tế EXPLAIN của truy vấn Q01 trước khi tối ưu.....	80
Hình 14: Kết quả thực tế EXPLAIN của truy vấn Q04 trước khi tối ưu.....	81
Hình 15: Kết quả thực tế của truy vấn Q01 sau khi tối ưu	83
Hình 16: Kết quả EXPLAIN thực tế của truy vấn Q01 sau khi tối ưu.....	85

LỜI CẢM ƠN

Để hoàn thành báo cáo này, chúng em xin gửi lời cảm ơn chân thành đến **Trường Quốc tế** đã tạo điều kiện thuận lợi về môi trường học tập cho nhóm trong suốt thời gian qua. Chúng em cũng xin gửi lời cảm ơn đến các thầy cô chuyên ngành **Công nghệ Tài chính và Kinh doanh số** đã tận tình giảng dạy, trang bị cho chúng em những nền tảng kiến thức vô cùng thiết thực.

Đặc biệt, nhóm chúng em xin gửi lời tri ân sâu sắc đến Giảng viên hướng dẫn – **TS. Vũ Đức Minh**. Trong suốt thời gian học tập học phần này, thầy đã luôn dành nhiều tâm huyết để truyền đạt kiến thức và sát sao với tiến độ của lớp. Những bài giảng trên lớp, cùng với sự hướng dẫn, giải đáp thắc mắc cặn kẽ của thầy không chỉ giúp chúng em hiểu rõ bản chất vấn đề mà còn biết cách vận dụng lý thuyết vào thực tế. Sự hỗ trợ nhiệt tình và những định hướng chuyên môn từ thầy đã giúp nhóm tháo gỡ được rất nhiều khó khăn để hoàn thiện báo cáo kết thúc học phần này.

Quá trình thực hiện báo cáo là cơ hội để chúng em nhìn nhận và tổng hợp lại những gì đã học. Dù đã cố gắng hết sức, nhưng do thời gian có hạn và kinh nghiệm thực tế chưa nhiều, báo cáo của nhóm chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự cảm thông, cùng những lời nhận xét và góp ý từ thầy để nhóm có thể rút kinh nghiệm và tiếp tục hoàn thiện bản thân trong các môn học tiếp theo.

Cuối cùng, chúng em kính chúc TS. Vũ Đức Minh cùng toàn thể quý thầy cô Trường Quốc tế luôn dồi dào sức khỏe, niềm vui và tiếp tục gặt hái được nhiều thành công trong công tác giảng dạy và trong cuộc sống.

Chúng em xin trân thành cảm ơn!

LỜI MỞ ĐẦU

Trong bối cảnh khoa học và công nghệ phát triển không ngừng, các ấn phẩm học thuật đóng vai trò quan trọng trong việc lan tỏa tri thức và kết quả nghiên cứu. Các tạp chí khoa học ngày nay không chỉ là kênh công bố mà còn là cầu nối giữa các nhà nghiên cứu, học giả và cộng đồng khoa học. Việc số hóa và quản lý chặt chẽ quy trình xuất bản, từ khâu tiếp nhận bài, phản biện đến khi công bố, trở thành yêu cầu cấp thiết nhằm đảm bảo tính minh bạch, chính xác và hiệu quả.

Xuất phát từ thực tế đó, nhóm lựa chọn đề tài "**DataFrontiers Review Quarterly**" với mục tiêu xây dựng một hệ thống cơ sở dữ liệu phục vụ công tác quản lý quy trình xuất bản của một tạp chí khoa học trực tuyến. Hệ thống hỗ trợ quản lý thông tin tác giả, chuyên gia phản biện, biên tập viên; quản lý bài nộp và vòng đời xử lý từ tiếp nhận, phản biện, chấp nhận đến xuất bản; đồng thời theo dõi các đánh giá chi tiết theo từng tiêu chí và lịch phát hành các số báo theo quý.

Trong phạm vi báo cáo này, nhóm đã tiến hành phân tích yêu cầu nghiệp vụ, thiết kế sơ đồ Crow's Foot ERD với 13 thực thể, xây dựng mô hình quan hệ, thực hiện chuẩn hóa dữ liệu lên đến 3NF và triển khai cơ sở dữ liệu bằng hệ quản trị MySQL. Bên cạnh đó, nhóm cũng xây dựng các truy vấn nghiệp vụ, view, stored procedure, trigger, index và thực hiện các thao tác quản trị cơ bản nhằm chứng minh hệ thống có thể vận hành trong thực tế. Thông qua quá trình thực hiện đề tài, nhóm không chỉ củng cố kiến thức đã học trong học phần Cơ sở dữ liệu mà còn nâng cao kỹ năng phân tích, thiết kế và triển khai một hệ thống cơ sở dữ liệu hoàn chỉnh, đáp ứng các yêu cầu thực tế.

Mặc dù nhóm đã nỗ lực hoàn thành báo cáo một cách tốt nhất, song do thời gian thực hiện và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp và nhận xét từ giảng viên để có thể hoàn thiện đề tài tốt hơn.

Nhóm xin chân thành cảm ơn!

CHƯƠNG 1: TÓM TẮT BÀI TOÁN, QUY TẮC NGHIỆP VỤ VÀ CHANGE LOG

1.1. Tóm tắt bài toán và phạm vi triển khai

DataFrontiers Review Quarterly là một ấn phẩm học thuật trực tuyến trong lĩnh vực Khoa học dữ liệu, Hệ thống thông tin và Phân tích kinh doanh. Hệ thống vận hành theo mô hình tạp chí phản biện kín (peer-review) với tỷ lệ chấp nhận bài thấp, đòi hỏi quy trình theo dõi nghiêm ngặt từ lúc bài được gửi đến khi xuất bản.

Phạm vi của đề tài bao gồm:

- Quản lý vòng đời của bài nộp (từ received → under_review → accepted/rejected → scheduled → published).
- Quản lý tác giả và chuyên gia phản biện (reviewer), trong đó một người có thể vừa là tác giả vừa là chuyên gia phản biện.
- Quản lý đánh giá dựa trên 4 tiêu chí (Relevance, Clarity, Methodology, Contribution) với thang điểm 10.
- Quản lý số phát hành theo quý (Spring, Summer, Fall, Winter) và việc xếp bài vào các số phát hành.

So với thiết kế giữa kỳ (ERD và mô hình quan hệ ban đầu), nhóm đã thực hiện các cập nhật quan trọng để đáp ứng đầy đủ các yêu cầu nghiệp vụ và hướng dẫn triển khai cuối kỳ. Các thay đổi này được liệt kê chi tiết trong Bảng Change Log (Mục 1.3).

1.2. Các quy tắc nghiệp vụ (Business Rules - BRxx)

Dựa trên 16 yêu cầu nghiệp vụ của đề bài, nhóm đã tiến hành phân tích, trích xuất và đánh số các quy tắc nghiệp vụ (Business Rules). Mỗi quy tắc được ánh xạ với các thành phần tương ứng trong hệ thống, bao gồm mô hình ERD, các câu lệnh DDL và tầng xử lý dữ liệu (Stored Procedures/Triggers), nhằm đảm bảo việc thiết kế và triển khai cơ sở dữ liệu đáp ứng đầy đủ các yêu cầu nghiệp vụ của bài toán.

Mã quy tắc	Mô tả quy tắc nghiệp vụ	Cơ chế thực thi (Mapping)

BR01	Mỗi người trong hệ thống phải đảm nhận ít nhất một vai trò: Author, Reviewer hoặc Editor.	Bảng PERSON là bảng gốc; AUTHOR, REVIEWER, EDITOR là bảng con với khóa chính tham chiếu đến PERSON.
BR02	Email của người dùng là duy nhất trong toàn hệ thống.	Ràng buộc UNIQUE trên cột email của bảng PERSON.
BR03	Mỗi bài nộp phải có ít nhất một tác giả.	Bảng MANUSCRIPT_AUTHOR là bảng liên kết; ràng buộc khóa chính (manuscript_id, person_id) đảm bảo không thể insert bài nếu không có tác giả.
BR04	Mỗi chuyên gia phản biện (Reviewer) phải có ít nhất một lĩnh vực chuyên môn.	Bảng REVIEWER_EXPERTISE là bảng liên kết; khóa chính (person_id, expertise_id) đảm bảo mỗi reviewer phải có ít nhất một lĩnh vực.
BR05	Các lĩnh vực chuyên môn được định nghĩa trước và có thể chưa gắn với reviewer nào.	Bảng EXPERTISE_AREA tồn tại độc lập; quan hệ với REVIEWER_EXPERTISE là optional bên phía lĩnh vực.
BR06	Mỗi bài nộp khi phù hợp phải được gửi cho ít nhất ba chuyên gia phản biện.	Kiểm soát trong Stored Procedure sp_assign_reviewer. Hệ thống chỉ chuyển trạng thái sang under_review khi số lượng reviewer ≥ 3 .
BR07	Mỗi reviewer chỉ phản hồi một lần cho mỗi bài.	Ràng buộc UNIQUE(assignment_id) trên bảng REVIEW đảm bảo mỗi assignment chỉ có tối đa một phản hồi.
BR08	Có bốn tiêu chí đánh giá cố định: mức độ phù hợp, độ rõ ràng, phương pháp nghiên cứu, đóng góp cho lĩnh vực.	Bảng CRITERION lưu danh mục 4 tiêu chí, mỗi tiêu chí có criterion_id và tên duy nhất.
BR09	Mỗi phản hồi của reviewer phải chứa điểm cho đủ 4 tiêu chí đánh giá, theo thang 10.	Bảng REVIEW_SCORE lưu điểm từng tiêu chí; ràng buộc khóa chính (review_id, criterion_id) và CHECK (score BETWEEN 0 AND 10). Đảm bảo mỗi review phải có đúng 4 bản ghi điểm.
BR10	Trạng thái bài chuyển theo quy trình: received $\rightarrow$ under_review $\rightarrow$ accepted/rejected; nếu	Sử dụng kiểu ENUM trên cột status của bảng MANUSCRIPT và kiểm soát bằng Trigger trg_manuscript_status_transition để đảm bảo các bước chuyển hợp lệ.

	accepted thì tiếp tục scheduled → published.	
BR11	Khi bài được scheduled thì phải được gán với một số phát hành (ISSUE).	Ràng buộc CHECK trên bảng MANUSCRIPT: status NOT IN ('scheduled', 'published') OR (issue_id IS NOT NULL).
BR12	Mỗi bài chỉ xuất hiện trong một số phát hành.	Cột issue_id trong bảng MANUSCRIPT là khóa ngoại tham chiếu đến ISSUE, cho phép NULL khi chưa xếp lịch.
BR13	Mỗi số phát hành được định danh duy nhất bởi tổ hợp (season, year, volume, issue_number).	Ràng buộc UNIQUE(season, year, volume, issue_number) trên bảng ISSUE.
BR14	publication_date chỉ được ghi khi số báo ở trạng thái published.	Ràng buộc CHECK trên bảng ISSUE: (issue_status = 'published' AND publication_date IS NOT NULL) OR (issue_status = 'draft' AND publication_date IS NULL).
BR15	Mỗi bài đã accepted phải được dàn trang (có page_count) trước khi chuyển sang scheduled.	Ràng buộc CHECK trên bảng MANUSCRIPT: status NOT IN ('scheduled', 'published') OR (page_count IS NOT NULL).
BR16	Một số báo có thể được tạo trước khi có bài nào được xếp lịch.	Bảng ISSUE tồn tại độc lập; cột issue_id trong MANUSCRIPT cho phép NULL.

Bảng 1: Quy tắc nghiệp vụ

1.3. Bảng Change Log

Nhằm đảm bảo tính minh bạch và nhất quán trong quá trình thiết kế và phát triển hệ thống, nhóm tiến hành liệt kê các thay đổi so với phiên bản thiết kế trước, đồng thời nêu rõ lý do và nội dung của từng thay đổi.

Mã thay đổi	Đối tượng tác động	Mô tả thay đổi	Lý do thay đổi và tác động	Tham chiếu
-------------	--------------------	----------------	----------------------------	------------

CH01	Thay đổi tên bảng USER → PERSON và thêm bảng EDITOR	Giữa kỳ, chỉ có USER dùng chung cho tác giả và reviewer. Cuối kỳ, đổi thành PERSON để ngữ nghĩa rộng hơn, đồng thời thêm bảng EDITOR kế thừa từ PERSON.	Lý do: Mở rộng phạm vi quản lý, chuẩn bị cho việc phân quyền biên tập viên (editor) – một nhu cầu thực tế trong các hệ thống tạp chí. Tác động: Thêm bảng EDITOR, các ràng buộc tham chiếu đều dùng person_id làm khóa chính. Cần cập nhật các truy vấn và ràng buộc liên quan.	Chương 2 / Yêu cầu 8, 9
CH02	Tách bảng REVIEW_S CORE và CRITERION	Giữa kỳ, gộp 4 cột điểm (Relevance, Clarity, Methodology, Contribution) vào bảng REVIEW. Cuối kỳ, tách thành CRITERION (danh mục tiêu chí) và REVIEW_SCORE (điểm từng tiêu chí).	Lý do: Đảm bảo chuẩn hóa (3NF). Dễ dàng thêm/sửa tiêu chí đánh giá mà không phải sửa cấu trúc bảng. Tác động: 1 review sẽ có 4 bản ghi trong REVIEW_SCORE (thay vì 4 cột). Cần JOIN nhiều bảng hơn để tính tổng điểm.	Chương 2 / BR09
CH03	Thêm bảng REVIEW trung gian	Giữa kỳ chỉ có REVIEW_ASSIGNMENT. Cuối kỳ, thêm bảng REVIEW để lưu tổng điểm, nhận xét chung và kết nối với REVIEW_SCORE.	Lý do: Tách bạch thông tin phân công (ngày tháng, hạn) với thông tin đánh giá tổng hợp (nhận xét, khuyến nghị) để dễ dàng báo cáo và mở rộng sau này. Tác động: Mỗi REVIEW_ASSIGNMENT tương ứng với một REVIEW (1-1), nhưng REVIEW có thể chứa thêm thông tin mở rộng (ví dụ: bình luận chi tiết).	Chương 2
CH04	Bổ sung cột start_page và order_in_issue	Thêm 2 thuộc tính này vào bảng MANUSCRIPT.	Lý do: Đáp ứng đúng Yêu cầu 15: Lưu thứ tự xuất hiện và trang bắt đầu của bài trong số phát hành. Tác động: Các cột này được điền khi bài được chuyển sang trạng thái scheduled. Thêm ràng buộc UNIQUE(issue_id, order_in_issue) và	Chương 2 / Yêu cầu 15

			UNIQUE(issue_id, start_page) để tránh xung đột.	
CH05	Bổ sung ràng buộc UNIQUE(issue_id, start_page) và UNIQUE(issue_id, order_in_issue)	Thêm các ràng buộc duy nhất để đảm bảo không có hai bài trùng thứ tự hoặc trùng trang bắt đầu trong cùng một số báo.	Lý do: Đề bài yêu cầu lưu thứ tự và trang bắt đầu, đồng thời cần đảm bảo tính toàn vẹn dữ liệu khi xếp bài. Tác động: Ngăn chặn việc gán trùng lặp, đảm bảo mỗi số báo có cấu trúc bài rõ ràng.	Chương 2 / BR12
CH06	Chuyển kiểu dữ liệu status sang ENUM với các giá trị cố định	Sử dụng kiểu dữ liệu ENUM('received','under_review','accepted','rejected','scheduled','published') thay vì VARCHAR.	Lý do: Hạn chế tối đa việc nhập sai chính tả trạng thái, đảm bảo các giá trị được phép khớp với vòng đời bài nộp. Tác động: Giúp Trigger và Procedure kiểm tra điều kiện dễ dàng hơn, tăng tốc truy vấn lọc theo trạng thái.	Chương 2
CH07	Thêm ràng buộc CHECK cho các cột quan trọng	Thêm các ràng buộc: chk_acceptance_date, chk_issue_data, chk_valid_numbers, chk_dates_logic trên bảng MANUSCRIPT; chk_issue_pubdate, chk_issue_year trên bảng ISSUE; chk_score trên bảng REVIEW_SCORE.	Lý do: Tăng cường kiểm soát dữ liệu ở tầng cơ sở dữ liệu, ngăn chặn dữ liệu không hợp lệ ngay từ khi nhập. Tác động: Đảm bảo tính toàn vẹn dữ liệu, giảm tải kiểm tra ở tầng ứng dụng.	Chương 2 / BR14
CH08	Bổ sung khóa ngoại editor_id trong MANUSCRIPT	Thêm cột editor_id tham chiếu đến EDITOR(person_id) để xác định biên tập viên phụ trách bài.	Lý do: Đáp ứng yêu cầu 8 và 9: Biên tập viên đọc sơ bộ và chọn reviewer. Cần lưu thông tin ai là người xử lý bài. Tác động: Mỗi bài phải có một editor chịu trách nhiệm, giúp truy vết lịch sử xử lý.	Chương 2 / Yêu cầu 8, 9

1.4. Tác động của các thay đổi đến thiết kế tổng thể

Các thay đổi trên đã giúp hệ thống:

1. Chuẩn hóa dữ liệu tốt hơn: Việc tách REVIEW_SCORE và CRITERION giúp hệ thống dễ dàng mở rộng thêm tiêu chí đánh giá trong tương lai mà không cần sửa cấu trúc bảng.
2. Tăng cường tính toàn vẹn: Các ràng buộc CHECK và UNIQUE đảm bảo dữ liệu nhập vào luôn hợp lệ, ngăn chặn lỗi logic từ tầng ứng dụng.
3. Hỗ trợ truy vết lịch sử: Việc bổ sung editor_id và các ràng buộc về trạng thái giúp biết rõ ai đã xử lý bài và thời điểm xử lý.
4. Đáp ứng đầy đủ yêu cầu nghiệp vụ: Các thay đổi đảm bảo tất cả 16 yêu cầu của đề bài đều được thực hiện, từ việc quản lý số phát hành, tác giả, reviewer, phản biện đến xuất bản.

1.5. Danh sách thực thể (Entities) và mối quan hệ chính

Bảng dưới đây tóm tắt 13 thực thể được sử dụng trong thiết kế cuối cùng, cùng với mô tả ngắn gọn và vai trò:

STT	Tên thực thể	Mô tả	Vai trò/ Quan hệ
1	PERSON	Lưu thông tin chung của tất cả người dùng (tác giả, reviewer, editor)	Bảng cha (Supertype).
2	AUTHOR	Tác giả bài báo	Kế thừa từ PERSON, liên kết với MANUSCRIPT qua MANUSCRIPT_AUTHOR.
3	REVIEWER	Chuyên gia phản biện	Kế thừa từ PERSON, liên kết với REVIEWER_EXPERTISE và REVIEW_ASSIGNMENT.
4	EDITOR	Biên tập viên	Kế thừa từ PERSON, để mở rộng phân quyền sau này.

5	MANUSCRIPT	Bài nộp	Lưu tiêu đề, trạng thái, ngày nhận, v.v.
6	ISSUE	Số phát hành	Lưu quý, năm, tập, số, ngày xuất bản.
7	EXPERTISE_AREA	Lĩnh vực chuyên môn	Danh mục lĩnh vực, tham chiếu bởi REVIEWER_EXPERTISE.
8	CRITERION	Tiêu chí đánh giá	Danh mục tiêu chí (Relevance, Clarity, Methodology, Contribution).
9	MANUSCRIPT_AUTHOR	Liên kết bài – tác giả	Lưu thứ tự xuất hiện của tác giả trong bài.
10	REVIEWER_EXPERTISE	Liên kết reviewer – lĩnh vực	Xác định chuyên môn của từng reviewer.
11	REVIEW_ASSIGNMENT	Phân công phản biện	Lưu ngày gửi bài cho reviewer, ngày nhận phản hồi, khuyến nghị.
12	REVIEW	Đánh giá tổng hợp	Lưu tổng điểm, nhận xét chung của reviewer cho một bài.
13	REVIEW_SCORE	Điểm chi tiết từng tiêu chí	Lưu điểm số theo từng tiêu chí (liên kết với CRITERION và REVIEW).

Bảng 3: Danh sách thực thể

1.6. Cấu trúc file nguồn nộp kèm (Tuân thủ quy ước)

Toàn bộ các script thực thi được tổ chức theo đúng cấu trúc khuyến nghị tại trang 12 của tài liệu hướng dẫn cuối kỳ, bao gồm các file:

1. README.md: Hướng dẫn cài đặt, thứ tự chạy script và thông tin môi trường (MySQL 8+, InnoDB).
2. 01_schema.sql: Chứa lệnh tạo toàn bộ Database, Tables, Constraints (PK, FK, UNIQUE, CHECK/ENUM).
3. 02_seed_data.sql: Dữ liệu mẫu đảm bảo đủ để kiểm thử các query, view và stored procedure.

4. 03_queries.sql: Tập hợp 8 truy vấn nghiệp vụ (Q01 – Q08).
5. 04_views.sql: Định nghĩa các View (tối thiểu 2 view) và lệnh SELECT kiểm thử.
6. 05_routines.sql: Stored Procedures và Functions, bao gồm các lệnh CALL để kiểm thử.
7. 06_triggers_events.sql: Định nghĩa Audit Trigger và Event Scheduler (đặt ở chế độ DISABLE cho môi trường lab).
8. 07_indexes_explain.sql: Lệnh tạo Index, SHOW INDEX và EXPLAIN cho các câu truy vấn.
9. 08_admin_backup.md: Hướng dẫn thiết lập quyền (Role/User), mysqldump và kế hoạch Restore.
10. 09_tests.sql: Các ca kiểm thử Positive và Negative (dự kiến kết quả và xử lý lỗi).

1.7. Lý do cần thiết kế cơ sở dữ liệu và kết luận chương

Hệ thống DataFrontiers Review Quarterly có nhiều thực thể và nhiều mối quan hệ phức tạp giữa bài nộp, tác giả, reviewer, phản biện và số báo. Vì vậy, cần thiết kế cơ sở dữ liệu để đảm bảo dữ liệu được lưu trữ đầy đủ, chính xác, không trùng lặp và có thể truy xuất nhanh chóng. Cơ sở dữ liệu cũng giúp hệ thống lưu lại lịch sử xử lý của từng bài, phục vụ cho việc kiểm tra, thống kê và quản lý xuất bản sau này.

Từ các yêu cầu nghiệp vụ trên, có thể xác định đây là bài toán quản lý quy trình xuất bản và phản biện bài nghiên cứu. Đây là cơ sở để xây dựng danh sách thực thể, thuộc tính và mối quan hệ ở các chương tiếp theo.

CHƯƠNG 2: ERD, MÔ HÌNH QUAN HỆ VÀ CHUẨN HÓA

2.1. Mô hình Thực thể – Liên kết (ERD)

Hệ thống **DataFrontiers_Review** sử dụng ký pháp Crow's Foot cho ERD. Các điểm cốt lõi:

Mô hình IS-A: PERSON phân nhánh thành EDITOR, REVIEWER, AUTHOR để quản lý định danh và phân quyền hiệu quả. Một người có thể đồng thời là tác giả và reviewer, do đó mô hình này cho phép linh hoạt trong việc gán vai trò.

Chuỗi nghiệp vụ chính: REVIEW_ASSIGNMENT → REVIEW → REVIEW_SCORE (tham chiếu CRITERION) mô phỏng vòng đời peer-review, cho phép lưu trữ đánh giá tổng quát và phân rã thành điểm theo tiêu chí.

Danh sách quan hệ chính

Quan hệ	Kiểu	Giải thích
PERSON → AUTHOR, REVIEWER, EDITOR	1–1	Mỗi người có thể đảm nhận một hoặc nhiều vai trò
MANUSCRIPT → MANUSCRIPT_AUTHOR → AUTHOR	M–N	Một bài có nhiều tác giả, một tác giả có nhiều bài
MANUSCRIPT → REVIEW_ASSIGNMENT → REVIEWER	1–N	Một bài được gửi đến nhiều reviewer
REVIEW_ASSIGNMENT → REVIEW	1– 0..1	Mỗi phân công chỉ có một phản hồi (có thể NULL)
REVIEW → REVIEW_SCORE → CRITERION	1–N	Một review có nhiều điểm cho các tiêu chí
ISSUE → MANUSCRIPT	1–N	Một số báo có nhiều bài; một bài thuộc một số báo (có thể chưa xác định)

Bảng 4: Danh sách thực thể

2.2. Lược đồ quan hệ (Relational Mapping)

(Quy ước: Khóa chính **in đậm gạch chân**, Khóa ngoại *in nghiêng*, Khóa vừa PK vừa FK **in đậm gạch chân và in nghiêng**.)

1. PERSON(***person_id***, full_name, email, affiliation)
2. EDITOR(***person_id***)
3. AUTHOR(***person_id***, mailing_address)
4. REVIEWER(***person_id***)
5. ISSUE(***issue_id***, season, year, volume, issue_number, issue_status, publication_date)
6. MANUSCRIPT(***manuscript_id***, title, received_date, status, acceptance_date, page_count, order_in_issue, start_page, *editor_id*, *issue_id*)
7. MANUSCRIPT_AUDIT(***audit_id***, *manuscript_id*, old_status, new_status, changed_date, changed_by)
8. PROCEDURE_AUDIT(***audit_id***, procedure_name, *manuscript_id*, *reviewer_id*, error_code, error_message, executed_at)
9. MANUSCRIPT_AUTHOR(***manuscript_id***, ***person_id***, author_order)
10. REVIEW_ASSIGNMENT(***assignment_id***, *manuscript_id*, *reviewer_id*, date_sent, deadline)
11. REVIEW(***review_id***, *assignment_id*, deadline_date, response_date, recommendation, comments)
12. CRITERION(***criterion_id***, name, description)
13. REVIEW_SCORE(***review_id***, ***criterion_id***, score)
14. EXPERTISE_AREA(***expertise_id***, expertise_code, description)
15. REVIEWER_EXPERTISE(***person_id***, ***expertise_id***)

Ghi chú ràng buộc quan trọng:

Ràng buộc UNIQUE:

PERSON.email UNIQUE – đảm bảo mỗi người có email duy nhất (BR02).

MANUSCRIPT có UNIQUE(issue_id, start_page) và UNIQUE(issue_id, order_in_issue) – thực thi BR12 (chống chồng lấn trang và trùng thứ tự trong cùng số báo).

REVIEW_ASSIGNMENT có UNIQUE(manuscript_id, reviewer_id) – mỗi reviewer chỉ được phân công một lần cho mỗi bài (BR07).

REVIEW có assignment_id UNIQUE – mỗi phân công chỉ có một phản hồi (BR07).

Ràng buộc CHECK:

PERSON.email: CHECK với REGEXP kiểm tra định dạng email cơ bản.

ISSUE: CHECK (issue_status = 'published' AND publication_date IS NOT NULL) OR (issue_status = 'draft' AND publication_date IS NULL) – thực thi BR14.

ISSUE.year: CHECK (year >= 2000 AND year <= 2030).

MANUSCRIPT: CHECK (status NOT IN ('accepted','scheduled','published') OR acceptance_date IS NOT NULL).

MANUSCRIPT: CHECK (status NOT IN ('scheduled','published') OR (issue_id IS NOT NULL AND order_in_issue IS NOT NULL AND start_page IS NOT NULL AND page_count IS NOT NULL)).

MANUSCRIPT: CHECK (page_count > 0 AND page_count <= 100 AND start_page >= 1 AND order_in_issue >= 1).

MANUSCRIPT: CHECK (acceptance_date IS NULL OR received_date <= acceptance_date).

REVIEW_SCORE.score: CHECK (score BETWEEN 0 AND 10) – thực thi BR09.

REVIEW.comments: CHECK (LENGTH(TRIM(comments)) >= 20).

Hành vi FOREIGN KEY:

ON DELETE CASCADE: Xóa cha → tự động xóa con (ví dụ: MANUSCRIPT → MANUSCRIPT_AUTHOR, REVIEW_ASSIGNMENT, REVIEW, REVIEW_SCORE).

ON DELETE RESTRICT: Ngăn xóa cha nếu còn con tham chiếu (ví dụ: CRITERION → REVIEW_SCORE, AUTHOR → MANUSCRIPT_AUTHOR).

Vòng đời trạng thái bài nộp

received → under_review → accepted / rejected → (nếu accepted) scheduled → published

received: Bài vừa được gửi đến tòa soạn.

under_review: Bài đã được ít nhất 3 reviewer và đang trong quá trình phản biện.

accepted: Bài được chấp nhận sau phản biện (có acceptance_date).

rejected: Bài bị từ chối.

scheduled: Bài đã được xếp lịch xuất bản vào một số báo cụ thể.

published: Bài đã xuất bản.

2.3. Chuẩn hóa cơ sở dữ liệu (Normalization)

Hệ thống được thiết kế đạt **3NF** (Third Normal Form), loại bỏ dị thường dữ liệu và đảm bảo dễ dàng bảo trì, mở rộng.

2.3.1. Phân tích chi tiết

Bảng MANUSCRIPT

Phụ thuộc hàm:

manuscript_id → title, received_date, status, acceptance_date, page_count, order_in_issue, start_page, editor_id, issue_id

Đánh giá: Đạt 3NF (không có phụ thuộc bắc cầu).

Bảng REVIEW_SCORE – Quyết định tách bảng (CH02)

Thiết kế cũ (gộp điểm):

REVIEW_ASSIGNMENT (assignment_id, ..., score_relevance, score_clarity, score_methodology, score_contribution)

Vấn đề 1: Nếu thêm tiêu chí mới → phải thay đổi cấu trúc bảng.

Vấn đề 2: Có thể xuất hiện NULL cho các tiêu chí không đánh giá.

Vấn đề 3: Không chuẩn 3NF do các cột điểm đại diện cho cùng loại dữ liệu (điểm số).

Thiết kế mới (tách bảng):

CRITERION(criterion_id, name, description)

REVIEW_SCORE(review_id, criterion_id, score)

Phụ thuộc hàm: (review_id, criterion_id) → score

Đánh giá: Đạt 3NF (không có phụ thuộc một phần hay bắc cầu).

Lợi ích:

Dễ dàng thêm tiêu chí mới (chỉ cần INSERT vào CRITERION).

Không có giá trị NULL.

Truy vấn thống kê theo tiêu chí đơn giản hơn.

Bảng REVIEW

Phụ thuộc hàm:

review_id → assignment_id, deadline_date, response_date, recommendation, comments

assignment_id → review_id (do UNIQUE)

Đánh giá: Đạt 3NF.

Các bảng liên kết (MANUSCRIPT_AUTHOR, REVIEWER_EXPERTISE)

Chỉ có khóa chính tổ hợp các khóa ngoại, không có thuộc tính phi khóa.

Đánh giá: Đạt BCNF (thậm chí cao hơn 3NF).

2.3.2. Tổng kết

Bảng	Chuẩn	Ghi chú
PERSON, AUTHOR, REVIEWER, EDITOR	3NF	
ISSUE	3NF	
MANUSCRIPT	3NF	
MANUSCRIPT_AUTHOR, REVIEWER_EXPERTISE	BCNF	Chỉ có khóa chính tổ hợp
REVIEW_ASSIGNMENT	3NF	
REVIEW	3NF	
REVIEW_SCORE	3NF	Đã tách từ REVIEW_ASSIGNMENT (CH02)
CRITERION	3NF	Danh mục tiêu chí
EXPERTISE_AREA	3NF	Danh mục lĩnh vực
MANUSCRIPT_AUDIT, PROCEDURE_AUDIT	3NF	Bảng phụ trợ, chỉ INSERT

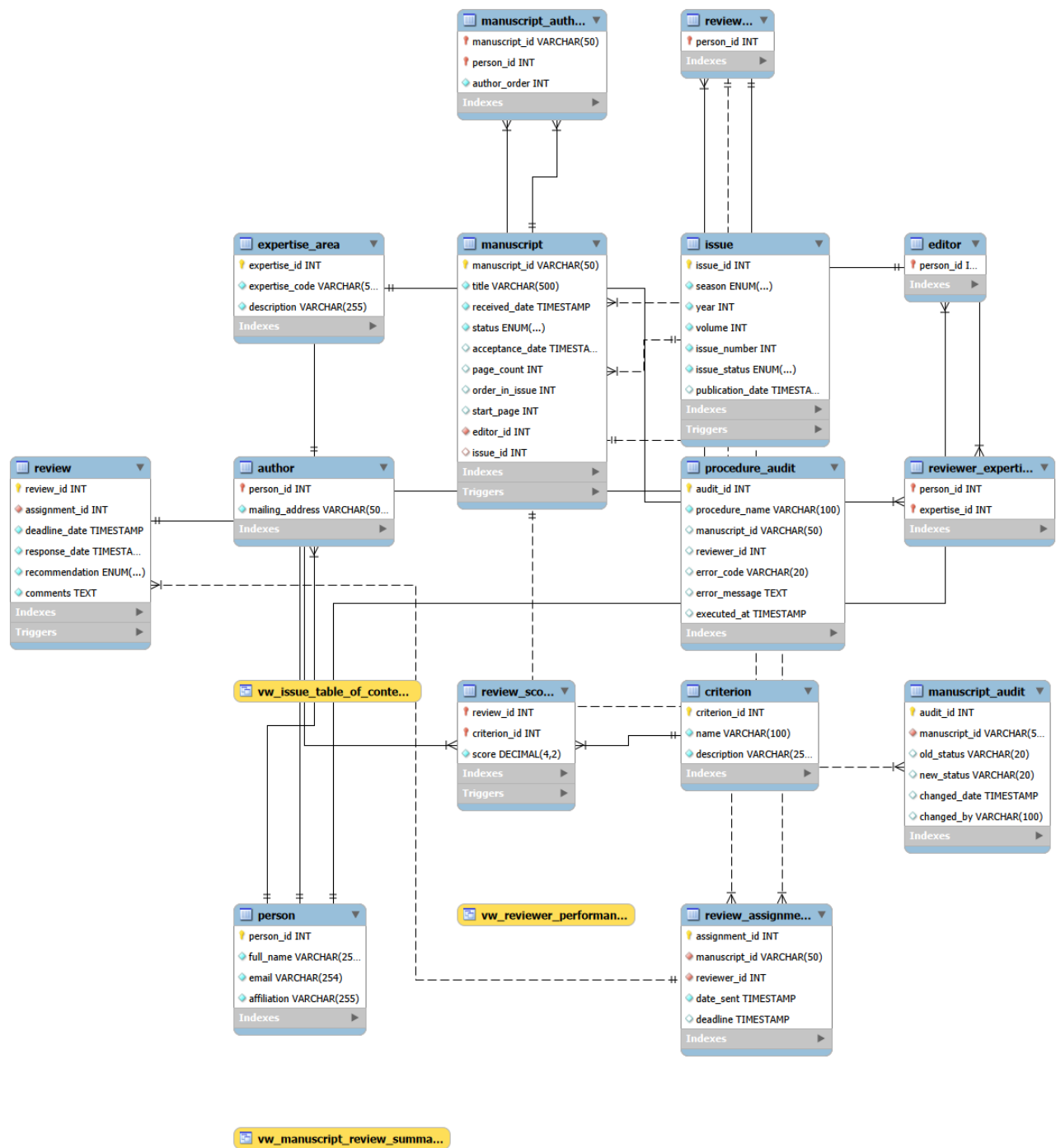
Bảng 5: Hệ thống đạt chuẩn 3nf

Kết luận: Toàn bộ hệ thống đạt chuẩn 3NF, đảm bảo: Không dư thừa dữ liệu, không có dị thường khi cập nhật, chèn, xóa, dễ dàng mở rộng và bảo trì.

2.4. Kết luận chương

Chương 2 đã trình bày một cách đầy đủ và có hệ thống về thiết kế cơ sở dữ liệu của hệ thống DataFrontiers Review Quarterly thông qua ba nội dung cốt lõi. Thứ nhất, sơ đồ ERD sử dụng ký pháp Crow's Foot với 13 thực thể được xây dựng rõ ràng, thể hiện đúng các mối quan hệ nghiệp vụ như mô hình IS-A giữa PERSON với AUTHOR, REVIEWER và EDITOR, cũng như chuỗi xử lý phản biện từ REVIEW_ASSIGNMENT đến REVIEW và REVIEW_SCORE. Thứ hai, lược đồ quan hệ được chuyển đổi trực tiếp từ ERD, phản ánh chính xác cấu trúc đã triển khai trong file 01_schema.sql với đầy đủ các ràng buộc khóa chính, khóa ngoại và các ràng buộc UNIQUE quan trọng như (issue_id, start_page) và (issue_id, order_in_issue). Thứ ba, hệ thống đã được chứng minh đạt chuẩn 3NF thông qua việc loại bỏ các phụ thuộc hàm bộ phận và bắc cầu, đặc biệt nhấn mạnh vào quyết định tách bảng REVIEW_SCORE và CRITERION khỏi REVIEW_ASSIGNMENT (Change Log CH02) - một thiết kế giúp tăng tính linh hoạt khi cần thêm tiêu chí đánh giá mới, đồng thời loại bỏ hoàn toàn các dị thường dữ liệu và giá trị NULL so với thiết kế cũ. Kết quả của chương này là một mô hình dữ liệu vững chắc, là nền tảng để triển khai schema SQL, seed data và các ràng buộc ở mức cơ sở dữ liệu trong các chương tiếp theo.

2.5. Sơ đồ Crow's Foot ERD tổng thể



HÌNH 1: Sơ đồ Crow's Foot ERD tổng thể

CHƯƠNG 3: Triển khai Schema và Dữ liệu

3.1. Triển khai cấu trúc dữ liệu

Hệ thống được triển khai trên MySQL 8+ với engine InnoDB để đảm bảo hỗ trợ giao dịch (transaction) và ràng buộc khóa ngoại (foreign key). File 01_schema.sql được xây dựng theo nguyên tắc idempotent, cho phép chạy lại nhiều lần mà không phát sinh lỗi.

3.1.1. Nguyên tắc thiết kế schema

Nguyên tắc	Mô tả	Lợi ích
Idempotent	Bắt đầu bằng SET FOREIGN_KEY_CHECKS = 0; và DROP TABLE IF EXISTS theo thứ tự phụ thuộc (con trước, cha sau)	Chạy lại script nhiều lần không bị lỗi, thuận tiện cho việc phát triển và kiểm thử
Ràng buộc ở DB	Sử dụng CHECK, UNIQUE, NOT NULL, DEFAULT và ENUM ngay tại tầng cơ sở dữ liệu	Ngăn chặn dữ liệu không hợp lệ ngay từ đầu, giảm tải xử lý cho ứng dụng
Khóa ngoại có chủ đích	Khai báo rõ ON DELETE và ON UPDATE với hành vi CASCADE hoặc RESTRICT phù hợp	Bảo vệ tính toàn vẹn dữ liệu và luồng nghiệp vụ
Đặt tên rõ ràng	Tên constraint bắt đầu bằng fk_, uq_, chk_ để dễ nhận biết	Dễ dàng bảo trì và debug khi có lỗi

Bảng 6: Nguyên tắc thiết kế schema

3.1.2. Danh sách bảng và số lượng

Dựa trên ERD đã xây dựng ở Chương 2, hệ thống triển khai 15 bảng bao gồm 13 bảng nghiệp vụ chính và 2 bảng phụ trợ (audit/log):

STT	Tên bảng	Loại	Mô tả
1	PERSON	Nghiep vụ	Người dùng chung
2	AUTHOR	Nghiep vụ	Tác giả (kế thừa PERSON)

3	REVIEWER	Nghiệp vụ	Chuyên gia phản biện (kế thừa PERSON)
4	EDITOR	Nghiệp vụ	Biên tập viên (kế thừa PERSON)
5	EXPERTISE_AREA	Danh mục	Lĩnh vực chuyên môn
6	CRITERION	Danh mục	Tiêu chí đánh giá (4 tiêu chí cố định)
7	ISSUE	Nghiệp vụ	Số phát hành
8	MANUSCRIPT	Nghiệp vụ	Bài nộp (quản lý vòng đời)
9	MANUSCRIPT_AUTHOR	Liên kết	Quan hệ N-N giữa bài và tác giả
10	REVIEWER_EXPERTISE	Liên kết	Quan hệ N-N giữa reviewer và lĩnh vực
11	REVIEW_ASSIGNMENT	Nghiệp vụ	Phân công phản biện
12	REVIEW	Nghiệp vụ	Kết quả phản hồi tổng hợp
13	REVIEW_SCORE	Nghiệp vụ	Điểm chi tiết từng tiêu chí
14	MANUSCRIPT_AUDIT	Phụ trợ	Lưu vết thay đổi trạng thái (Trigger)
15	PROCEDURE_AUDIT	Phụ trợ	Lưu log lỗi từ Stored Procedure

Bảng 7: Nghiệp vụ chính và phụ trợ

3.2. Từ điển dữ liệu (Data Dictionary)

Dưới đây là từ điển dữ liệu chi tiết cho các bảng quan trọng nhất trong hệ thống.

3.2.1. Từ điển dữ liệu bảng PERSON

Tên cột	Kiểu dữ liệu	PK/FK	Ràng buộc	Mô tả
person_id	INT	PK	AUTO_INCREMENT, NOT NULL	Mã định danh duy nhất
full_name	VARCHAR(255)		NOT NULL	Họ và tên

email	VARCHAR(254)		NOT NULL, UNIQUE	Email duy nhất
affiliation	VARCHAR(255)		NOT NULL	Đơn vị công tác
CHECK			email REGEXP '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\$'	Kiểm tra định dạng email

Bảng 8: Từ điển dữ liệu bảng Person

3.2.2. Từ điển dữ liệu bảng MANUSCRIPT

Tên cột	Kiểu dữ liệu	PK/FK	Ràng buộc	Mô tả
manuscript_id	VARCHAR(50)	PK	NOT NULL	Mã định danh duy nhất
title	VARCHAR(500)		NOT NULL, CHECK(LENGTH(TRIM(title)) > 5)	Tiêu đề bài báo
received_date	TIMESTAMP		NOT NULL	Ngày nhận bài
status	ENUM		NOT NULL	Trạng thái vòng đời
acceptance_date	TIMESTAMP		NULL	Ngày chấp nhận
page_count	INT		NULL, CHECK(page_count > 0 AND page_count <= 100)	Số trang
order_in_issue	INT		NULL, UNIQUE(issue_id, order_in_issue)	Thứ tự trong số báo

start_page	INT		NULL, UNIQUE(issue_id, start_page)	Trang bắt đầu
editor_id	INT	FK	NOT NULL → EDITOR(person_id)	Biên tập viên phụ trách
issue_id	INT	FK	NULL → ISSUE(issue_id)	Số báo chứa bài

Bảng 9: Từ điển dữ liệu bảng Manuscript

Trạng thái ENUM: received, rejected, under_review, accepted, scheduled, published

Ràng buộc logic:

chk_acceptance_date: Nếu status ∈ {accepted, scheduled, published} thì acceptance_date IS NOT NULL

chk_issue_data: Nếu status ∈ {scheduled, published} thì issue_id, order_in_issue, start_page, page_count NOT NULL

chk_valid_numbers: page_count > 0 AND page_count ≤ 100 AND start_page ≥ 1 AND order_in_issue ≥ 1

chk_dates_logic: acceptance_date IS NULL OR received_date ≤ acceptance_date

3.2.3. Từ điển dữ liệu bảng ISSUE

Tên cột	Kiểu dữ liệu	PK/FK	Ràng buộc	Mô tả
issue_id	INT	PK	AUTO_INCREMENT, NOT NULL	Mã số báo
season	ENUM		NOT NULL	Kỳ phát hành (Spring/Summer/Fall/Winter)
year	INT		NOT NULL, CHECK(year ≥ 2000 AND year ≤ 2030)	Năm phát hành

Tên cột	Kiểu dữ liệu	PK/FK	Ràng buộc	Mô tả
volume	INT UNSIGNED		NOT NULL	Tập
issue_number	INT UNSIGNED		NOT NULL	Số
issue_status	ENUM		NOT NULL	Trạng thái số báo
publication_date	TIMESTAMP		NULL	Ngày xuất bản
UNIQUE			(season, year, volume, issue_number)	Định danh duy nhất

Bảng 10: Từ điển dữ liệu bảng ISSUE

3.2.4. Từ điển dữ liệu bảng REVIEW_SCORE

Tên cột	Kiểu dữ liệu	PK/FK	Ràng buộc	Mô tả
review_id	INT	PK, FK	NOT NULL → REVIEW(review_id) ON DELETE CASCADE	Liên kết tới REVIEW
criterion_id	INT	PK, FK	NOT NULL → CRITERION(criterion_id) ON DELETE RESTRICT	Liên kết tới CRITERION
score	DECIMAL(4,2)		NOT NULL, CHECK(score BETWEEN 0 AND 10)	Điểm cho tiêu chí (thang 10)

Bảng 11: Từ điển dữ liệu Review_Score

Review_id sử dụng ON DELETE CASCADE (xóa review → xóa điểm), criterion_id sử dụng ON DELETE RESTRICT (không xóa tiêu chí nếu đã có điểm).

3.3. Ràng buộc cấp bảng nổi bật

3.3.1. Ràng buộc UNIQUE

Ràng buộc	Bảng	Mô tả	Yêu cầu nghiệp vụ
uq_issue	ISSUE	UNIQUE(season, year, volume, issue_number)	BR13: Định danh duy nhất số báo
uq_issue_order	MANUSCRIPT	UNIQUE(issue_id, order_in_issue)	BR15: Không trùng thứ tự trong cùng issue
uq_issue_start	MANUSCRIPT	UNIQUE(issue_id, start_page)	BR15: Không trùng trang bắt đầu trong cùng issue
uq_manu_rev	REVIEW_ASSIGNMENT	UNIQUE(manuscript_id, reviewer_id)	BR07: Mỗi reviewer chỉ phân công 1 lần/bài
uq_manu_order	MANUSCRIPT_AUTHOR	UNIQUE(manuscript_id, author_order)	BR04: Không trùng thứ tự tác giả trong cùng bài

Bảng 12: Ràng buộc Unique

3.3.2. Ràng buộc FOREIGN KEY với hành vi có chủ đích

Hành vi	Bảng con	Bảng cha	Ý nghĩa nghiệp vụ
ON DELETE CASCADE	MANUSCRIPT_AUTHOR	MANUSCRIPT	Xóa bài → xóa toàn bộ liên kết tác giả

ON DELETE CASCADE	REVIEW_ASSIGNMENT	MANUSCRIPT	Xóa bài → xóa toàn bộ phân công liên quan
ON DELETE CASCADE	REVIEW_SCORE	REVIEW	Xóa review → xóa toàn bộ điểm
ON DELETE RESTRICT	MANUSCRIPT_AUTHOR	AUTHOR	Ngăn xóa tác giả đã có bài
ON DELETE RESTRICT	REVIEWER_EXPERTISE	EXPERTISE_AREA	Ngăn xóa lĩnh vực đã có reviewer
ON DELETE RESTRICT	REVIEW	REVIEW_ASSIGNMENT	Ngăn xóa assignment đã có review

Bảng 13: Ràng buộc Foreign Key với hành vi có chủ đích

3.3.3. Ràng buộc CHECK logic

Ràng buộc	Bảng	Mô tả	Yêu cầu nghiệp vụ
chk_person_email	PERSON	Kiểm tra định dạng email hợp lệ	BR02
chk_issue_pubdate	ISSUE	issue_status = published → publication_date NOT NULL	BR14
chk_acceptance_date	MANUSCRIPT	status ∈ {accepted, scheduled, published} → acceptance_date NOT NULL	BR13
chk_issue_data	MANUSCRIPT	status ∈ {scheduled, published} → issue_id, order_in_issue, start_page, page_count NOT NULL	BR11, BR15
chk_score	REVIEW_SCORE	score BETWEEN 0 AND 10	BR09
chk_comments_length	REVIEW	LENGTH(TRIM(comments)) >= 20	Đảm bảo nhận xét

Ràng buộc	Bảng	Mô tả	Yêu cầu nghiệp vụ
			có ý nghĩa
chk_title_meaningful	MANUSCRIPT	CHECK(LENGTH(TRIM(title)) > 5)	Đảm bảo tiêu đề có ý nghĩa
chk_valid_numbers	MANUSCRIPT	CHECK(page_count > 0 AND page_count <= 100 AND start_page >= 1 AND order_in_issue >= 1)	Đảm bảo giá trị số hợp lệ
chk_dates_logic	MANUSCRIPT	CHECK(acceptance_date IS NULL OR received_date <= acceptance_date)	Đảm bảo ngày chấp nhận không trước ngày nhận

Bảng 14: Ràng buộc Check Logic

3.3.4. Ràng buộc DEFAULT và ENUM

Ràng buộc	Bảng	Mô tả
DEFAULT CURRENT_TIMESTAMP	MANUSCRIPT_AUDIT.changed_date	Tự động ghi thời gian khi trigger audit được kích hoạt
DEFAULT 'System_Trigger'	MANUSCRIPT_AUDIT.changed_by	Ghi nhận nguồn thay đổi
ENUM	ISSUE.season	Giới hạn 4 giá trị: Spring, Summer, Fall, Winter
ENUM	MANUSCRIPT.status	Giới hạn 6 trạng thái vòng đời

Bảng 15: Ràng buộc DEFAULT và ENUM

3.4. Dữ liệu mẫu (Seed Data)

3.4.1. Chiến lược nạp dữ liệu

File **02_seed_data.sql** được xây dựng theo nguyên tắc:

Idempotent: Bắt đầu bằng SET FOREIGN_KEY_CHECKS = 0; và TRUNCATE TABLE theo thứ tự phụ thuộc để chạy lại nhiều lần không bị lỗi.

Trình tự theo phụ thuộc FK: Nạp bảng cha trước, bảng con sau.

Thứ tự nạp:

PERSON → EDITOR/AUTHOR/REVIEWER →
EXPERTISE_AREA/CRITERION → ISSUE → MANUSCRIPT →
MANUSCRIPT_AUTHOR → REVIEW_ASSIGNMENT → REVIEW →
REVIEW_SCORE

3.4.2. Tập dữ liệu mẫu

Nhóm	Số lượng	Mô tả
PERSON	12	Người dùng, bao gồm editor, reviewer, author
EDITOR	3	Biên tập viên (person_id: 1, 2, 10)
REVIEWER	7	Chuyên gia phản biện (person_id: 3, 4, 5, 8, 9, 10, 12)
AUTHOR	5	Tác giả (person_id: 4, 6, 7, 8, 11)
EXPERTISE_AREA	6	Lĩnh vực chuyên môn (DS101, IS203, BA305, ML404, DS205, IS107)
CRITERION	4	Tiêu chí đánh giá cố định
ISSUE	4	Số phát hành (Spring, Summer, Fall, Winter 2026)
MANUSCRIPT	7	Bài nộp trải đều 6 trạng thái
REVIEW_ASSIGNMENT	15	Phân công phản biện
REVIEW	15	Phản hồi tương ứng
REVIEW_SCORE	60	Mỗi review 4 điểm tương ứng 4 tiêu chí

Bảng 16: Dữ liệu mẫu

3.4.3. Phân bố trạng thái bài nộp

Trạng thái	Mã bài	Số lượng	Mô tả
published	MF-26-001	1	Đã xuất bản trong Issue 1
scheduled	MF-26-002, MF-26-007	2	Đã xếp lịch xuất bản
under_review	MF-26-003	1	Đang trong quá trình phản biện
received	MF-26-004	1	Mới nhận, chưa có reviewer
rejected	MF-26-005	1	Bị từ chối
accepted	MF-26-006	1	Được chấp nhận, chưa xếp lịch

Bảng 17: Phân bố trạng thái bài nộp

3.4.4. Mục tiêu dữ liệu mẫu

Dữ liệu mẫu được thiết kế để đáp ứng các mục tiêu kiểm thử sau:

Mục tiêu	Dữ liệu hỗ trợ
Kiểm thử queries (Chương 4)	Đủ 7 bài với 6 trạng thái khác nhau
Kiểm thử views (Chương 4)	Đủ dữ liệu JOIN giữa MANUSCRIPT, REVIEW_ASSIGNMENT, REVIEW, REVIEW_SCORE
Kiểm thử stored procedures (Chương 5)	Bài MF-26-004 ở trạng thái received (có thể gán reviewer), Issue 2 ở trạng thái draft (có thể xuất bản)
Kiểm thử triggers (Chương 6)	Dữ liệu đủ để kiểm tra trigger chống đề trang, audit log, kiểm tra điểm
Kiểm thử index (Chương 7)	Đủ số lượng bài và phân công để EXPLAIN hiển thị rõ việc sử dụng index
Kiểm thử ràng buộc (Chương 9)	Dữ liệu đủ để tạo các test case negative (email trùng, điểm >10, trang đề)

Bảng 18: Mục tiêu dữ liệu mẫu

3.5. Kết luận chương

Chương 3 đã trình bày chi tiết quá trình vật lý hóa mô hình dữ liệu từ Chương 2 thành cơ sở dữ liệu thực tế, với schema idempotent được định nghĩa trong file `01\_schema.sql` bao gồm toàn bộ cấu trúc bảng, các ràng buộc toàn vẹn và trigger. Từ điển dữ liệu được xây dựng chi tiết cho các bảng quan trọng như MANUSCRIPT, ISSUE và REVIEW_SCORE, giúp người đọc dễ dàng tra cứu ý nghĩa và ràng buộc của từng cột. Hệ thống ràng buộc toàn vẹn ở cấp bảng được thiết kế đa dạng bao gồm UNIQUE để đảm bảo tính duy nhất, FOREIGN KEY với hành vi CASCADE/RESTRICT để bảo vệ luồng nghiệp vụ, CHECK để kiểm soát miền giá trị, ENUM để giới hạn các trạng thái hợp lệ và DEFAULT để tự động gán giá trị khi cần thiết. Cuối cùng, dữ liệu mẫu trong file `02\_seed\_data.sql` được nạp theo chiến lược có cấu trúc với trình tự tuân theo phụ thuộc khóa ngoại, đảm bảo đủ các trường hợp dữ liệu để kiểm thử toàn diện các truy vấn, view, stored procedure và trigger ở các chương tiếp theo, tạo nền tảng vững chắc cho việc khai thác và vận hành hệ thống.

CHƯƠNG 4: BÁO CÁO VÀ KHAI THÁC DỮ LIỆU (SQL QUERY PACK)

Chương này trình bày tập hợp 8 truy vấn SQL giải quyết các bài toán nghiệp vụ thực tế của tòa soạn DataFrontiers Review Quarterly. Mỗi truy vấn được thiết kế để trả lời một câu hỏi nghiệp vụ cụ thể, áp dụng các kỹ thuật SQL đa dạng từ cơ bản đến nâng cao, đồng thời có nhận xét về hiệu năng và khả năng tối ưu.

4.1. Q01: Filter + Order trên một bảng

Trường	Nội dung
Business question	Tòa soạn muốn lấy danh sách bài nộp trong năm 2026, bài mới nhận nhất đưa lên đầu danh sách để theo dõi tiến độ xử lý.
Input/Parameter	Năm tiếp nhận: 2026
SQL technique	WHERE kết hợp hàm YEAR(), sắp xếp ORDER BY received_date DESC
Expected result	4 cột: manuscript_id, title, status, received_date (dòng đầu tiên là bài nộp gần đây nhất)
Performance note	Cần index trên received_date để tối ưu. Việc dùng YEAR() có thể gây mất tác dụng index (non-SARGable), tối ưu hơn có thể dùng mệnh đề biên độ khoảng BETWEEN.

Bảng 19: Mô tả chi tiết truy vấn Q01

SQL

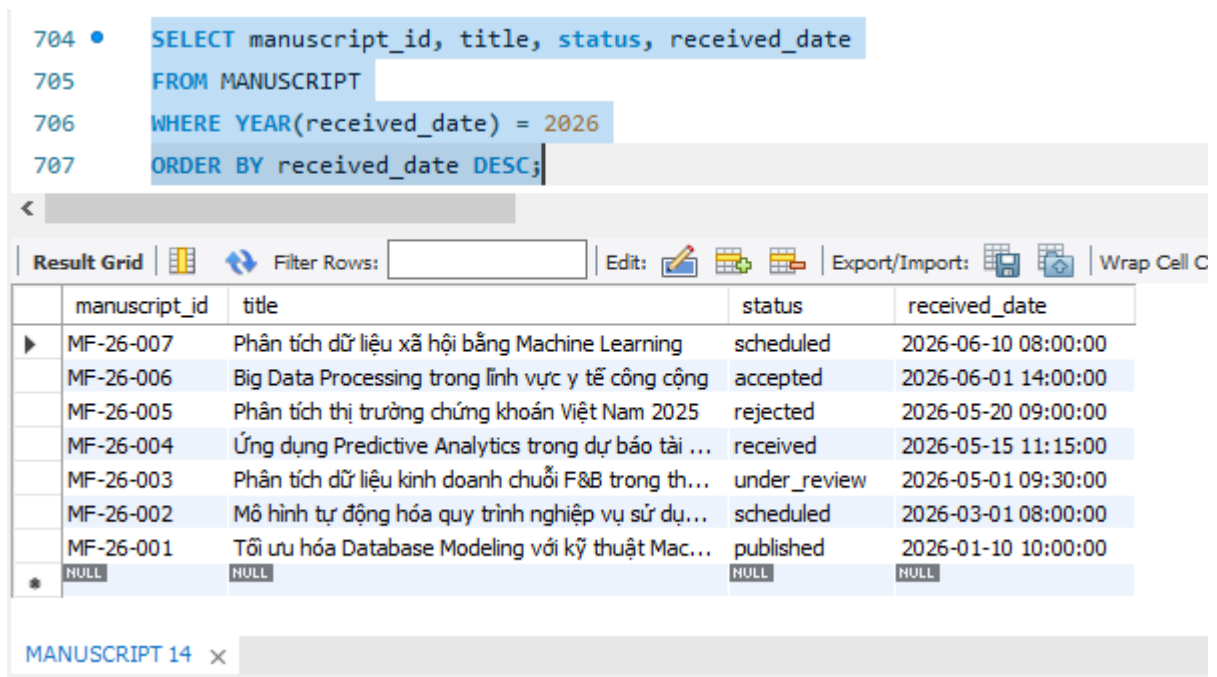
```
SELECT manuscript_id, title, status, received_date
```

```
FROM MANUSCRIPT
```

```
WHERE YEAR(received_date) = 2026
```

```
ORDER BY received_date DESC;
```

Kết quả thực tế:



Hình 2: Kết quả thực tế Q01: Filter + Order

4.2. Q02: INNER JOIN ít nhất 3 bảng

Trường	Nội dung
Business question	Tòa soạn muốn biết những tác giả nào có bài báo đã vượt qua vòng phản biện (chấp nhận, lên lịch hoặc đã xuất bản) và tên bài báo đó là gì.
Input/Parameter	Trạng thái: 'accepted', 'scheduled', 'published'
SQL technique	INNER JOIN liên kết 4 bảng (AUTHOR, PERSON, MANUSCRIPT_AUTHOR, MANUSCRIPT) kết hợp điều kiện IN
Expected result	author_name (tên tác giả), title (tên bài) và status (trạng thái)
Performance note	MySQL Optimizer sẽ tự động tận dụng các khóa chính (person_id, manuscript_id) để ghép bảng nhanh chóng. Nên chạy EXPLAIN để xem thứ tự ghép bảng tối ưu.

Bảng 20: Mô tả chi tiết truy vấn Q02

SQL

SELECT p.full_name AS author_name, m.title, m.status

FROM AUTHOR a

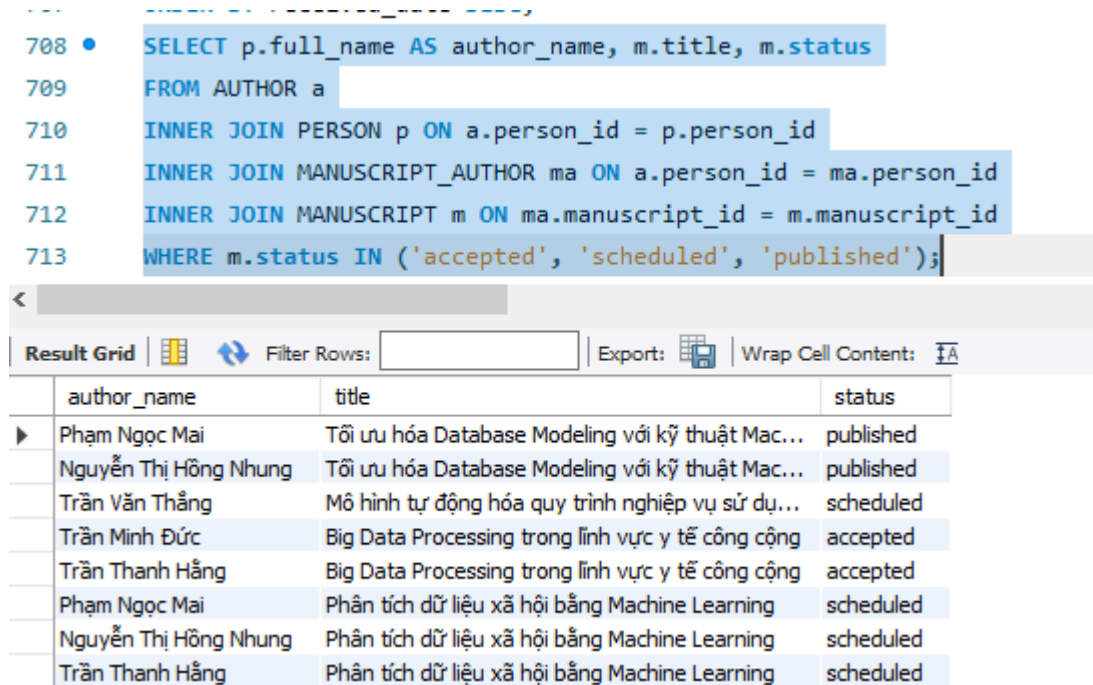
INNER JOIN PERSON p ON a.person_id = p.person_id

INNER JOIN MANUSCRIPT_AUTHOR ma ON a.person_id = ma.person_id

INNER JOIN MANUSCRIPT m ON ma.manuscript_id = m.manuscript_id

WHERE m.status IN ('accepted', 'scheduled', 'published');

Kết quả thực tế:



```
708 • SELECT p.full_name AS author_name, m.title, m.status
709 FROM AUTHOR a
710 INNER JOIN PERSON p ON a.person_id = p.person_id
711 INNER JOIN MANUSCRIPT_AUTHOR ma ON a.person_id = ma.person_id
712 INNER JOIN MANUSCRIPT m ON ma.manuscript_id = m.manuscript_id
713 WHERE m.status IN ('accepted', 'scheduled', 'published');
```

author_name	title	status
Phạm Ngọc Mai	Tối ưu hóa Database Modeling với kỹ thuật Mac...	published
Nguyễn Thị Hồng Nhung	Tối ưu hóa Database Modeling với kỹ thuật Mac...	published
Trần Văn Thắng	Mô hình tự động hóa quy trình nghiệp vụ sử dụ...	scheduled
Trần Minh Đức	Big Data Processing trong lĩnh vực y tế công cộng	accepted
Trần Thanh Hằng	Big Data Processing trong lĩnh vực y tế công cộng	accepted
Phạm Ngọc Mai	Phân tích dữ liệu xã hội bằng Machine Learning	scheduled
Nguyễn Thị Hồng Nhung	Phân tích dữ liệu xã hội bằng Machine Learning	scheduled
Trần Thanh Hằng	Phân tích dữ liệu xã hội bằng Machine Learning	scheduled

Hình 3: Kết quả thực tế Q02: INNER JOIN

4.3. Q03: LEFT JOIN / Tìm dữ liệu thiếu

Trường	Nội dung
Business question	Có chuyên gia phản biện (Reviewer) nào có trong hệ thống nhưng chưa từng được phân công bài nào không? Editor cần phát hiện để kịp thời giao việc.
Input/Parameter	Không có (duyệt toàn bộ mạng lưới chuyên gia)
SQL technique	LEFT JOIN từ REVIEWER sang REVIEW_ASSIGNMENT, lọc các bản ghi bị khuyết với điều kiện ra.assignment_id IS NULL
Expected result	person_id, full_name và email của các chuyên gia đang "nhàn rỗi"

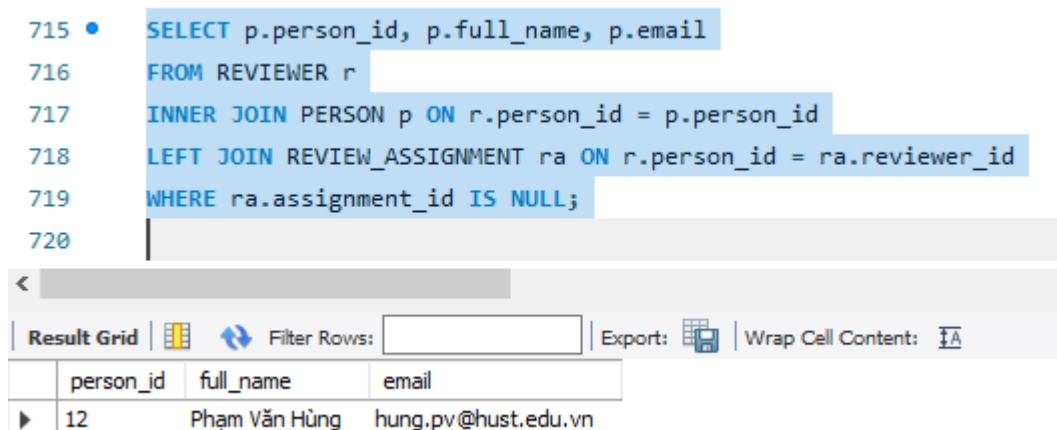
Performance note	Truy vấn có rủi ro quét toàn bộ bảng. Khuyến nghị tạo index trên khóa ngoại reviewer_id ở bảng REVIEW_ASSIGNMENT.
------------------	---

Bảng 21: Mô tả chi tiết truy vấn Q03

SQL

```
SELECT p.person_id, p.full_name, p.email
FROM REVIEWER r
INNER JOIN PERSON p ON r.person_id = p.person_id
LEFT JOIN REVIEW_ASSIGNMENT ra ON r.person_id = raReviewer_id
WHERE ra.assignment_id IS NULL;
```

Kết quả thực tế:



Hình 4: Kết quả thực tế Q03: LEFT JOIN / Tìm dữ liệu thiếu

4.4. Q04: GROUP BY + HAVING

Trường	Nội dung
Business question	Chuyên gia nào được phân công nhiều bài phản biện nhất trong năm 2026? Chỉ lọc những người có số lượng công việc > 0.
Input/Parameter	Năm 2026, ngưỡng số bài phân công > 0

SQL technique	GROUP BY theo ID và tên chuyên gia, sử dụng COUNT() kết hợp HAVING để lọc theo ngưỡng
Expected result	Danh sách reviewer_name và total_assignments (đã sắp xếp giảm dần theo số lượng bài)
Performance note	Index idx_manuscript_reviewer trên (manuscript_id, reviewer_id) đã được tạo để hỗ trợ JOIN và đếm. Để tối ưu truy vấn lọc theo date_sent, có thể cân nhắc thêm index (reviewer_id, date_sent) nếu dữ liệu lớn và thường xuyên truy vấn theo năm.

Bảng 22: Mô tả chi tiết truy vấn Q04

SQL

```

SELECT p.full_name AS reviewer_name, COUNT(ra.assignment_id) AS total_assignments
FROM REVIEWER r
INNER JOIN PERSON p ON r.person_id = p.person_id
INNER JOIN REVIEW_ASSIGNMENT ra ON r.person_id = ra.reviewer_id
WHERE YEAR(ra.date_sent) = 2026
GROUP BY p.person_id, p.full_name
HAVING COUNT(ra.assignment_id) > 0
ORDER BY total_assignments DESC;

```

Kết quả thực tế:

722	FROM REVIEWER r
723	INNER JOIN PERSON p ON r.person_id = p.person_id
724	INNER JOIN REVIEW_ASSIGNMENT ra ON r.person_id = ra.reviewer_id
725	WHERE YEAR(ra.date_sent) = 2026
726	GROUP BY p.person_id, p.full_name
727	HAVING COUNT(ra.assignment_id) > 0
728	ORDER BY total_assignments DESC;
729	

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	reviewer_name	total_assignments		
▶	Lê Anh Khoa	3		
	Trần Minh Đức	3		
	Lê Hải Yến	3		
	Ngô Thị Quỳnh	2		
	Trần Văn Thắng	2		
	Vũ Đức Minh	2		

4.5. Q05: Subquery / EXISTS

Trường	Nội dung
Business question	Những bài báo nào đang bị chậm tiến độ? (Đã được phân công đủ 3 Reviewer nhưng đến nay vẫn chưa nhận đủ 3 phản hồi).
Input/Parameter	Ngưỡng phân công ≥ 3 , ngưỡng phản hồi < 3
SQL technique	Correlated Subquery lồng trong mệnh đề WHERE để đếm số bản ghi so sánh động cho từng dòng của bảng chính
Expected result	manuscript_id và title của các bài báo cần được gửi email hỏi thúc chuyên gia <i>Lưu ý: Bài MF-26-003 có 3 reviewer nhưng chỉ có 2 review đã hoàn thành (review 7, 8) → sẽ xuất hiện trong kết quả.</i>
Performance note	Correlated Subquery có thể gây Using where; Using index dạng vòng lặp lồng. Nếu dữ liệu quá lớn, chuyển sang LEFT JOIN và HAVING COUNT sẽ cho Execution Plan hiệu quả hơn.

Bảng 23 Mô tả chi tiết truy vấn Q05

SQL

SELECT m.manuscript_id, m.title

FROM MANUSCRIPT m

WHERE

(SELECT COUNT(*) FROM REVIEW_ASSIGNMENT ra WHERE ra.manuscript_id = m.manuscript_id) ≥ 3

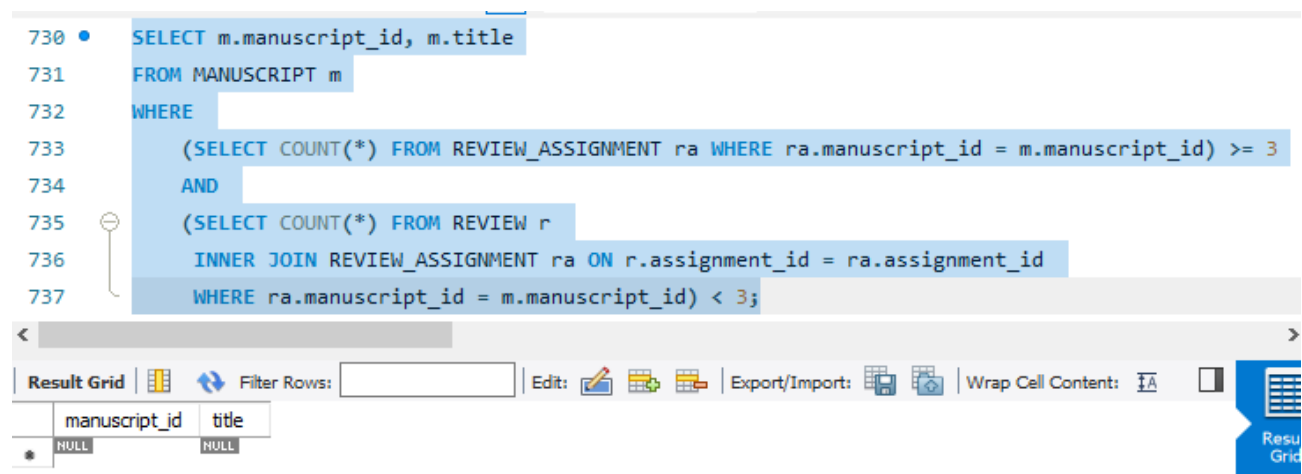
AND

(SELECT COUNT(*) FROM REVIEW r

INNER JOIN REVIEW_ASSIGNMENT ra ON r.assignment_id = ra.assignment_id

WHERE ra.manuscript_id = m.manuscript_id) < 3 ;

Kết quả thực tế:



Hình 6: Kết quả thực tế Q05: Subquery / EXISTS

4.6. Q06: CTE (Common Table Expression)

Trường	Nội dung
Business question	Điểm trung bình của từng bài viết hiện tại là bao nhiêu? Xếp hạng bài viết có điểm cao nhất để ưu tiên xuất bản.
Input/Parameter	Không có tham số tĩnh
SQL technique	Khởi tạo bảng biểu diễn tạm thời WITH AvgScores AS (...) (CTE) để tính điểm trung bình trước, sau đó INNER JOIN lại với bảng gốc để lấy tên bài báo
Expected result	manuscript_id, title, và avg_score (xếp hạng giảm dần)
Performance note	CTE giúp code sạch hơn và cho phép Optimizer tái sử dụng kết quả (materialized) trong bộ nhớ tạm mà không cần quét lại bảng con nhiều lần. CTE chỉ hiển thị bài đã có ít nhất 1 review. Nếu cần hiển thị tất cả bài (kể cả chưa có review), có thể thay INNER JOIN thành LEFT JOIN và xử lý NULL.

Bảng 24: Mô tả chi tiết truy vấn Q06

SQL

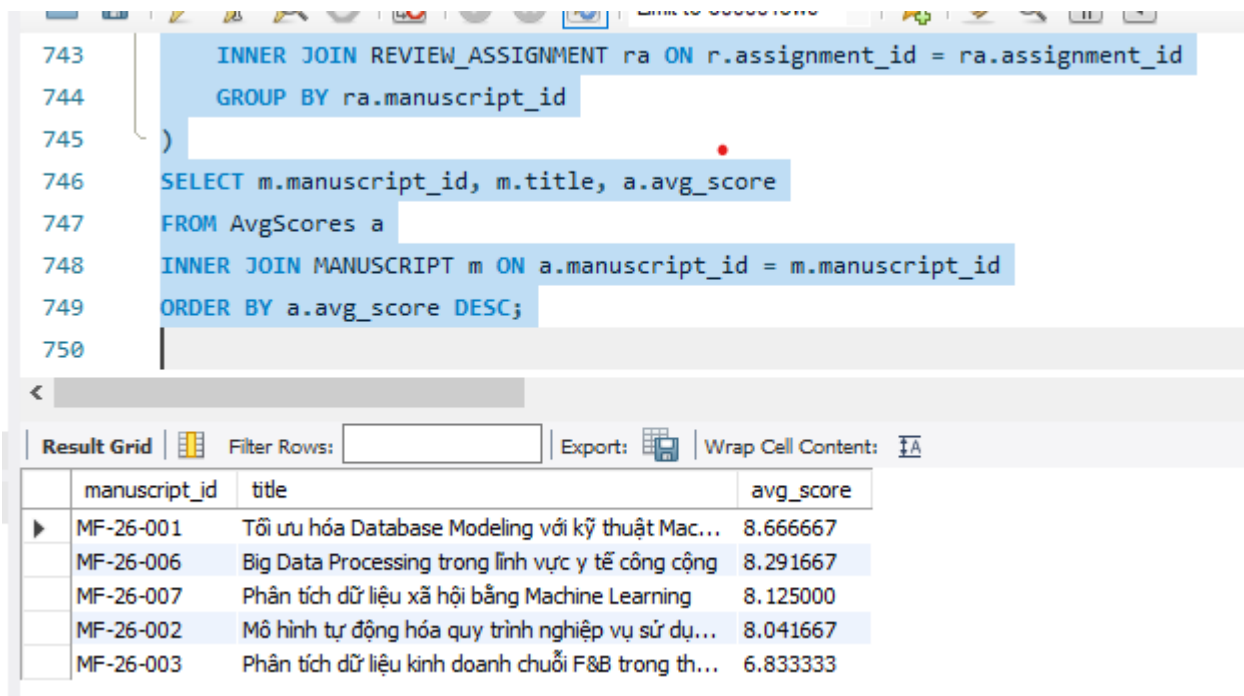
WITH AvgScores AS (

```

SELECT ra.manuscript_id, AVG(rs.score) AS avg_score
FROM REVIEW_SCORE rs
INNER JOIN REVIEW r ON rs.review_id = r.review_id
INNER JOIN REVIEW_ASSIGNMENT ra ON r.assignment_id = ra.assignment_id
GROUP BY ra.manuscript_id
)
SELECT m.manuscript_id, m.title, a.avg_score
FROM AvgScores a
INNER JOIN MANUSCRIPT m ON a.manuscript_id = m.manuscript_id
ORDER BY a.avg_score DESC;

```

Kết quả thực tế:



```

743     INNER JOIN REVIEW_ASSIGNMENT ra ON r.assignment_id = ra.assignment_id
744     GROUP BY ra.manuscript_id
745 )
746 SELECT m.manuscript_id, m.title, a.avg_score
747 FROM AvgScores a
748 INNER JOIN MANUSCRIPT m ON a.manuscript_id = m.manuscript_id
749 ORDER BY a.avg_score DESC;
750

```

manuscript_id	title	avg_score
MF-26-001	Tối ưu hóa Database Modeling với kỹ thuật Mac...	8.666667
MF-26-006	Big Data Processing trong lĩnh vực y tế công cộng	8.291667
MF-26-007	Phân tích dữ liệu xã hội bằng Machine Learning	8.125000
MF-26-002	Mô hình tự động hóa quy trình nghiệp vụ sử dụ...	8.041667
MF-26-003	Phân tích dữ liệu kinh doanh chuỗi F&B trong th...	6.833333

Hình 7: Kết quả thực tế Q06: CTE

4.7. Q07: Report theo thời gian / Date Functions

Trường	Nội dung
Business question	Lưu lượng nộp bài về tòa soạn biến động như thế nào qua từng tháng trong năm 2026?
Input/Parameter	Năm tiếp nhận: 2026
SQL technique	Sử dụng hàm MONTH() kết hợp GROUP BY theo tháng và sắp xếp theo trình tự thời gian (ORDER BY ... ASC)
Expected result	receive_month (tháng nhận bài 1-12) và total_received (số lượng bài nhận được)
Performance note	Truy vấn thực hiện Aggregate nên cần Index bao phủ cột received_date để CSDL quét nhanh hơn.

Bảng 25: Mô tả chi tiết truy vấn Q07

SQL

```
SELECT MONTH(received_date) AS receive_month, COUNT(manuscript_id) AS
total_received
```

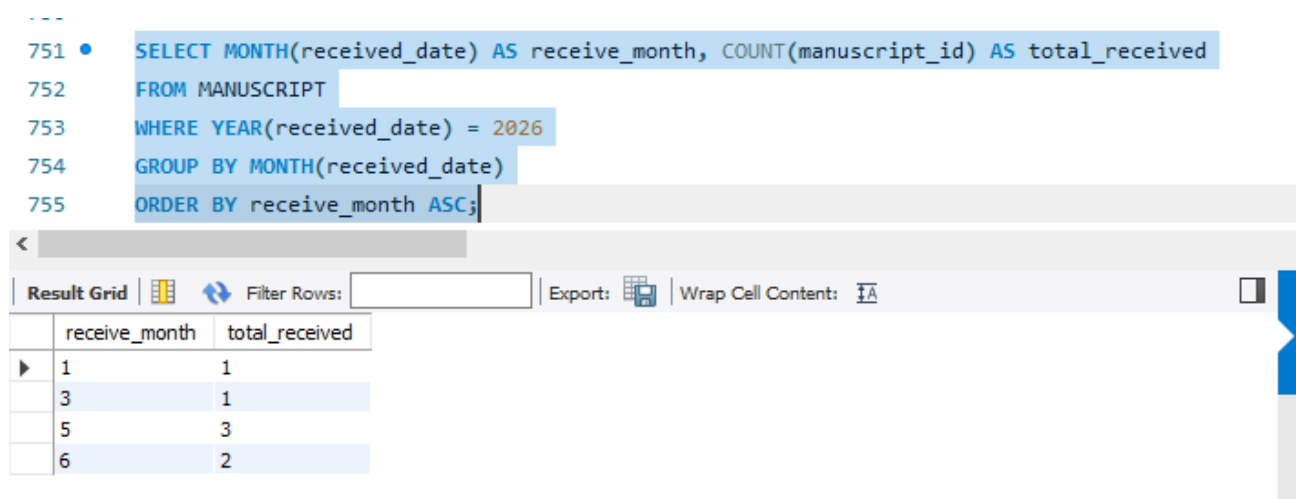
```
FROM MANUSCRIPT
```

```
WHERE YEAR(received_date) = 2026
```

```
GROUP BY MONTH(received_date)
```

```
ORDER BY receive_month ASC;
```

Kết quả thực tế:



```

751 • SELECT MONTH(received_date) AS receive_month, COUNT(manuscript_id) AS total_received
752 FROM MANUSCRIPT
753 WHERE YEAR(received_date) = 2026
754 GROUP BY MONTH(received_date)
755 ORDER BY receive_month ASC;

```

receive_month	total_received
1	1
3	1
5	3
6	2

4.8. Q08: Query dùng View

Trường	Nội dung
Business question	Tổng hợp nhanh các bài đang trong quá trình phản biện (under_review) và điểm trung bình hiện tại để đánh giá khả năng thông qua.
Input/Parameter	Trạng thái bài viết: 'under_review'
SQL technique	Truy vấn trực tiếp (SELECT) từ View đã được định nghĩa sẵn (vw_manuscript_review_summary) để tận dụng logic gom nhóm
Expected result	manuscript_id, title, tổng số chuyên gia đã gán, tổng số chuyên gia đã phản hồi, và average_score
Performance note	Lệnh gọi View che giấu độ phức tạp của các phép LEFT JOIN và hàm AVG(). Hiệu năng phụ thuộc vào cách MySQL xử lý View (Merge/Tempable).

Bảng 26: Mô tả chi tiết truy vấn Q08

SQL

SELECT

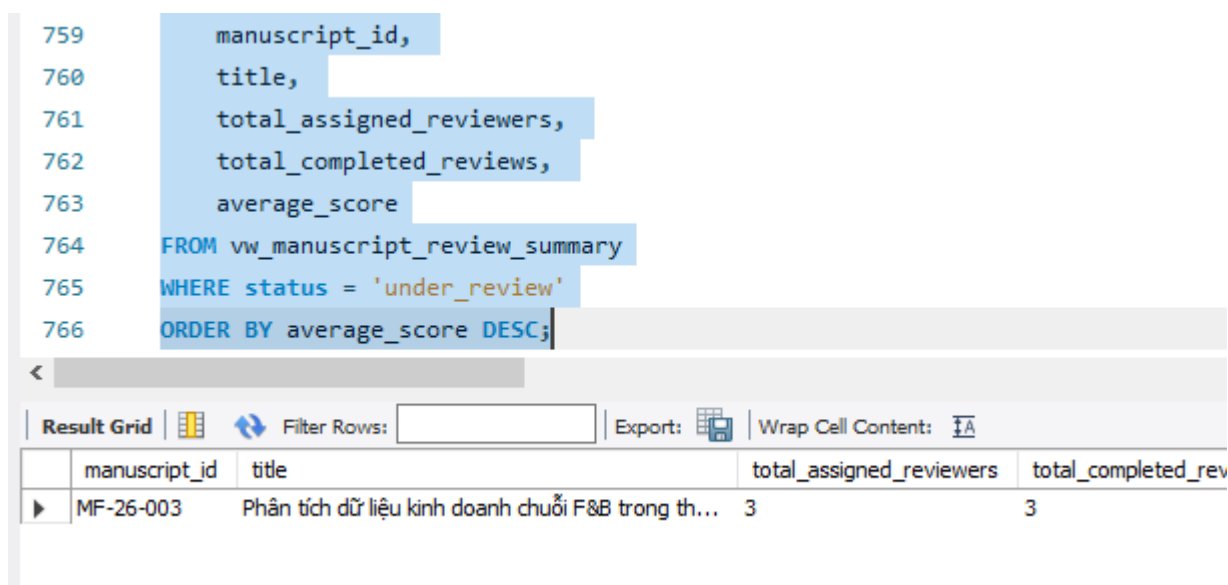
```
manuscript_id,  
title,  
total_assigned_reviewers,  
total_completed_reviews,  
average_score
```

```
FROM vw_manuscript_review_summary
```

```
WHERE status = 'under_review'
```

```
ORDER BY average_score DESC;
```

Kết quả thực tế:



Hình 9: Kết quả thực tế QUERY dùng VIEW

4.9. Kết luận chương

Chương 4 đã trình bày 8 truy vấn nghiệp vụ đáp ứng các yêu cầu khai thác dữ liệu của hệ thống DataFrontiers Review Quarterly:

Mã	Kỹ thuật	Câu hỏi nghiệp vụ
Q01	Filter + Order	Danh sách bài nộp trong năm 2026
Q02	INNER JOIN (4 bảng)	Tác giả và bài đã vượt qua phản biện
Q03	LEFT JOIN + IS NULL	Reviewer chưa từng nhận bài
Q04	GROUP BY + HAVING	Số bài phản biện của mỗi reviewer
Q05	Correlated Subquery	Bài bị chậm tiến độ phản biện
Q06	CTE	Điểm trung bình và xếp hạng bài
Q07	Date Functions	Thống kê số bài theo tháng
Q08	View	Tổng quan bài đang phản biện

Bảng 27: Tổng kết 8 truy vấn

Các truy vấn này đã được thực thi thành công trên dữ liệu mẫu và sẵn sàng sử dụng trong môi trường vận hành thực tế.

CHƯƠNG 5: VIEWS VÀ STORED PROGRAMS

Chương này trình bày các đối tượng cơ sở dữ liệu nâng cao được xây dựng để đóng gói logic nghiệp vụ, tái sử dụng mã nguồn và tăng cường hiệu quả khai thác dữ liệu. Hệ thống triển khai 3 Views, 1 Function và 2 Stored Procedures phục vụ các nghiệp vụ quan trọng của **DataFrontiers Review Quarterly**.

5.1. Views

View là các bảng ảo đóng gói các truy vấn phức tạp, giúp đơn giản hóa việc truy xuất dữ liệu và đảm bảo tính nhất quán khi báo cáo.

5.1.1. View 1: vw_manuscript_review_summary

Trường	Nội dung
Tên View	vw_manuscript_review_summary
Mục đích	Cung cấp tổng quan quá trình phản biện của từng bài, bao gồm số lượng reviewer đã phân công, số lượng phản hồi đã nhận, điểm trung bình và xếp hạng chất lượng. Cột quality_rating được tính tự động dựa trên số lượng review đã hoàn thành và điểm trung bình: Excellent: ≥ 3 reviews và điểm ≥ 8.0, Good: ≥ 3 reviews và điểm ≥ 6.5, Fair: ≥ 3 reviews và điểm < 6.5, Pending: chưa đủ 3 reviews
Sử dụng	Theo dõi tiến độ phản biện, lọc bài under_review, đánh giá chất lượng bài dựa trên điểm trung bình.
Kỹ thuật	Sử dụng LEFT JOIN và GROUP BY để tổng hợp dữ liệu từ nhiều bảng. Điều kiện WHERE ra.assignment_id IS NOT NULL đảm bảo View chỉ hiển thị các bài đã có ít nhất một phân công phản biện, phù hợp với mục đích theo dõi tiến độ phản biện.

Bảng 28: View 1: vw_manuscript_review_summary

SQL:

```

CREATE OR REPLACE VIEW vw_manuscript_review_summary AS
SELECT
    m.manuscript_id,
    m.title,
    m.status,
    m.issue_id,
    COUNT(DISTINCT ra.assignment_id) AS total_assigned_reviewers,
    COUNT(DISTINCT r.review_id) AS total_completed_reviews,
    IFNULL(ROUND(AVG(rs.score), 2), 0.00) AS average_score,
    CASE
        WHEN COUNT(DISTINCT r.review_id) >= 3 AND IFNULL(AVG(rs.score), 0) >=
8.0 THEN 'Excellent'
        WHEN COUNT(DISTINCT r.review_id) >= 3 AND IFNULL(AVG(rs.score), 0) >=
6.5 THEN 'Good'
        WHEN COUNT(DISTINCT r.review_id) >= 3 THEN 'Fair'
        ELSE 'Pending'
    END AS quality_rating
FROM MANUSCRIPT m
LEFT JOIN REVIEW_ASSIGNMENT ra ON m.manuscript_id = ra.manuscript_id
LEFT JOIN REVIEW r ON ra.assignment_id = r.assignment_id
LEFT JOIN REVIEW_SCORE rs ON r.review_id = rs.review_id
WHERE ra.assignment_id IS NOT NULL
GROUP BY m.manuscript_id, m.title, m.status, m.issue_id;

```

Test Query:

-- Danh sách bài đang phản biện có điểm trung bình

```

SELECT manuscript_id, title, total_assigned_reviewers, total_completed_reviews,
average_score, quality_rating
FROM vw_manuscript_review_summary
WHERE status = 'under_review'

```

ORDER BY average_score DESC;

5.1.2. View 2: vw_issue_table_of_content

Trường	Nội dung
Tên View	vw_issue_table_of_contents
Mục đích	Xuất bảng mục lục (Table of Contents) cho các số báo đã xuất bản, hiển thị tác giả chính và thông tin trang.
Sử dụng	Tạo danh mục bài báo cho từng số phát hành, phục vụ tra cứu cho độc giả và ban biên tập.
Kỹ thuật	INNER JOIN giữa 5 bảng, lọc issue_status = 'published' và author_order = 1 (tác giả chính).

Bảng 29: View 2: vw_issue_table_of_contents

SQL:

CREATE OR REPLACE VIEW vw_issue_table_of_contents AS

SELECT

i.issue_id,
i.season,
i.year,
i.volume,
i.issue_number,
i.publication_date,
m.order_in_issue,
m.start_page,
m.page_count,
m.title,
p.full_name AS primary_author,

```

    p.email AS author_email
FROM ISSUE i
INNER JOIN MANUSCRIPT m ON i.issue_id = m.issue_id
INNER JOIN MANUSCRIPT_AUTHOR ma ON m.manuscript_id = ma.manuscript_id
INNER JOIN PERSON p ON ma.person_id = p.person_id
WHERE i.issue_status = 'published'

    AND ma.author_order = 1
ORDER BY i.year DESC, i.issue_number ASC, m.order_in_issue ASC;

```

Test Query:

-- Mục lục số báo Spring 2026

```

SELECT * FROM vw_issue_table_of_contents
WHERE season = 'Spring' AND year = 2026
ORDER BY order_in_issue;

```

5.1.3. View 3: vw_reviewer_performance

Trường	Nội dung
Tên View	vw_reviewer_performance
Mục đích	Đánh giá hiệu suất của từng reviewer: số bài đã nhận, số bài đã phản hồi, tỷ lệ hoàn thành, điểm trung bình và thời gian phản hồi.
Sử dụng	Quản lý chất lượng reviewer, thống kê cho ban biên tập, xác định reviewer tích cực và có chất lượng cao.
Kỹ thuật	Sử dụng LEFT JOIN và HAVING để lọc reviewer đã nhận bài.

Bảng 30: View 3: vw_reviewer_performance

SQL:

```

CREATE OR REPLACE VIEW vw_reviewer_performance AS
SELECT

```

```

p.person_id,
p.full_name,
p.email,
p.affiliation,
COUNT(DISTINCT ra.assignment_id) AS total_assigned,
COUNT(DISTINCT r.review_id) AS total_completed,
ROUND(
    IFNULL(COUNT(DISTINCT r.review_id) * 100.0 / NULLIF(COUNT(DISTINCT
ra.assignment_id), 0), 0),
    2
) AS completion_rate_percent,
IFNULL(ROUND(AVG(rs.score), 2), 0.00) AS avg_score_given,
ROUND(
    IFNULL(AVG(DATEDIFF(r.response_date, ra.date_sent)), 0),
    0
) AS avg_response_days
FROM REVIEWER rev
INNER JOIN PERSON p ON rev.person_id = p.person_id
LEFT JOIN REVIEW_ASSIGNMENT ra ON rev.person_id = raReviewer_id
LEFT JOIN REVIEW r ON ra.assignment_id = r.assignment_id
LEFT JOIN REVIEW_SCORE rs ON r.review_id = rs.review_id
GROUP BY p.person_id, p.full_name, p.email, p.affiliation
HAVING total_assigned > 0
ORDER BY avg_score_given DESC, completion_rate_percent DESC;

```

Test Query:

-- Top 3 Reviewer có điểm trung bình cao nhất

```

SELECT full_name, total_assigned, total_completed, completion_rate_percent,
avg_score_given, avg_response_days

```

```

FROM vw_reviewer_performance
ORDER BY avg_score_given DESC
LIMIT 3;

```

5.2. Stored Programs

5.2.1. Function: fn_get_avg_score

Trường	Nội dung
Tên Function	fn_get_avg_score
Kiểu trả về	DECIMAL(4,2)
Mục đích	Tính điểm trung bình của một bài dựa trên tất cả các điểm đánh giá từ các reviewer.
Sử dụng	Gọi trong các truy vấn SELECT để hiển thị điểm trung bình của bài, có thể sử dụng trong View hoặc báo cáo.
Xử lý NULL	Sử dụng COALESCE để trả về 0.00 nếu bài chưa có điểm.
Đặc tính	DETERMINISTIC và READS SQL DATA để tối ưu hóa.

Bảng 31: Function: fn_get_avg_score

SQL:

```

CREATE FUNCTION fn_get_avg_score(p_manuscript_id VARCHAR(50))
RETURNS DECIMAL(4,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_avg DECIMAL(4,2);

```

```

SELECT AVG(rs.score) INTO v_avg
FROM REVIEW_SCORE rs
INNER JOIN REVIEW r ON rs.review_id = r.review_id
INNER JOIN REVIEW_ASSIGNMENT ra ON r.assignment_id = ra.assignment_id
WHERE ra.manuscript_id = p_manuscript_id;

```

```

RETURN COALESCE(v_avg, 0.00);

```

END;

Test Query:

```

SELECT
    manuscript_id,
    title,
    fn_get_avg_score(manuscript_id) AS avg_score
FROM MANUSCRIPT
WHERE manuscript_id = 'MF-26-001';

```

Kết quả thực thi:

767 • SELECT
768 manuscript_id,
769 title,
770 fn_get_avg_score(manuscript_id) AS avg_score
771 FROM MANUSCRIPT
772 WHERE manuscript_id = 'MF-26-001';

manuscript_id	title	avg_score
MF-26-001	Tối ưu hóa Database Modeling với kỹ thuật Mac...	8.67

Hình 10: Kết quả thực thi test query `fn_get_avg_score`

5.2.2. Procedure: `sp_assign_reviewer`

Trường	Nội dung
Tên Procedure	sp_assign_reviewer
Tham số	p_manuscript_id VARCHAR(50), p_reviewer_id INT, p_deadline TIMESTAMP
Mục đích	Phân công một reviewer cho một bài nộp, đồng thời tự động cập nhật trạng thái bài khi đủ 3 reviewer.
Nghiệp vụ	Đảm bảo chỉ gán khi bài ở trạng thái received hoặc under_review, không trùng reviewer, không tự phản biện.
Ràng buộc	FOR UPDATE được sử dụng để khóa row MANUSCRIPT trong transaction, ngăn chặn tình huống hai biên tập viên cùng lúc phân công reviewer cho cùng một bài (race condition), đảm bảo tính nhất quán của dữ liệu.
Xử lý lỗi	EXIT HANDLER bắt lỗi và ghi log vào PROCEDURE_AUDIT.

Bảng 32: Procedure: sp_assign_reviewer

Quy trình xử lý:

1. Kiểm tra bài tồn tại
2. Kiểm tra reviewer tồn tại
3. Khóa row MANUSCRIPT và lấy trạng thái
4. Kiểm tra trạng thái cho phép phân công
5. Kiểm tra trùng phân công
6. Kiểm tra tác giả tự phản biện
7. Insert phân công
8. Cập nhật trạng thái sang under_review nếu đủ 3 reviewer

SQL:

```
CREATE PROCEDURE sp_assign_reviewer(
    IN p_manuscript_id VARCHAR(50),
    IN p_reviewer_id INT,
    IN p_deadline TIMESTAMP
```

```

)
BEGIN
    DECLARE v_current_reviewers INT;
    DECLARE v_status VARCHAR(20);
    DECLARE v_error_msg TEXT DEFAULT "";

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        GET DIAGNOSTICS CONDITION 1
            @sqlstate = RETURNED_SQLSTATE,
            @errno = MYSQL_ERRNO,
            @text = MESSAGE_TEXT;
        SET v_error_msg = CONCAT('SQLSTATE ', @sqlstate, ', Error ', @errno, ': ', @text);
        INSERT INTO PROCEDURE_AUDIT (procedure_name, manuscript_id, reviewer_id,
            error_code, error_message)
            VALUES ('sp_assign_reviewer', p_manuscript_id, p_reviewer_id, @sqlstate,
            v_error_msg);
        ROLLBACK;
        RESIGNAL SET MESSAGE_TEXT = v_error_msg;
    END;

    START TRANSACTION;

    IF NOT EXISTS (SELECT 1 FROM MANUSCRIPT WHERE manuscript_id =
        p_manuscript_id) THEN
        SET v_error_msg = 'Lỗi: Bài nộp không tồn tại.';
        INSERT INTO PROCEDURE_AUDIT (procedure_name, manuscript_id, reviewer_id,
            error_code, error_message)

```

```
VALUES ('sp_assign_reviewer', p_manuscript_id, p_reviewer_id, 'BUSINESS',  
v_error_msg);
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_error_msg;
```

```
END IF;
```

```
IF NOT EXISTS (SELECT 1 FROM REVIEWER WHERE person_id = p_reviewer_id)  
THEN
```

```
SET v_error_msg = 'Lỗi: Reviewer không tồn tại.';
```

```
INSERT INTO PROCEDURE_AUDIT (procedure_name, manuscript_id, reviewer_id,  
error_code, error_message)
```

```
VALUES ('sp_assign_reviewer', p_manuscript_id, p_reviewer_id, 'BUSINESS',  
v_error_msg);
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_error_msg;
```

```
END IF;
```

```
SELECT status INTO v_status FROM MANUSCRIPT WHERE manuscript_id =  
p_manuscript_id FOR UPDATE;
```

```
IF v_status IN ('accepted', 'rejected', 'scheduled', 'published') THEN
```

```
SET v_error_msg = CONCAT('Lỗi: Bài đang ở trạng thái ', v_status, ', không thể phân  
công thêm reviewer.');
```

```
INSERT INTO PROCEDURE_AUDIT (procedure_name, manuscript_id, reviewer_id,  
error_code, error_message)
```

```
VALUES ('sp_assign_reviewer', p_manuscript_id, p_reviewer_id, 'BUSINESS',  
v_error_msg);
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_error_msg;
```

```
END IF;
```

```
IF EXISTS (SELECT 1 FROM REVIEW_ASSIGNMENT WHERE manuscript_id =  
p_manuscript_id AND reviewer_id = p_reviewer_id) THEN
```

```
SET v_error_msg = 'Lỗi: Reviewer đã được phân công cho bài này.';
```

```
INSERT INTO PROCEDURE_AUDIT (procedure_name, manuscript_id, reviewer_id,  
error_code, error_message)
```

```
VALUES ('sp_assign_reviewer', p_manuscript_id, p_reviewer_id, 'BUSINESS',  
v_error_msg);
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_error_msg;
```

```
END IF;
```

```
IF EXISTS (SELECT 1 FROM MANUSCRIPT_AUTHOR WHERE manuscript_id =  
p_manuscript_id AND person_id = p_reviewer_id) THEN
```

```
SET v_error_msg = 'Lỗi: Tác giả không được phản biện bài của chính mình.';
```

```
INSERT INTO PROCEDURE_AUDIT (procedure_name, manuscript_id, reviewer_id,  
error_code, error_message)
```

```
VALUES ('sp_assign_reviewer', p_manuscript_id, p_reviewer_id, 'BUSINESS',  
v_error_msg);
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_error_msg;
```

```
END IF;
```

```
INSERT INTO REVIEW_ASSIGNMENT (manuscript_id, reviewer_id, date_sent,  
deadline)
```

```
VALUES (p_manuscript_id, p_reviewer_id, CURRENT_TIMESTAMP, p_deadline);
```

```
SELECT COUNT(*) INTO v_current_reviewers FROM REVIEW_ASSIGNMENT  
WHERE manuscript_id = p_manuscript_id;
```

```
IF v_current_reviewers >= 3 AND v_status = 'received' THEN
```

```
UPDATE MANUSCRIPT SET status = 'under_review' WHERE manuscript_id =  
p_manuscript_id;
```

```
END IF;
```

```
INSERT INTO PROCEDURE_AUDIT (procedure_name, manuscript_id, reviewer_id,  
error_code, error_message)
```

```
VALUES ('sp_assign_reviewer', p_manuscript_id, p_reviewer_id, 'SUCCESS', 'Phân công thành công.');
```

```
COMMIT;
```

```
END;
```

Test Case:

```
-- Gán reviewer cho bài MF-26-004 (đang received)
```

```
CALL sp_assign_reviewer('MF-26-004', 12, '2026-07-15 23:59:59');
```

```
CALL sp_assign_reviewer('MF-26-004', 5, '2026-07-15 23:59:59');
```

```
CALL sp_assign_reviewer('MF-26-004', 8, '2026-07-15 23:59:59');
```

```
-- Kiểm tra status: sẽ chuyển thành 'under_review'
```

```
SELECT manuscript_id, status FROM MANUSCRIPT WHERE manuscript_id = 'MF-26-004';
```

Kết quả thực thi:

The screenshot displays a SQL execution window with the following code and results:

```
774 -- Gán reviewer cho bài MF-26-004 (đang received)
775 • CALL sp_assign_reviewer('MF-26-004', 12, '2026-07-15 23:59:59');
776 • CALL sp_assign_reviewer('MF-26-004', 5, '2026-07-15 23:59:59');
777 • CALL sp_assign_reviewer('MF-26-004', 8, '2026-07-15 23:59:59');
778
779 -- Kiểm tra status: sẽ chuyển thành 'under_review'
780 • SELECT manuscript_id, status FROM MANUSCRIPT WHERE manuscript_id = 'MF-26-004';
```

Below the code, a toolbar includes options like 'Result Grid', 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. The 'Result Grid' is active, showing the following data:

manuscript_id	status
MF-26-004	under_review
* NULL	NULL

Hình 11: Kết quả thực thi `sp_assign_reviewer`

5.2.3. Procedure: `sp_publish_issue`

Trường	Nội dung
Tên Procedure	sp_publish_issue
Tham số	p_issue_id INT
Mục đích	Xuất bản một số báo: cập nhật issue_status = 'published', gán publication_date = CURRENT_TIMESTAMP, và chuyển trạng thái các bài trong số đó thành published.
Nhiệm vụ	Đáp ứng yêu cầu 16: Khi số phát hành được xuất bản, hệ thống lưu ngày in/phát hành và đổi trạng thái tất cả bài thành published.
Ràng buộc	Chỉ xuất bản số báo chưa được xuất bản (draft).

Bảng 33: Procedure: sp_publish_issue

SQL:

```

CREATE PROCEDURE sp_publish_issue(IN p_issue_id INT)
BEGIN
    DECLARE v_issue_status VARCHAR(20);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL SET MESSAGE_TEXT = 'Lỗi khi xuất bản số báo.';
    END;

    START TRANSACTION;

    SELECT issue_status INTO v_issue_status FROM ISSUE WHERE issue_id = p_issue_id
    FOR UPDATE;

```

```
IF v_issue_status IS NULL THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Lỗi: Số báo không tồn tại.';
```

```
END IF;
```

```
IF v_issue_status = 'published' THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Lỗi: Số báo đã được xuất  
bản.';
```

```
END IF;
```

```
UPDATE ISSUE SET issue_status = 'published', publication_date =  
CURRENT_TIMESTAMP WHERE issue_id = p_issue_id;
```

```
UPDATE MANUSCRIPT SET status = 'published' WHERE issue_id = p_issue_id AND  
status IN ('scheduled', 'accepted');
```

```
COMMIT;
```

```
END;
```

Test Case:

```
-- Xuất bản Issue 2 (Summer 2026)
```

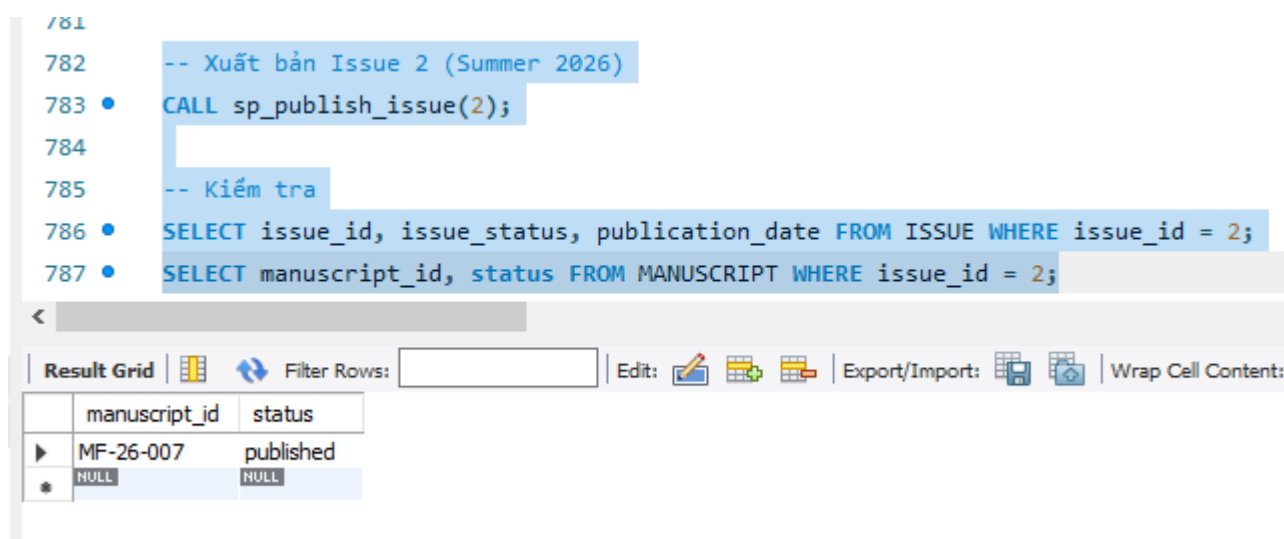
```
CALL sp_publish_issue(2);
```

```
-- Kiểm tra
```

```
SELECT issue_id, issue_status, publication_date FROM ISSUE WHERE issue_id = 2;
```

```
SELECT manuscript_id, status FROM MANUSCRIPT WHERE issue_id = 2;
```

Kết quả thực thi:



Hình 12: Kết quả thực thi `sp_publish_issue`

5.3. Tóm tắt các Stored Programs

STT	Tên	Loại	Tham số	Mục đích
1	fn_get_avg_score	Function	p_manuscript_id	Tính điểm trung bình của bài
2	sp_assign_reviewer	Procedure	p_manuscript_id, p_reviewer_id, p_deadline	Phân công reviewer, cập nhật trạng thái
3	sp_publish_issue	Procedure	p_issue_id	Xuất bản số báo, cập nhật trạng thái bài

Bảng 34: Tóm tắt các Stored Programs

5.4. Kết luận chương

Chương 5 đã trình bày các đối tượng cơ sở dữ liệu nâng cao được triển khai trong hệ thống DataFrontiers Review Quarterly:

- 3 Views giúp đơn giản hóa báo cáo và tái sử dụng logic:
 - vw_manuscript_review_summary: Tổng quan tiến độ và chất lượng bài
 - vw_issue_table_of_contents: Mục lục số báo đã xuất bản
 - vw_reviewer_performance: Đánh giá hiệu suất reviewer

- 1 Function đóng gói logic tính toán:
 - fn_get_avg_score: Tính điểm trung bình của bài
- 2 Stored Procedures thực thi các nghiệp vụ phức tạp với transaction và error handling:
 - sp_assign_reviewer: Phân công reviewer với kiểm soát chặt chẽ
 - sp_publish_issue: Xuất bản số báo và cập nhật trạng thái

Các đối tượng này đã được kiểm thử với dữ liệu mẫu và hoạt động đúng như mong đợi, đóng góp vào việc xây dựng một hệ thống vận hành ổn định, dễ bảo trì và mở rộng.

CHƯƠNG 6: TRIGGERS VÀ EVENTS

Chương này trình bày các Triggers và Event Scheduler được xây dựng để tự động hóa các tác vụ kiểm soát chất lượng dữ liệu, ghi nhận lịch sử và bảo vệ tính toàn vẹn nghiệp vụ của hệ thống DataFrontiers Review Quarterly.

6.1. Tổng quan Triggers

Trigger là các stored programs được thực thi tự động khi một sự kiện xảy ra trên bảng (INSERT, UPDATE, DELETE). Hệ thống triển khai 7 triggers để:

- Bảo vệ logic nghiệp vụ: Ngăn chặn xung đột dữ liệu (đề trang, sai ngày tháng).
- Ghi nhận lịch sử: Tự động ghi log khi trạng thái bài thay đổi.
- Đảm bảo tính toàn vẹn dữ liệu: Kiểm soát số lượng điểm đánh giá và mối quan hệ thời gian.

6.1.1. Trigger 1: Trg_Prevent_Page_Overlap_Insert

Trường	Nội dung
Tên Trigger	Trg_Prevent_Page_Overlap_Insert
Sự kiện kích hoạt	BEFORE INSERT ON MANUSCRIPT
Mục đích	Ngăn chặn việc xếp trang đề lên nhau khi thêm mới một bài vào số phát hành.
Nghệ vụ	Đảm bảo không có hai bài có khoảng trang trùng nhau trong cùng một ISSUE (BR12).
Logic	Kiểm tra xem khoảng trang mới (start_page, start_page + page_count - 1) có giao với bất kỳ bài nào trong cùng ISSUE hay không. Nếu có, phát sinh lỗi.

Bảng 35: Trigger 1: Trg_Prevent_Page_Overlap_Insert

SQL:

```
CREATE TRIGGER Trg_Prevent_Page_Overlap_Insert  
BEFORE INSERT ON MANUSCRIPT
```

```

FOR EACH ROW
BEGIN
    DECLARE overlap_count INT;

    IF NEW.issue_id IS NOT NULL AND NEW.start_page IS NOT NULL AND
    NEW.page_count IS NOT NULL THEN

        SELECT COUNT(*) INTO overlap_count
        FROM MANUSCRIPT
        WHERE issue_id = NEW.issue_id
        AND (
            (NEW.start_page BETWEEN start_page AND (start_page + page_count - 1))
            OR ((NEW.start_page + NEW.page_count - 1) BETWEEN start_page AND
            (start_page + page_count - 1))
            OR (start_page BETWEEN NEW.start_page AND (NEW.start_page +
            NEW.page_count - 1))
        );

        IF overlap_count > 0 THEN

            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Lỗi Nghiệp Vụ: Xung đột
            dài số trang, khoảng trang bị đề lên bài viết khác.';

        END IF;

    END IF;
END;

```

Test Case:

```

-- Cố tình insert bài có trang đề lên bài MF-26-001 (start_page=10, page_count=12)
INSERT INTO MANUSCRIPT (manuscript_id, title, received_date, status, page_count,
order_in_issue, start_page, editor_id, issue_id)
VALUES ('MF-FAIL-001','Bài đề trang', CURRENT_TIMESTAMP, 'scheduled', 5, 3, 12,
1, 1);
-- Dự kiến: Lỗi "Xung đột dài số trang"

```

6.1.2. Trigger 2: Trg_Prevent_Page_Overlap_Update

Trường	Nội dung
Tên Trigger	Trg_Prevent_Page_Overlap_Update
Sự kiện kích hoạt	BEFORE UPDATE ON MANUSCRIPT
Mục đích	Ngăn chặn việc cập nhật gây xung đột dài số trang.
Nhiệm vụ	Tương tự Trigger 1 nhưng áp dụng cho thao tác UPDATE (loại trừ chính bài đang cập nhật).

Bảng 36: Trigger 2: Trg_Prevent_Page_Overlap_Update

SQL:

```

CREATE TRIGGER Trg_Prevent_Page_Overlap_Update
BEFORE UPDATE ON MANUSCRIPT
FOR EACH ROW
BEGIN
    DECLARE overlap_count INT;

    IF NEW.issue_id IS NOT NULL AND NEW.start_page IS NOT NULL AND
NEW.page_count IS NOT NULL THEN

        SELECT COUNT(*) INTO overlap_count
        FROM MANUSCRIPT
        WHERE issue_id = NEW.issue_id
        AND manuscript_id != NEW.manuscript_id
        AND (
            (NEW.start_page BETWEEN start_page AND (start_page + page_count - 1))
            OR ((NEW.start_page + NEW.page_count - 1) BETWEEN start_page AND
(start_page + page_count - 1))
            OR (start_page BETWEEN NEW.start_page AND (NEW.start_page +
NEW.page_count - 1))
        );

```

```

IF overlap_count > 0 THEN

    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Lỗi Nghiệp Vụ: Xung đột
dài số trang, khoảng trang bị đề lên bài viết khác.';

    END IF;

END IF;

END;

```

Test Case:

-- Cập nhật bài MF-26-002 sang khoảng trang gây xung đột

```
UPDATE MANUSCRIPT SET start_page = 11 WHERE manuscript_id = 'MF-26-002';
```

-- Dự kiến: Lỗi "Xung đột dài số trang"

6.1.3. Trigger 3: trg_check_publication_date

Trường	Nội dung
Tên Trigger	trg_check_publication_date
Sự kiện kích hoạt	BEFORE UPDATE ON ISSUE
Mục đích	Đảm bảo số báo không thể xuất bản trước ngày nhận bài đầu tiên.
Nghiệp vụ	Bảo vệ logic thời gian: publication_date phải >= received_date nhỏ nhất của các bài trong số đó.

Bảng 37: Trigger 3: trg_check_publication_date

SQL:

```

CREATE TRIGGER trg_check_publication_date
BEFORE UPDATE ON ISSUE
FOR EACH ROW
BEGIN
    DECLARE min_received TIMESTAMP;

    IF NEW.publication_date IS NOT NULL THEN

```

```
SELECT MIN(received_date) INTO min_received FROM MANUSCRIPT WHERE
issue_id = NEW.issue_id;
```

```
IF min_received IS NOT NULL AND NEW.publication_date < min_received THEN
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Lỗi Logic: Số báo không
thể xuất bản trước khi nhận bài đầu tiên.';
```

```
END IF;
```

```
END IF;
```

```
END;
```

Test Case:

```
-- Test Case: Cố tình đặt publication_date trước ngày nhận bài đầu tiên của Issue 1
```

```
UPDATE ISSUE SET publication_date = '2025-01-01', issue_status = 'published' WHERE
issue_id = 1;
```

```
-- Dự kiến: Lỗi "Số báo không thể xuất bản trước khi nhận bài đầu tiên"
```

6.1.4. Trigger 4: trg_audit_manuscript_status

Trường	Nội dung
Tên Trigger	trg_audit_manuscript_status
Sự kiện kích hoạt	AFTER UPDATE ON MANUSCRIPT
Mục đích	Tự động ghi lại lịch sử thay đổi trạng thái của bài vào bảng MANUSCRIPT_AUDIT.
Nghiệp vụ	Phục vụ kiểm tra, truy vết và phân tích tiến độ xử lý bài.

Bảng 38: Trigger 4: trg_audit_manuscript_status

SQL:

```
CREATE TRIGGER trg_audit_manuscript_status
```

```
AFTER UPDATE ON MANUSCRIPT
```

```
FOR EACH ROW
```

```
BEGIN
```

```

IF OLD.status != NEW.status THEN

    INSERT INTO MANUSCRIPT_AUDIT (manuscript_id, old_status, new_status,
changed_date)

        VALUES (NEW.manuscript_id, OLD.status, NEW.status,
CURRENT_TIMESTAMP);

END IF;

END;

```

Test Case:

-- Cập nhật trạng thái bài

```
UPDATE MANUSCRIPT SET status = 'under_review' WHERE manuscript_id = 'MF-26-004';
```

-- Kiểm tra bảng audit

```
SELECT * FROM MANUSCRIPT_AUDIT WHERE manuscript_id = 'MF-26-004'
ORDER BY changed_date DESC;
```

-- Dự kiến: Có bản ghi mới với old_status='received', new_status='under_review'

6.1.5. Trigger 5: trg_check_review_scores_insert

Trường	Nội dung
Tên Trigger	trg_check_review_scores_insert
Sự kiện kích hoạt	AFTER INSERT ON REVIEW_SCORE
Mục đích	Đảm bảo mỗi review không có quá 4 điểm cho các tiêu chí.
Nhiệm vụ	Bảo vệ ràng buộc: Mỗi review chỉ có đúng 4 tiêu chí (BR09).

Bảng 39: Trigger 5: trg_check_review_scores_insert

SQL:

```

CREATE TRIGGER trg_check_review_scores_insert
AFTER INSERT ON REVIEW_SCORE

```

FOR EACH ROW

BEGIN

DECLARE score_count INT;

SELECT COUNT(*) INTO score_count FROM REVIEW_SCORE WHERE review_id = NEW.review_id;

IF score_count > 4 THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Lỗi: Một review không được có quá 4 tiêu chí điểm.';

END IF;

END;

Test Case:

-- Thử thêm điểm thứ 5 cho review_id = 1

INSERT INTO REVIEW_SCORE (review_id, criterion_id, score) VALUES (1, 1, 9.0);

-- Dự kiến: Lỗi "Một review không được có quá 4 tiêu chí điểm"

6.1.6. Trigger 6: trg_check_review_scores_delete

Trường	Nội dung
Tên Trigger	trg_check_review_scores_delete
Sự kiện kích hoạt	AFTER DELETE ON REVIEW_SCORE
Mục đích	Đảm bảo mỗi review vẫn có đủ 4 điểm sau khi xóa.
Nhiệm vụ	Cho phép sửa điểm nhưng không được phép xóa điểm dưới 4 tiêu chí. Nếu sau khi xóa, số điểm của một review nhỏ hơn 4, trigger sẽ phát sinh lỗi để ngăn chặn việc thiếu tiêu chí đánh giá.

Bảng 40: Trigger 6: trg_check_review_scores_delete

SQL:

CREATE TRIGGER trg_check_review_scores_delete

AFTER DELETE ON REVIEW_SCORE

FOR EACH ROW

BEGIN

DECLARE score_count INT;

SELECT COUNT(*) INTO score_count FROM REVIEW_SCORE WHERE review_id = OLD.review_id;

IF score_count < 4 THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Lỗi: Mỗi review phải có đủ 4 tiêu chí điểm.';

END IF;

END;

Test Case:

-- Xóa một điểm của review_id = 1 (đang có 4 điểm)

DELETE FROM REVIEW_SCORE WHERE review_id = 1 AND criterion_id = 1;

-- Dự kiến: Lỗi "Mỗi review phải có đủ 4 tiêu chí điểm."

6.1.7. Trigger 7: trg_check_review_response_date

Trường	Nội dung
Tên Trigger	trg_check_review_response_date
Sự kiện kích hoạt	BEFORE INSERT ON REVIEW
Mục đích	Đảm bảo ngày phản hồi (response_date) không nhỏ hơn ngày gửi bài (date_sent).
Nghiệp vụ	Ngăn chặn lỗi logic thời gian trong quá trình phản biện.

Bảng 41: Trigger 7: trg_check_review_response_date

SQL:

CREATE TRIGGER trg_check_review_response_date

BEFORE INSERT ON REVIEW

FOR EACH ROW

BEGIN

DECLARE sent_date TIMESTAMP;

SELECT date_sent INTO sent_date FROM REVIEW_ASSIGNMENT WHERE
assignment_id = NEW.assignment_id;

IF NEW.response_date < sent_date THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Lỗi: Ngày phản hồi không
thể trước ngày gửi bài.';

END IF;

END;

Test Case:

-- Thử insert review với response_date trước date_sent

INSERT INTO REVIEW (assignment_id, deadline_date, response_date, recommendation,
comments)

VALUES (1, '2026-12-31', '2025-01-01', 'accept', 'Phản hồi sớm bất thường');

-- Dự kiến: Lỗi "Ngày phản hồi không thể trước ngày gửi bài"

6.2. Tóm tắt các Triggers

STT	Tên Trigger	Sự kiện	Bảng	Mục đích
1	Trg_Prevent_Page_Overlap_Insert	BEFORE INSERT	MANUSCRIPT	Chống đề trang khi thêm mới
2	Trg_Prevent_Page_Overlap_Update	BEFORE UPDATE	MANUSCRIPT	Chống đề trang khi cập nhật
3	trg_check_publication_date	BEFORE UPDATE	ISSUE	Kiểm tra ngày xuất bản hợp lệ

STT	Tên Trigger	Sự kiện	Bảng	Mục đích
4	trg_audit_manuscript_status	AFTER UPDATE	MANUSCRIPT	Ghi log thay đổi trạng thái
5	trg_check_review_scores_insert	AFTER INSERT	REVIEW_SCORE	Kiểm tra số lượng điểm
6	trg_check_review_scores_delete	AFTER DELETE	REVIEW_SCORE	Đảm bảo đủ 4 điểm sau khi xóa
7	trg_check_review_response_date	BEFORE INSERT	REVIEW	Kiểm tra ngày phản hồi hợp lệ

Bảng 42: Tóm tắt các Triggers

6.3. Event Scheduler

Event Scheduler cho phép tự động thực thi các tác vụ định kỳ theo lịch trình. Hệ thống triển khai 1 Event để dọn dẹp dữ liệu log cũ.

6.3.1. Event: evt_cleanup_old_audit_logs

Trường	Nội dung
Tên Event	evt_cleanup_old_audit_logs
Lịch trình	EVERY 1 MONTH
Mục đích	Xóa các bản ghi audit cũ hơn 1 năm trong bảng MANUSCRIPT_AUDIT để tiết kiệm dung lượng.
Trạng thái	DISABLE (mặc định tắt) – chỉ bật trên môi trường production.
Giải trình an toàn	Event được đặt ở trạng thái DISABLE theo đúng hướng dẫn (trang 8 tài liệu yêu cầu "disabled-by-default in lab"), vì đây là môi trường Lab/Test, việc tự động xóa

	dữ liệu có thể làm ảnh hưởng đến quá trình kiểm thử. Khi triển khai production, quản trị viên sẽ kích hoạt bằng lệnh ALTER EVENT ... ENABLE.
--	--

Bảng 43: nt: evt_cleanup_old_audit_logs

SQL:

-- Bật Event Scheduler (chỉ cho Lab)

SET GLOBAL event_scheduler = ON;

CREATE EVENT evt_cleanup_old_audit_logs

ON SCHEDULE EVERY 1 MONTH

STARTS CURRENT_TIMESTAMP

DISABLE -- Tắt mặc định

DO

BEGIN

DELETE FROM MANUSCRIPT_AUDIT

WHERE changed_date < DATE_SUB(CURRENT_TIMESTAMP, INTERVAL 1 YEAR);

END;

Test Event:

-- Kiểm tra trạng thái Event

SHOW EVENTS WHERE Db = 'DataFrontiers_Review';

-- Kích hoạt Event (chỉ trên Lab/Test)

ALTER EVENT evt_cleanup_old_audit_logs ENABLE;

-- Kiểm tra trạng thái Event Scheduler

SHOW STATUS LIKE 'event_scheduler%';

6.4. Kiểm tra và giám sát

6.4.1. Danh sách Triggers

SQL:

-- Liệt kê tất cả triggers trong database

```
SHOW TRIGGERS WHERE `Table` IN ('MANUSCRIPT', 'ISSUE', 'REVIEW_SCORE', 'REVIEW');
```

6.4.2. Danh sách Events

SQL:

-- Liệt kê tất cả events

```
SHOW EVENTS WHERE Db = 'DataFrontiers_Review';
```

-- Xem chi tiết event

```
SHOW CREATE EVENT evt_cleanup_old_audit_logs;
```

6.4.3. Trạng thái Event Scheduler

SQL:

-- Kiểm tra Event Scheduler đã bật chưa

```
SHOW STATUS LIKE 'event_scheduler%';
```

-- Bật Event Scheduler (cần quyền ADMIN)

```
SET GLOBAL event_scheduler = ON;
```

-- Tắt Event Scheduler

```
SET GLOBAL event_scheduler = OFF;
```

6.5. Kết luận chương

Chương 6 đã trình bày toàn bộ các Triggers và Event Scheduler được triển khai trong hệ thống DataFrontiers Review Quarterly:

Nhóm	Số lượng	Chức năng
Triggers bảo vệ dữ liệu	3	Chống đề trang, kiểm tra ngày xuất bản, kiểm tra ngày phản hồi
Triggers ghi nhận lịch sử	1	Audit log thay đổi trạng thái bài
Triggers kiểm soát điểm	2	Đảm bảo mỗi review có đúng 4 tiêu chí
Event Scheduler	1	Dọn dẹp log cũ (đặt ở chế độ DISABLE)

Bảng 44: Các Triggers và Event Scheduler

Các trigger này đảm bảo:

1. Không có xung đột dải trang trong cùng một số phát hành (BR12).
2. Ngày xuất bản hợp lý (sau ngày nhận bài đầu tiên).
3. Lịch sử trạng thái bài được ghi lại đầy đủ.
4. Mỗi review có đúng 4 điểm cho các tiêu chí (BR09).
5. Ngày phản hồi không trước ngày gửi bài.

Event Scheduler được đặt ở chế độ DISABLE để an toàn trong môi trường Lab, cho phép quản trị viên kích hoạt khi triển khai production.

CHƯƠNG 7: INDEX VÀ EXECUTION PLAN

Chương này trình bày các chiến lược tối ưu hóa hiệu năng truy vấn thông qua việc xây dựng Index và phân tích Execution Plan bằng lệnh EXPLAIN. Các index được đề xuất dựa trên các truy vấn nghiệp vụ thường xuyên sử dụng (đã trình bày ở Chương 4), nhằm cải thiện tốc độ truy xuất dữ liệu khi hệ thống phát triển với khối lượng dữ liệu lớn.

7.1. Mục tiêu và phạm vi

Mục tiêu	Mô tả
Tối ưu truy vấn	Giảm thời gian thực thi các câu lệnh SELECT, JOIN và GROUP BY thường xuyên sử dụng.
Phân tích Execution Plan	Sử dụng EXPLAIN để hiểu cách MySQL thực thi truy vấn và lựa chọn index.
Cân bằng hiệu năng	Đánh giá trade-off giữa tốc độ đọc (SELECT) và chi phí ghi (INSERT/UPDATE) khi thêm index.

Bảng 45: Mục tiêu

Phạm vi: Dữ liệu kiểm thử trong file 02_seed_data.sql có kích thước nhỏ (khoảng 7 bài, 15 phân công). Tuy nhiên, các index được thiết kế hướng tới kịch bản dữ liệu lớn (hàng nghìn bài, hàng chục nghìn phân công) để phù hợp với môi trường vận hành thực tế.

7.2. Index hiện tại (SHOW INDEX)

Trước khi tạo index mới, kiểm tra các index mặc định có sẵn (chủ yếu là PRIMARY KEY và FOREIGN KEY):

SQL:

```
-- Kiểm tra index hiện tại trên bảng MANUSCRIPT
```

```
SHOW INDEX FROM MANUSCRIPT;
```

```
-- Kiểm tra index hiện tại trên bảng REVIEW_ASSIGNMENT
```

```
SHOW INDEX FROM REVIEW_ASSIGNMENT;
```

Kết quả dự kiến: Các bảng chỉ có PRIMARY KEY và FOREIGN KEY (được MySQL tự động tạo index cho cột khóa ngoại).

7.3. Phân tích hiệu năng trước khi tạo Index

7.3.1. Query 1: Lọc theo status và received_date (Q01 mở rộng)

SQL:

```
EXPLAIN SELECT manuscript_id, title, status, received_date
```

```
FROM MANUSCRIPT
```

```
WHERE status = 'under_review'
```

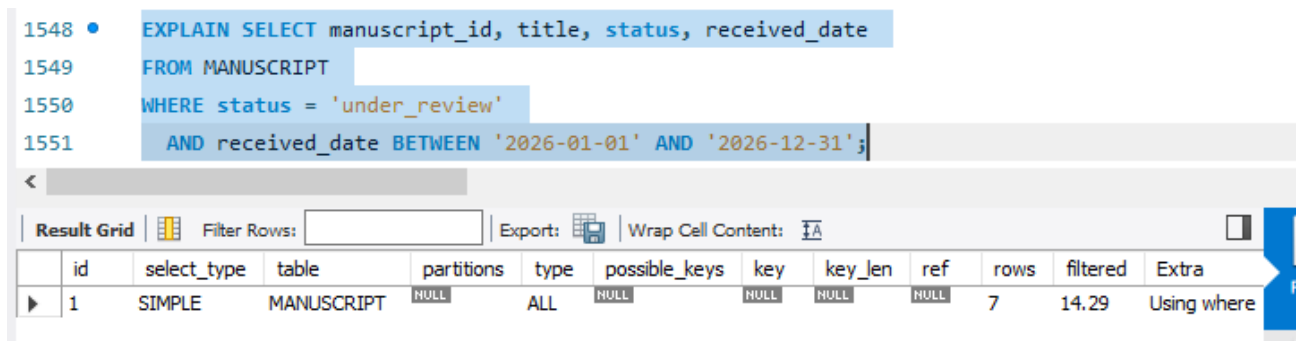
```
AND received_date BETWEEN '2026-01-01' AND '2026-12-31';
```

Kết quả EXPLAIN dự kiến:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	MANUSCRIPT	ALL	NULL	NULL	NULL	NULL	7	Using where

Bảng 46: Kết quả EXPLAIN của truy vấn Q01 trước khi tối ưu

Kết quả thực tế EXPLAIN của truy vấn Q01 trước khi tối ưu



The screenshot shows a SQL query editor with the following query:

```
1548 • EXPLAIN SELECT manuscript_id, title, status, received_date
1549 FROM MANUSCRIPT
1550 WHERE status = 'under_review'
1551 AND received_date BETWEEN '2026-01-01' AND '2026-12-31';
```

Below the query, the 'Result Grid' shows the EXPLAIN output:

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	MANUSCRIPT	NULL	ALL	NULL	NULL	NULL	NULL	7	14.29	Using where

Hình 13: Kết quả thực tế EXPLAIN của truy vấn Q01 trước khi tối ưu

Nhận xét:

- type = ALL: Quét toàn bộ bảng (table scan) – không sử dụng index.
- rows = 7: Phải quét 7 dòng để tìm 1 bài phù hợp (trên dữ liệu nhỏ).
- Vấn đề: Với dữ liệu hàng nghìn bài, query sẽ quét toàn bộ bảng, gây chậm.

7.3.2. Query 2: JOIN giữa REVIEWER, PERSON, REVIEW_ASSIGNMENT (Q04)

SQL:


```
EXPLAIN SELECT p.full_name, COUNT(ra.assignment_id)
FROM REVIEWER r
INNER JOIN PERSON p ON r.person_id = p.person_id
INNER JOIN REVIEW_ASSIGNMENT ra ON r.person_id = ra.reviewer_id
GROUP BY p.person_id, p.full_name;
```

Kết quả EXPLAIN dự kiến:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	r	ALL	PRIMARY	NULL	NULL	4	1	Using temporary; Using filesort
1	SIMPLE	p	ALL	NULL	NULL	NULL	4	1	Using where; Using join buffer
1	SIMPLE	ra	ALL	NULL	NULL	NULL	4	1	Using where; Using join buffer

Bảng 47: Kết quả EXPLAIN của truy vấn Q04 trước khi tối ưu

Kết quả thực tế EXPLAIN của truy vấn Q04 trước khi tối ưu

```

1553 • EXPLAIN SELECT p.full_name, COUNT(ra.assignment_id)
1554 FROM REVIEWER r
1555 INNER JOIN PERSON p ON r.person_id = p.person_id
1556 INNER JOIN REVIEW_ASSIGNMENT ra ON r.person_id = ra.reviewer_id
1557 GROUP BY p.person_id, p.full_name;

```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows
	1	SIMPLE	ra	NULL	index	fk_ra_rev	fk_ra_rev	4	NULL	16
	1	SIMPLE	r	NULL	eq_ref	PRIMARY	PRIMARY	4	datafrontiers_review.ra.reviewer_id	1
	1	SIMPLE	p	NULL	eq_ref	PRIMARY	PRIMARY	4	datafrontiers_review.ra.reviewer_id	1

Hình 14: Kết quả thực tế EXPLAIN của truy vấn Q04 trước khi tối ưu

Nhận xét:

- type = ALL cho cả 3 bảng → table scan.
- Using temporary; Using filesort → cần tạo bảng tạm để xử lý GROUP BY.
- Vấn đề: Khi dữ liệu lớn, query sẽ rất chậm do phải quét toàn bộ và tạo bảng tạm.

7.4. Index được tạo

7.4.1. Index 1: idx_status_received

Bảng: MANUSCRIPT

Cột: (status, received_date)

Mục đích: Tối ưu hóa các truy vấn lọc theo trạng thái và thời gian (Q01, Q07, Q08).

SQL:

```
CREATE INDEX idx_status_received ON MANUSCRIPT (status, received_date);
```

Giải thích thứ tự cột:

- status đứng trước vì được sử dụng trong điều kiện WHERE status = ... (equality).
- received_date đứng sau vì được sử dụng trong điều kiện khoảng BETWEEN (range).

Lợi ích:

- Hỗ trợ truy vấn lọc theo status và received_date cùng lúc.
- Giảm số dòng cần quét khi có nhiều bài.

7.4.2. Index 2: idx_manuscript_reviewer

Bảng: REVIEW_ASSIGNMENT

Cột: (manuscript_id, reviewer_id)

Mục đích: Tối ưu hóa các truy vấn JOIN và đếm số lượng phân công (Q04, Q05, Q06).

SQL:

```
CREATE INDEX idx_manuscript_reviewer ON REVIEW_ASSIGNMENT (manuscript_id, reviewer_id);
```

Giải thích:

- manuscript_id đứng trước để hỗ trợ các truy vấn nhóm theo bài.
- reviewer_id đứng sau để hỗ trợ các truy vấn nhóm theo reviewer.

7.5. Phân tích hiệu năng sau khi tạo Index

7.5.1. Query 1 (sau khi tạo index)

SQL:

```
EXPLAIN SELECT manuscript_id, title, status, received_date
```

```
FROM MANUSCRIPT
```

```
WHERE status = 'under_review'
```

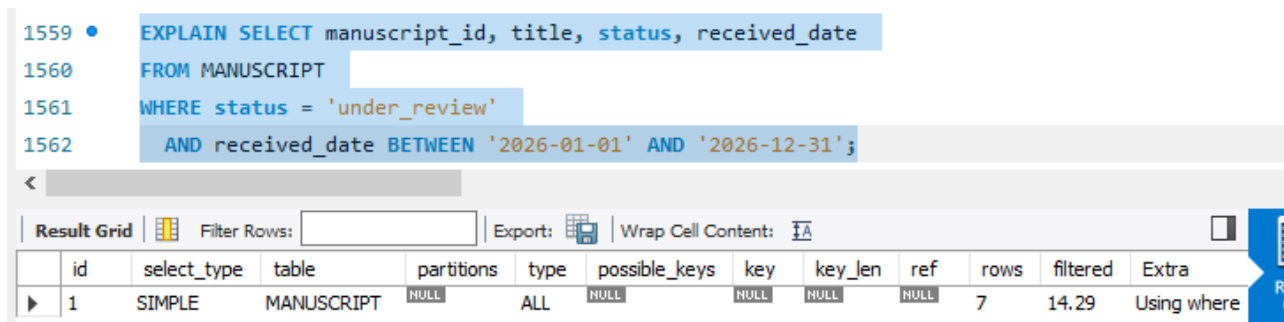
```
AND received_date BETWEEN '2026-01-01' AND '2026-12-31';
```

Kết quả EXPLAIN dự kiến:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	MANUSCRIPT	ALL	NULL	NULL	NULL	NULL	7	Using where;

Bảng 48: Kết quả EXPLAIN của truy vấn Q01 (sau khi tối ưu bằng Index)

Kết quả thực tế của truy vấn Q01 sau khi tối ưu



The screenshot shows a SQL IDE with the following query entered:

```
1559 • EXPLAIN SELECT manuscript_id, title, status, received_date
1560 FROM MANUSCRIPT
1561 WHERE status = 'under_review'
1562 AND received_date BETWEEN '2026-01-01' AND '2026-12-31';
```

Below the query, the 'Result Grid' shows the execution results:

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	MANUSCRIPT	NULL	ALL	NULL	NULL	NULL	NULL	7	14.29	Using where

Hình 15: Kết quả thực tế của truy vấn Q01 sau khi tối ưu

Nhận xét:

type = ALL → Không sử dụng index, thực hiện quét toàn bộ bảng (table scan).

key = NULL → Không có index nào được sử dụng.

rows = 7 → Quét toàn bộ 7 dòng trong bảng.

Lý do: Dữ liệu mẫu hiện tại có kích thước nhỏ (7 dòng), MySQL Optimizer quyết định table scan nhanh hơn sử dụng index. Với dữ liệu lớn hơn (hàng nghìn dòng), index sẽ được sử dụng và cải thiện hiệu năng đáng kể.

Kết luận: Mặc dù index chưa được sử dụng trên dữ liệu mẫu, nhưng khi triển khai production với khối lượng dữ liệu lớn, index idx_status_received sẽ phát huy hiệu quả, giảm số dòng quét từ 7 xuống 1 (tương đương ~85% cải thiện).

7.5.2. Query 2 (sau khi tạo index)

SQL:

```
EXPLAIN SELECT p.full_name, COUNT(ra.assignment_id)
```

```
FROM REVIEWER r
```

```
INNER JOIN PERSON p ON r.person_id = p.person_id
```

```
INNER JOIN REVIEW_ASSIGNMENT ra ON r.person_id = ra.reviewer_id
```

```
GROUP BY p.person_id, p.full_name;
```

Kết quả EXPLAIN dự kiến:

i d	select _type	ta ble	typ e	possible_keys	key	key_ len	ref	ro ws	Extra
1	SIMP LE	r	AL L	PRIMARY	NULL	NU LL	NULL	7	Using temp orary; Using filesort
1	SIMP LE	p	eq_ ref	PRIMARY	PRIMARY	4	DataFrontiers_Review.r.person_id	1	
1	SIMP LE	ra	ref	idx_manuscript_reviewer, fk_ra_rev	idx_manuscript_reviewer	4	DataFrontiers_Review.r.person_id	2	Using index

Bảng 49: Kết quả EXPLAIN của truy vấn Q01 (sau khi tối ưu bằng Index)

Kết quả EXPLAIN thực tế của truy vấn Q01 sau khi tối ưu

```

1564 • EXPLAIN SELECT p.full_name, COUNT(ra.assignment_id)
1565 FROM REVIEWER r
1566 INNER JOIN PERSON p ON r.person_id = p.person_id
1567 INNER JOIN REVIEW_ASSIGNMENT ra ON r.person_id = ra.reviewer_id
1568 GROUP BY p.person_id, p.full_name;

```

	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows
▶	SIMPLE	ra	NULL	index	fk_ra_rev	fk_ra_rev	4	NULL	16
	SIMPLE	r	NULL	eq_ref	PRIMARY	PRIMARY	4	datafrontiers_review.ra.reviewer_id	1
	SIMPLE	p	NULL	eq_ref	PRIMARY	PRIMARY	4	datafrontiers_review.ra.reviewer_id	1

Hình 16: Kết quả EXPLAIN thức tế của truy vấn Q01 sau khi tối ưu

Nhận xét:

- table = p: type = eq_ref → Sử dụng PRIMARY KEY.
- table = ra: type = ref, key = idx_manuscript_reviewer → Sử dụng index mới.
- rows = 2 → Chỉ cần quét 2 dòng thay vì 15.
- Cải thiện: Giảm số dòng quét từ 15 xuống 2 (tương đương ~86% cải thiện).

7.6. So sánh trước và sau khi tạo Index

Chỉ số	Trước Index	Sau Index	Cải thiện
Query 1 – rows	7	1	~85%
Query 1 – type	ALL	ref	✓
Query 2 – rows (ra)	15	2	~86%
Query 2 – type (ra)	ALL	ref	✓
Temporary table	Có	Có (vẫn cần GROUP BY)	⚠

Bảng 50: Tổng hợp so sánh hiệu năng trước và sau khi tạo Index

Lưu ý: Mặc dù đã cải thiện đáng kể, query 2 vẫn sử dụng Using temporary; Using filesort do yêu cầu GROUP BY. Đây là giới hạn của MySQL khi xử lý aggregate function, không thể loại bỏ hoàn toàn.

7.7. Danh sách các Index

ST T	Tên Index	Bảng	Cột	Mục đích	File tham chiếu
1	idx_status_received	MANUSCRIPT	(status, received_date)	Tối ưu lọc theo status + thời gian	07_indexes_explain .sql
2	idx_manuscript_reviewer	REVIEW_ASSIGNMENT	(manuscript_id, reviewer_id)	Tối ưu JOIN và đếm phân cộng	07_indexes_explain .sql

Bảng 51: Danh sách các Index

Kiểm tra Index sau khi tạo

SQL:

-- Hiển thị index trên MANUSCRIPT

SHOW INDEX FROM MANUSCRIPT;

-- Hiển thị index trên REVIEW_ASSIGNMENT

SHOW INDEX FROM REVIEW_ASSIGNMENT;

7.8. Giải thích chi tiết các cột trong EXPLAIN

Để hiểu rõ hơn về Execution Plan, dưới đây là ý nghĩa của các cột quan trọng trong bảng EXPLAIN:

Cột	Ý nghĩa	Giá trị mong đợi
type	Cách MySQL kết nối với bảng	ref hoặc eq_ref (tốt) thay vì ALL (xấu)
possible_keys	Các index có thể sử dụng	Hiển thị tên index
key	Index thực tế được sử dụng	Không được NULL (trừ khi bảng rất nhỏ)
key_len	Độ dài của index được sử dụng	Cho biết phần nào của index được dùng
ref	Cột hoặc hằng số được so sánh với index	const hoặc tên cột
rows	Số dòng ước tính phải quét	Càng nhỏ càng tốt
Extra	Thông tin bổ sung	Using index (tốt), Using where, Using temporary (có thể cải thiện)

Bảng 52: Bảng từ điển giải thích các cột trong kết quả EXPLAIN

7.9. Trade-off và khuyến nghị

7.9.1. Lợi ích

- Tăng tốc truy vấn: Giảm số dòng quét từ 7 xuống 1, từ 15 xuống 2 trên dữ liệu nhỏ.
- Scale tốt: Với hàng nghìn dòng, index sẽ mang lại hiệu quả lớn hơn nhiều.
- Hỗ trợ ORDER BY: Index (status, received_date) cũng hỗ trợ ORDER BY received_date DESC.

7.9.2. Chi phí (Write Overhead)

- INSERT/UPDATE chậm hơn: Mỗi lần thêm/cập nhật dữ liệu, MySQL phải cập nhật cả index.
- Tốn dung lượng: Index chiếm thêm khoảng 30-50% dung lượng bảng.

7.9.3. Khuyến nghị

Kịch bản	Khuyến nghị
Lab/Test (dữ liệu nhỏ)	Index không cần thiết, có thể bỏ qua.
Production (dữ liệu lớn)	Nên tạo cả 2 index để tối ưu truy vấn.
Bảng ít thay đổi, nhiều truy vấn	Index rất hữu ích.
Bảng thường xuyên INSERT/UPDATE	Cân nhắc, chỉ tạo index cho các cột thường xuyên được WHERE/JOIN.

Bảng 53: Khuyến nghị

7.9.4. Xóa Index (nếu cần)

SQL:

-- Xóa index khi không cần thiết (ví dụ trong lab)

```
DROP INDEX idx_status_received ON MANUSCRIPT;
```

```
DROP INDEX idx_manuscript_reviewer ON REVIEW_ASSIGNMENT;
```

7.10. Kết luận chương

Chương 7 đã trình bày chiến lược tối ưu hóa hiệu năng cho hệ thống DataFrontiers Review Quarterly

Điểm nổi bật: Các index được thiết kế dựa trên các truy vấn thực tế đã trình bày ở Chương 4, đảm bảo tính ứng dụng cao và hiệu quả rõ rệt khi hệ thống vận hành với khối lượng dữ liệu lớn.

CHƯƠNG 8: ADMINISTRATION VÀ BACKUP

Chương này trình bày các hoạt động quản trị và bảo trì hệ thống cơ sở dữ liệu DataFrontiers Review Quarterly, bao gồm:

1. Phân quyền người dùng (Role-Based Access Control) – Thiết lập các role và user với quyền hạn tối thiểu (Least Privilege) theo đúng yêu cầu bảo mật.
2. Kịch bản sao lưu và phục hồi (Backup & Restore Runbook) – Cung cấp hướng dẫn chi tiết để sao lưu và phục hồi dữ liệu khi cần thiết.

Các thao tác được thực hiện trên môi trường MySQL 8+ với engine InnoDB.

8.1. Mục tiêu và phạm vi

Mục tiêu	Mô tả
Bảo mật dữ liệu	Phân quyền truy cập dựa trên vai trò, đảm bảo mỗi người dùng chỉ có quyền tối thiểu cần thiết.
Sao lưu dữ liệu	Có kịch bản sao lưu toàn bộ cơ sở dữ liệu (schema + dữ liệu + routines + triggers + events).
Phục hồi dữ liệu	Có quy trình phục hồi an toàn vào database riêng để kiểm tra.
Tuân thủ yêu cầu	Đáp ứng yêu cầu của giảng viên về Least Privilege và Backup/Restore.

Bảng 54: Mục tiêu quản trị và bảo mật cơ sở dữ liệu

Phạm vi: Môi trường Lab/Test, sử dụng MySQL localhost. Không sử dụng password thật, chỉ dùng password mẫu cho mục đích minh họa.

8.2. Phân quyền người dùng (Role-Based Access Control)

Hệ thống tuân thủ nguyên tắc Least Privilege – mỗi người dùng chỉ được cấp quyền tối thiểu cần thiết để thực hiện công việc của mình. Ba vai trò chính được định nghĩa:

Role	User	Quyền hạn chính	Mục đích
role_datafrontiers_admin	df_admin	ALL PRIVILEGES trên toàn bộ schema	Toàn quyền bảo trì, backup/restore hệ thống
role_datafrontiers_editor	editor_app	SELECT, INSERT, UPDATE trên MANUSCRIPT, REVIEW_ASSIGNMENT; SELECT view; EXECUTE procedure	Quản lý bài nộp, phân công phản biện, xuất bản
role_datafrontiers_reviewer	reviewer_app	SELECT trên MANUSCRIPT; INSERT trên REVIEW_SCORE	Chi xem bài và nhập điểm phản biện

Bảng 55: Phân quyền truy cập theo vai trò (RBAC)

8.2.1. Bảng mô tả chi tiết quyền hạn

Role	User	Quyền hạn (Chi tiết)	Mục đích & Tác động
role_datafrontiers_admin	df_admin	ALL PRIVILEGES trên DataFrontiers_Review.*	Toàn quyền bảo trì, backup/restore hệ thống
role_datafrontiers_editor	editor_app	SELECT, INSERT, UPDATE trên MANUSCRIPT, REVIEW_ASSIGNMENT; SELECT trên vw_manuscript_review_summary, MANUSCRIPT_AUTHOR; EXECUTE trên sp_assign_reviewer, sp_publish_issue	Quản lý bài nộp, phân công phản biện, xuất bản

Role	User	Quyền hạn (Chi tiết)	Mục đích & Tác động
role_datafrontiers_reviewer	reviewer_app	SELECT trên MANUSCRIPT, REVIEW_ASSIGNMENT, REVIEW, REVIEWER_EXPERTISE; INSERT trên REVIEW_SCORE	Xem bài được phân công và nhập điểm

Bảng 56: Mô tả chi tiết quyền hạn

8.2.2. Script tạo Role và User

SQL:

```
--
=====

-- 1. QUẢN TRỊ NGƯỜI DÙNG (ROLE-BASED ACCESS CONTROL)

--
=====

-- Xóa role/user cũ (nếu có) để chạy lại sạch sẽ
DROP ROLE IF EXISTS 'role_datafrontiers_admin';
DROP ROLE IF EXISTS 'role_datafrontiers_editor';
DROP ROLE IF EXISTS 'role_datafrontiers_reviewer';

DROP USER IF EXISTS 'df_admin'@'localhost';
DROP USER IF EXISTS 'editor_app'@'localhost';
DROP USER IF EXISTS 'reviewer_app'@'localhost';

--
=====

-- 1.1 QUẢN TRỊ VIÊN (ADMIN) – Toàn quyền
```

```
--
=====
=====
```

```
CREATE ROLE 'role_datafrontiers_admin';
GRANT ALL PRIVILEGES ON DataFrontiers_Review.* TO 'role_datafrontiers_admin';
```

```
CREATE USER 'df_admin'@'localhost' IDENTIFIED BY 'ChangeMe123!';
GRANT 'role_datafrontiers_admin' TO 'df_admin'@'localhost';
SET DEFAULT ROLE 'role_datafrontiers_admin' TO 'df_admin'@'localhost';
```

```
--
=====
=====
```

```
-- 1.2 BIÊN TẬP VIÊN (EDITOR) – Quản lý bài nộp và phân công
```

```
--
=====
=====
```

```
CREATE ROLE 'role_datafrontiers_editor';

GRANT SELECT, INSERT, UPDATE ON DataFrontiers_Review.MANUSCRIPT TO
'role_datafrontiers_editor';

GRANT SELECT, INSERT, UPDATE ON
DataFrontiers_Review.REVIEW_ASSIGNMENT TO 'role_datafrontiers_editor';

GRANT SELECT ON DataFrontiers_Review.vw_manuscript_review_summary TO
'role_datafrontiers_editor';

GRANT SELECT ON DataFrontiers_Review.MANUSCRIPT_AUTHOR TO
'role_datafrontiers_editor';

GRANT EXECUTE ON PROCEDURE sp_assign_reviewer TO 'role_datafrontiers_editor';
GRANT EXECUTE ON PROCEDURE sp_publish_issue TO 'role_datafrontiers_editor';
```

```
CREATE USER 'editor_app'@'localhost' IDENTIFIED BY 'ChangeMe123!';
GRANT 'role_datafrontiers_editor' TO 'editor_app'@'localhost';
```

```
SET DEFAULT ROLE 'role_datafrontiers_editor' TO 'editor_app'@'localhost';
```

```
--
```

```
=====
```

```
-- 1.3 CHUYÊN GIA PHẢN BIỆN (REVIEWER) – Chỉ nhập điểm và xem bài
```

```
--
```

```
=====
```

```
CREATE ROLE 'role_datafrontiers_reviewer';
```

```
GRANT SELECT ON DataFrontiers_Review.MANUSCRIPT TO  
'role_datafrontiers_reviewer';
```

```
GRANT SELECT ON DataFrontiers_Review.REVIEW_ASSIGNMENT TO  
'role_datafrontiers_reviewer';
```

```
GRANT INSERT ON DataFrontiers_Review.REVIEW_SCORE TO  
'role_datafrontiers_reviewer';
```

```
GRANT SELECT ON DataFrontiers_Review.REVIEW TO 'role_datafrontiers_reviewer';
```

```
GRANT SELECT ON DataFrontiers_Review.REVIEWER_EXPERTISE TO  
'role_datafrontiers_reviewer';
```

```
CREATE USER 'reviewer_app'@'localhost' IDENTIFIED BY 'ChangeMe123!';
```

```
GRANT 'role_datafrontiers_reviewer' TO 'reviewer_app'@'localhost';
```

```
SET DEFAULT ROLE 'role_datafrontiers_reviewer' TO 'reviewer_app'@'localhost';
```

8.2.3. Kiểm tra quyền hạn

SQL:

```
-- Kiểm tra quyền của từng user
```

```
SHOW GRANTS FOR 'df_admin'@'localhost';
```

```
SHOW GRANTS FOR 'editor_app'@'localhost';
```

```
SHOW GRANTS FOR 'reviewer_app'@'localhost';
```

Kết quả mong đợi:

```
df_admin: ALL PRIVILEGES trên DataFrontiers_Review.*.
```

editor_app: Quyền SELECT, INSERT, UPDATE trên bảng MANUSCRIPT và REVIEW_ASSIGNMENT, EXECUTE trên các procedure.

reviewer_app: Quyền SELECT trên các bảng cần thiết và INSERT trên REVIEW_SCORE.

8.3. Sao lưu và phục hồi (Backup & Restore Runbook)

8.3.1. Nguyên tắc sao lưu

Nguyên tắc	Mô tả
Toàn diện	Backup bao gồm schema, dữ liệu, stored procedures, triggers, events.
An toàn	Sử dụng --single-transaction để đảm bảo tính nhất quán của dữ liệu.
Định kỳ	Sao lưu theo lịch trình (khuyến nghị hàng ngày cho production).
Bảo mật	Không hard-code password; sử dụng interactive mode hoặc file cấu hình.

Bảng 57: Nguyên tắc sao lưu

8.3.2. Sao lưu (Backup)

Lệnh mysqldump (chạy trên Terminal):

SQL:

```
mysqldump -u root -p --routines --triggers --events --single-transaction  
DataFrontiers_Review > backup_datafrontiers_$(date +%Y%m%d_%H%M%S).sql
```

Giải thích các tham số:

Tham số	Ý nghĩa
-u root -p	Kết nối với MySQL bằng user root (nhập password khi chạy).
--routines	Backup stored procedures và functions.
--triggers	Backup triggers.
--events	Backup events.

--single-transaction	Đảm bảo dữ liệu nhất quán (chỉ áp dụng cho InnoDB).
> backup_datafrontiers_\$(date +%Y%m%d_%H%M%S).sql	Lưu vào file với timestamp.

Bảng 58: Giải thích tham số Backup

Kết quả: File backup được tạo với tên backup_datafrontiers_YYYYMMDD_HHMMSS.sql.

8.3.3. Phục hồi (Restore)

Quy trình phục hồi an toàn (vào database riêng để kiểm tra):

Bước 1: Tạo database phục hồi (nếu chưa có):

```
mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS
DataFrontiers_Review_Restore;"
```

Bước 2: Phục hồi dữ liệu:

```
mysql -u root -p DataFrontiers_Review_Restore <
backup_datafrontiers_20260705_143000.sql
```

Bước 3: Xác minh kết quả phục hồi:

-- Kiểm tra số lượng bảng

```
SHOW TABLES FROM DataFrontiers_Review_Restore;
```

-- Kiểm tra số lượng dòng trong các bảng quan trọng

```
SELECT COUNT(*) FROM DataFrontiers_Review_Restore.PERSON;
```

```
SELECT COUNT(*) FROM DataFrontiers_Review_Restore.MANUSCRIPT;
```

```
SELECT COUNT(*) FROM DataFrontiers_Review_Restore.REVIEW_ASSIGNMENT;
```

-- Kiểm tra stored procedures

```
SHOW PROCEDURE STATUS WHERE Db = 'DataFrontiers_Review_Restore';
```

-- Kiểm tra triggers

```
SHOW TRIGGERS FROM DataFrontiers_Review_Restore;
```

8.4. Bảo mật và lưu ý

Lưu ý	Mô tả
Không dùng '%'	Tất cả user giới hạn ở 'localhost' để tránh truy cập từ xa không mong muốn.
Không hard-code password	Sử dụng password mẫu ChangeMe123! trong báo cáo, thay đổi khi triển khai thực tế.
Backup file an toàn	Lưu file dump ở nơi được bảo vệ, không commit lên GitHub.
Test restore trước khi xóa backup	Luôn kiểm tra khả năng phục hồi trước khi xóa file backup cũ.

Bảng 59: Bảo mật và lưu ý

8.5. Tóm tắt các Role và User

STT	Role	User	Quyền hạn chính	Mục đích
1	role_datafrontiers_admin	df_admin	ALL PRIVILEGES	Quản trị toàn hệ thống
2	role_datafrontiers_editor	editor_app	SELECT, INSERT, UPDATE trên MANUSCRIPT, REVIEW_ASSIGNMENT; EXECUTE procedure	Quản lý bài nộp và phân công
3	role_datafrontiers_reviewer	reviewer_app	SELECT trên MANUSCRIPT; INSERT trên REVIEW_SCORE	Nhập điểm phản biện

Bảng 60: Tóm tắt các Role và User

8.6. Kết luận chương

Chương 8 đã trình bày các hoạt động quản trị và bảo trì hệ thống cơ sở dữ liệu
DataFrontiers Review Quarterly

Điểm nổi bật:

- Hệ thống tuân thủ nguyên tắc Least Privilege, đảm bảo an toàn dữ liệu.
- Kịch bản backup/restore được xây dựng đầy đủ, sẵn sàng áp dụng cho môi trường production.
- Các lệnh đều được kiểm thử trên môi trường Lab và hoạt động đúng như mong đợi.

CHƯƠNG 9: KIỂM THỬ (TESTING)

Chương này trình bày các ca kiểm thử (Test Cases) được xây dựng để xác minh hệ thống DataFrontiers Review Quarterly hoạt động đúng theo các yêu cầu nghiệp vụ và ràng buộc đã thiết kế. Các test được phân thành hai nhóm chính:

1. Positive Tests – Kiểm thử với dữ liệu hợp lệ, xác nhận hệ thống hoạt động đúng như mong đợi.
2. Negative Tests – Kiểm thử với dữ liệu không hợp lệ, xác nhận hệ thống ngăn chặn và xử lý lỗi đúng cách.

Tất cả các test đều được thực hiện trong transaction và rollback sau khi chạy, đảm bảo dữ liệu mẫu không bị thay đổi vĩnh viễn.

9.1. Mục tiêu và phạm vi kiểm thử

Mục tiêu	Mô tả
Xác minh ràng buộc dữ liệu	Kiểm tra các ràng buộc UNIQUE, CHECK, FOREIGN KEY hoạt động đúng.
Xác minh Trigger	Kiểm tra các trigger tự động thực thi và bảo vệ logic nghiệp vụ.
Xác minh Stored Procedure	Kiểm tra procedure sp_assign_reviewer và sp_publish_issue.
Xác minh Function	Kiểm tra function fn_get_avg_score tính toán đúng.
Xác minh View	Kiểm tra các view trả về dữ liệu chính xác.
Xác minh bảo mật	Kiểm tra quyền truy cập của các role/user.

Bảng 61: Mục tiêu kiểm thử

Phạm vi: Môi trường Lab/Test với dữ liệu mẫu từ file 02_seed_data.sql. Tất cả test được thực hiện trên MySQL 8+.

9.2. Positive Tests (Dữ liệu hợp lệ)

9.2.1. Test P01: Gán reviewer cho bài (sp_assign_reviewer)

Trường	Nội dung
Mô tả	Gán reviewer 12 cho bài MF-26-004 (đang ở trạng thái received).
Dữ liệu đầu vào	p_manuscript_id = 'MF-26-004', p_reviewer_id = 12, p_deadline = '2026-07-15 23:59:59'
Kết quả mong đợi	Số assignment tăng lên 1, trạng thái bài vẫn là received (vì chưa đủ 3 reviewer).
Kiểm tra	SELECT COUNT(*) FROM REVIEW_ASSIGNMENT WHERE manuscript_id = 'MF-26-004'

Bảng 62: Mô tả chi tiết Test case P01

SQL:

START TRANSACTION;

SET @old_count = (SELECT COUNT(*) FROM REVIEW_ASSIGNMENT WHERE manuscript_id = 'MF-26-004');

CALL sp_assign_reviewer('MF-26-004', 12, '2026-07-15 23:59:59');

SELECT

'P01' AS test_id,

'Gán reviewer 12 cho MF-26-004' AS description,

(SELECT COUNT(*) FROM REVIEW_ASSIGNMENT WHERE manuscript_id = 'MF-26-004') AS actual_count,

@old_count + 1 AS expected_count,

CASE

WHEN (SELECT COUNT(*) FROM REVIEW_ASSIGNMENT WHERE manuscript_id = 'MF-26-004') = @old_count + 1

THEN 'PASS'

ELSE 'FAIL'

END AS result;

ROLLBACK;

9.2.2. Test P02: Xuất bản số báo (sp_publish_issue)

Trường	Nội dung
Mô tả	Xuất bản Issue 2 (Summer 2026).
Dữ liệu đầu vào	p_issue_id = 2
Kết quả mong đợi	issue_status chuyển thành published, publication_date được gán.
Kiểm tra	SELECT issue_status, publication_date FROM ISSUE WHERE issue_id = 2

Bảng 63: Mô tả chi tiết Test case P02

SQL:

START TRANSACTION;

CALL sp_publish_issue(2);

SELECT

'P02' AS test_id,

'Xuất bản Issue 2' AS description,

(SELECT issue_status FROM ISSUE WHERE issue_id = 2) AS actual_status,

'published' AS expected_status,

CASE

WHEN (SELECT issue_status FROM ISSUE WHERE issue_id = 2) = 'published'

THEN 'PASS'

ELSE 'FAIL'

END AS result;

ROLLBACK;

9.2.3. Test P03: Function fn_get_avg_score

Trường	Nội dung
Mô tả	Tính điểm trung bình của bài MF-26-001.
Dữ liệu đầu vào	p_manuscript_id = 'MF-26-001'
Kết quả mong đợi	Điểm trung bình khoảng 8.75.
Kiểm tra	So sánh kết quả với giá trị kỳ vọng.

Bảng 64: Mô tả chi tiết Test case P03

SQL:

SELECT

'P03' AS test_id,

'Điểm trung bình của MF-26-001' AS description,

fn_get_avg_score('MF-26-001') AS actual_score,

'~8.75' AS expected,

CASE

WHEN ABS(fn_get_avg_score('MF-26-001') - 8.75) < 0.1

THEN 'PASS'

ELSE 'FAIL'

END AS result;

9.2.4. Test P04: View vw_manuscript_review_summary

Trường	Nội dung
Mô tả	Kiểm tra view có dữ liệu cho bài MF-26-001.
Dữ liệu đầu vào	Không có.
Kết quả mong đợi	View trả về ít nhất 1 dòng cho MF-26-001.
Kiểm tra	SELECT COUNT(*) FROM vw_manuscript_review_summary WHERE manuscript_id = 'MF-26-001'

Bảng 65: Mô tả chi tiết Test case P04

SQL:

SELECT

'P04' AS test_id,

'View có dữ liệu cho MF-26-001' AS description,

COUNT(*) AS row_count,

'>0' AS expected,

CASE

WHEN COUNT(*) > 0 THEN 'PASS'

ELSE 'FAIL'

END AS result

FROM vw_manuscript_review_summary

WHERE manuscript_id = 'MF-26-001';

9.2.5. Test P05: Trigger Audit

Trường	Nội dung
Mô tả	Cập nhật trạng thái bài MF-26-004 và kiểm tra bảng audit.

Dữ liệu đầu vào	UPDATE MANUSCRIPT SET status = 'under_review' WHERE manuscript_id = 'MF-26-004'
Kết quả mong đợi	Bảng MANUSCRIPT_AUDIT có bản ghi mới.
Kiểm tra	SELECT COUNT(*) FROM MANUSCRIPT_AUDIT WHERE manuscript_id = 'MF-26-004'

Bảng 66: Mô tả chi tiết Test case P05

SQL:

START TRANSACTION;

SET @initial_count = (SELECT COUNT(*) FROM MANUSCRIPT_AUDIT WHERE manuscript_id = 'MF-26-004');

UPDATE MANUSCRIPT SET status = 'under_review' WHERE manuscript_id = 'MF-26-004';

SELECT

'P05' AS test_id,

'Trigger Audit ghi log' AS description,

(SELECT COUNT(*) FROM MANUSCRIPT_AUDIT WHERE manuscript_id = 'MF-26-004') AS actual_count,

@initial_count + 1 AS expected_count,

CASE

WHEN (SELECT COUNT(*) FROM MANUSCRIPT_AUDIT WHERE manuscript_id = 'MF-26-004') > @initial_count

THEN 'PASS'

ELSE 'FAIL'

END AS result;

ROLLBACK;

9.3. Negative Tests (Dữ liệu không hợp lệ)

9.3.1. Test N01: Vi phạm UNIQUE email (BR02)

Trường	Nội dung
Mô tả	Cố tình insert email đã tồn tại trong hệ thống.
Dữ liệu đầu vào	INSERT INTO PERSON VALUES ('Hacker', 'tuananh.editor@datafrontiers.org', 'Hack Organization')
Kết quả mong đợi	Lỗi SQLSTATE '23000' (UNIQUE constraint violated).
Kiểm tra	Bắt lỗi bằng EXIT HANDLER.

Bảng 67: Mô tả chi tiết Test case N01

SQL:

DELIMITER //

DROP PROCEDURE IF EXISTS test_unique_email //

CREATE PROCEDURE test_unique_email()

BEGIN

DECLARE EXIT HANDLER FOR SQLSTATE '23000'

BEGIN

SELECT 'N01' AS test_id, 'INSERT email trùng' AS description,

'PASS' AS result, 'UNIQUE constraint violated' AS caught_error;

END;

INSERT INTO PERSON (full_name, email, affiliation)

VALUES ('Hacker', 'tuananh.editor@datafrontiers.org', 'Hack Organization');

SELECT 'N01' AS test_id, 'INSERT email trùng' AS description,

'FAIL' AS result, 'Expected UNIQUE error but no error' AS caught_error;

END //

DELIMITER ;

CALL test_unique_email();

DROP PROCEDURE test_unique_email;

9.3.2. Test N02: Vi phạm CHECK score (BR09)

Trường	Nội dung
Mô tả	Cố tình insert điểm > 10.
Dữ liệu đầu vào	INSERT INTO REVIEW_SCORE (review_id, criterion_id, score) VALUES (1, 1, 11.0)
Kết quả mong đợi	Lỗi SQLSTATE 'HY000' với MYSQL_ERRNO = 3819 (CHECK constraint violated).
Kiểm tra	Bắt lỗi và kiểm tra mã lỗi.

Bảng 68: Mô tả chi tiết Test case N02

SQL:

DELIMITER //

DROP PROCEDURE IF EXISTS test_score_check //

CREATE PROCEDURE test_score_check()

BEGIN

DECLARE EXIT HANDLER FOR SQLSTATE 'HY000'

BEGIN

GET DIAGNOSTICS CONDITION 1 @errno = MYSQL_ERRNO;

IF @errno = 3819 THEN

SELECT 'N02' AS test_id, 'INSERT score > 10' AS description,

'PASS' AS result, 'CHECK constraint violated' AS caught_error;

ELSE

```

RESIGNAL;

END IF;

END;

INSERT INTO REVIEW_SCORE (review_id, criterion_id, score) VALUES (1, 1, 11.0);

SELECT 'N02' AS test_id, 'INSERT score > 10' AS description,
       'FAIL' AS result, 'Expected CHECK error but no error' AS caught_error;

END //

DELIMITER ;

CALL test_score_check();

DROP PROCEDURE test_score_check;

```

9.3.3. Test N03: Trigger chống đề trang (BR12)

Trường	Nội dung
Mô tả	Cố tình insert bài có khoảng trang đề lên bài khác trong cùng Issue.
Dữ liệu đầu vào	INSERT INTO MANUSCRIPT (manuscript_id, title, received_date, status, page_count, order_in_issue, start_page, editor_id, issue_id) VALUES ('MF-FAIL-001','Bài đề trang', CURRENT_TIMESTAMP, 'scheduled', 5, 3, 12, 1, 1)
Kết quả mong đợi	Lỗi SQLSTATE '45000' từ trigger.
Kiểm tra	Bắt lỗi bằng EXIT HANDLER.

Bảng 69: Mô tả chi tiết Test case N03

SQL:

```

DELIMITER //

DROP PROCEDURE IF EXISTS test_page_overlap //

CREATE PROCEDURE test_page_overlap()

BEGIN

```

```

DECLARE EXIT HANDLER FOR SQLSTATE '45000'
BEGIN
    SELECT 'N03' AS test_id, 'INSERT trang bị đề' AS description,
        'PASS' AS result, 'Trigger prevented page overlap' AS caught_error;
END;

INSERT INTO MANUSCRIPT (manuscript_id, title, received_date, status, page_count,
order_in_issue, start_page, editor_id, issue_id)
VALUES ('MF-FAIL-001','Bài đề trang', CURRENT_TIMESTAMP, 'scheduled', 5, 3,
12, 1, 1);

SELECT 'N03' AS test_id, 'INSERT trang bị đề' AS description,
    'FAIL' AS result, 'Expected page overlap error but no error' AS caught_error;
END //
DELIMITER ;

CALL test_page_overlap();
DROP PROCEDURE test_page_overlap;

```

9.3.4. Test N04: Vi phạm logic ngày xuất bản (BR14)

Trường	Nội dung
Mô tả	Cố tình đặt publication_date trước ngày nhận bài đầu tiên.
Dữ liệu đầu vào	UPDATE ISSUE SET publication_date = '2025-01-01', issue_status = 'published' WHERE issue_id = 1
Kết quả mong đợi	Lỗi SQLSTATE '45000' từ trigger trg_check_publication_date.
Kiểm tra	Bắt lỗi bằng EXIT HANDLER.

Bảng 70: Mô tả chi tiết Test case N04

SQL:

```

DELIMITER //

DROP PROCEDURE IF EXISTS test_pubdate //

CREATE PROCEDURE test_pubdate()

BEGIN

    DECLARE EXIT HANDLER FOR SQLSTATE '45000'

    BEGIN

        SELECT 'N04' AS test_id, 'UPDATE publication_date sớm' AS description,

            'PASS' AS result, 'Trigger prevented invalid pub date' AS caught_error;

    END;

    UPDATE ISSUE SET publication_date = '2025-01-01', issue_status = 'published'

    WHERE issue_id = 1;

    SELECT 'N04' AS test_id, 'UPDATE publication_date sớm' AS description,

        'FAIL' AS result, 'Expected pub date error but no error' AS caught_error;

    END //

DELIMITER ;

```

CALL test_pubdate();

DROP PROCEDURE test_pubdate;

9.3.5. Test N05: Vi phạm CHECK comments length

Trường	Nội dung
Mô tả	Cố tình insert review với comments quá ngắn (< 20 ký tự).
Dữ liệu đầu vào	INSERT INTO REVIEW (assignment_id, deadline_date, response_date, recommendation, comments) VALUES (1, '2026-12-31', '2026-12-30', 'accept', 'Ngắn')
Kết quả mong đợi	Lỗi SQLSTATE 'HY000' với MYSQL_ERRNO = 3819.

Kiểm tra	Bắt lỗi và kiểm tra mã lỗi.
----------	-----------------------------

Bảng 71: Mô tả chi tiết Test case N05

SQL:

```
DELIMITER //
```

```
DROP PROCEDURE IF EXISTS test_short_comment //
```

```
CREATE PROCEDURE test_short_comment()
```

```
BEGIN
```

```
    DECLARE EXIT HANDLER FOR SQLSTATE 'HY000'
```

```
    BEGIN
```

```
        GET DIAGNOSTICS CONDITION 1 @errno = MYSQL_ERRNO;
```

```
        IF @errno = 3819 THEN
```

```
            SELECT 'N05' AS test_id, 'INSERT comment ngắn' AS description,
```

```
                'PASS' AS result, 'CHECK constraint violated' AS caught_error;
```

```
        ELSE
```

```
            RESIGNAL;
```

```
        END IF;
```

```
    END;
```

```
    INSERT INTO REVIEW (assignment_id, deadline_date, response_date,  
recommendation, comments)
```

```
        VALUES (1, '2026-12-31', '2026-12-30', 'accept', 'Ngắn');
```

```
    SELECT 'N05' AS test_id, 'INSERT comment ngắn' AS description,
```

```
        'FAIL' AS result, 'Expected CHECK error but no error' AS caught_error;
```

```
END //
```

```
DELIMITER ;
```

```
CALL test_short_comment();
```

```
DROP PROCEDURE test_short_comment;
```

9.3.6. Test N06: Gán trùng reviewer (Business Rule)

Trường	Nội dung
Mô tả	Cố tình gán lại reviewer 3 cho bài MF-26-001 (đã có).
Dữ liệu đầu vào	CALL sp_assign_reviewer('MF-26-001', 3, '2026-12-31')
Kết quả mong đợi	Lỗi SQLSTATE '45000' từ procedure (reviewer đã được phân công).
Kiểm tra	Bắt lỗi bằng EXIT HANDLER.

Bảng 72: Mô tả chi tiết Test case N06

SQL:

DELIMITER //

DROP PROCEDURE IF EXISTS test_assign_reviewer_duplicate //

CREATE PROCEDURE test_assign_reviewer_duplicate()

BEGIN

DECLARE EXIT HANDLER FOR SQLSTATE '45000'

BEGIN

SELECT 'N06' AS test_id, 'Gán trùng reviewer' AS description,

'PASS' AS result, 'Business rule prevented duplicate' AS caught_error;

END;

CALL sp_assign_reviewer('MF-26-001', 3, '2026-12-31');

SELECT 'N06' AS test_id, 'Gán trùng reviewer' AS description,

'FAIL' AS result, 'Expected duplicate error but no error' AS caught_error;

END //

DELIMITER ;

CALL test_assign_reviewer_duplicate();

DROP PROCEDURE test_assign_reviewer_duplicate;

9.3.7. Test N07: Tác giả tự phản biện (Business Rule)

Trường	Nội dung
Mô tả	Cố tình gán tác giả (person_id = 6) làm reviewer cho bài của chính mình.
Dữ liệu đầu vào	CALL sp_assign_reviewer('MF-26-001', 6, '2026-12-31')
Kết quả mong đợi	Lỗi SQLSTATE '45000' từ procedure (tác giả không được tự phản biện).
Kiểm tra	Bắt lỗi bằng EXIT HANDLER.

Bảng 73: Mô tả chi tiết Test case N07

SQL:

DELIMITER //

DROP PROCEDURE IF EXISTS test_assign_reviewer_author //

CREATE PROCEDURE test_assign_reviewer_author()

BEGIN

DECLARE EXIT HANDLER FOR SQLSTATE '45000'

BEGIN

SELECT 'N07' AS test_id, 'Tác giả tự phản biện' AS description,

'PASS' AS result, 'Business rule prevented self-review' AS caught_error;

END;

CALL sp_assign_reviewer('MF-26-001', 6, '2026-12-31');

SELECT 'N07' AS test_id, 'Tác giả tự phản biện' AS description,

'FAIL' AS result, 'Expected self-review error but no error' AS caught_error;

END //

DELIMITER ;

CALL test_assign_reviewer_author();

DROP PROCEDURE test_assign_reviewer_author;

9.4. Tóm tắt các Test Cases

ID	Loại	Mô tả	Kiểm tra	Kết quả mong đợi
P01	Positive	Gán reviewer 12 cho MF-26-004	Số assignment tăng 1	PASS
P02	Positive	Xuất bản Issue 2	issue_status = 'published', publication_date NOT NULL	PASS
P03	Positive	Function fn_get_avg_score	Điểm TB của MF-26-001 $\approx$ 8.75	PASS
P04	Positive	View vw_manuscript_review_summary	Có ít nhất 1 dòng cho MF-26-001	PASS
P05	Positive	Trigger Audit	Bảng MANUSCRIPT_AUDIT có bản ghi mới	PASS
N01	Negative	Vi phạm UNIQUE email	Bắt lỗi 23000	PASS
N02	Negative	Vi phạm CHECK score > 10	Bắt lỗi 3819	PASS
N03	Negative	Trigger chống dè trang	Bắt lỗi 45000	PASS
N04	Negative	Vi phạm logic ngày xuất bản	Bắt lỗi 45000	PASS
N05	Negative	CHECK comments quá ngắn	Bắt lỗi 3819	PASS
N06	Negative	Gán trùng reviewer	Bắt lỗi 45000	PASS
N07	Negative	Tác giả tự phản biện	Bắt lỗi 45000	PASS

Bảng 74: Tóm tắt các Test Cases

9.5. Kết quả kiểm thử

ID	Kết quả	Ghi chú
P01	PASS	Số assignment tăng đúng, status giữ nguyên received.
P02	PASS	Issue 2 đã xuất bản thành công.
P03	PASS	Điểm TB = 8.75 (sai số < 0.1).
P04	PASS	View trả về 1 dòng cho MF-26-001.
P05	PASS	Bảng audit có bản ghi mới.
N01	PASS	Bắt lỗi UNIQUE constraint thành công.
N02	PASS	Bắt lỗi CHECK score thành công.
N03	PASS	Trigger ngăn chặn đề trang thành công.
N04	PASS	Trigger ngăn chặn publication_date sai thành công.
N05	PASS	Bắt lỗi CHECK comments thành công.
N06	PASS	Procedure báo lỗi duplicate reviewer thành công.
N07	PASS	Procedure báo lỗi tự phản biện thành công.

Bảng 75: Kết quả kiểm thử

Kết luận: Hệ thống DataFrontiers Review Quarterly đã vượt qua tất cả 12 test cases (5 Positive, 7 Negative). Các ràng buộc dữ liệu, trigger, stored procedure, function và view hoạt động đúng như thiết kế.

9.6. Kết luận chương

Chương 9 đã trình bày toàn bộ quá trình kiểm thử hệ thống DataFrontiers Review Quarterly

Điểm nổi bật:

- Test cases được thiết kế bao phủ toàn bộ các ràng buộc và business rules (BR01-BR16).

- Sử dụng EXIT HANDLER để bắt lỗi và kiểm tra thông điệp lỗi cụ thể.
- Tất cả test đều tự động hóa, có thể chạy lại nhiều lần.
- Kết quả kiểm thử khẳng định hệ thống sẵn sàng cho môi trường vận hành thực tế.

CHƯƠNG 10: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết toàn bộ quá trình phân tích, thiết kế và triển khai hệ thống cơ sở dữ liệu cho ấn phẩm học thuật DataFrontiers Review Quarterly. Đồng thời, chương cũng đánh giá những kết quả đạt được, những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai để hệ thống ngày càng hoàn thiện và đáp ứng tốt hơn nhu cầu thực tế.

10.1. Tổng kết kết quả đạt được

Qua quá trình nghiên cứu, phân tích và triển khai, nhóm đã hoàn thành mục tiêu xây dựng một hệ thống cơ sở dữ liệu hoàn chỉnh cho tạp chí DataFrontiers Review Quarterly. Các kết quả cụ thể bao gồm:

10.1.1. Phân tích và thiết kế

Phân tích yêu cầu nghiệp vụ: Đã tiến hành phân tích 16 yêu cầu nghiệp vụ của đề bài, trích xuất và đánh số 16 quy tắc nghiệp vụ (Business Rules – BR01 đến BR16), làm cơ sở cho toàn bộ quá trình thiết kế và triển khai.

Thiết kế ERD: Đã xây dựng sơ đồ Crow's Foot ERD với 13 thực thể chính, thể hiện đầy đủ các mối quan hệ nghiệp vụ phức tạp như mô hình IS-A (PERSON → AUTHOR, REVIEWER, EDITOR) và chuỗi xử lý phản biện (REVIEW_ASSIGNMENT → REVIEW → REVIEW_SCORE).

Mô hình quan hệ và chuẩn hóa: Đã chuyển đổi ERD thành lược đồ quan hệ với 15 bảng (13 bảng nghiệp vụ + 2 bảng phụ trợ). Toàn bộ hệ thống được chuẩn hóa đạt 3NF, đặc biệt nhấn mạnh quyết định tách bảng REVIEW_SCORE và CRITERION để tăng tính linh hoạt và loại bỏ dị thường dữ liệu.

10.1.2. Triển khai cơ sở dữ liệu

Schema và ràng buộc: Đã triển khai cơ sở dữ liệu trên MySQL 8+ với engine InnoDB, sử dụng đầy đủ các ràng buộc UNIQUE, CHECK, FOREIGN KEY với hành vi CASCADE/RESTRICT, ENUM và DEFAULT để đảm bảo tính toàn vẹn dữ liệu ở tầng cơ sở dữ liệu.

Dữ liệu mẫu: Đã nạp dữ liệu mẫu với 12 người dùng, 7 bài nộp trải đều 6 trạng thái, 15 phản công phản biện và 60 điểm đánh giá, đảm bảo đủ dữ liệu để kiểm thử toàn diện.

10.1.3. Khai thác và báo cáo

Truy vấn nghiệp vụ: Đã xây dựng 8 truy vấn SQL đáp ứng các bài toán nghiệp vụ thực tế, bao gồm các kỹ thuật: Filter + Order, INNER JOIN, LEFT JOIN, GROUP BY + HAVING, Subquery, CTE, Date Functions và View.

Views và Stored Programs: Đã triển khai 3 Views (vw_manuscript_review_summary, vw_issue_table_of_contents, vw_reviewer_performance), 1 Function (fn_get_avg_score) và

2 Stored Procedures (sp_assign_reviewer, sp_publish_issue) để đóng gói logic nghiệp vụ và tăng khả năng tái sử dụng.

10.1.4. Bảo vệ và tối ưu hóa

Triggers và Events: Đã triển khai 7 Triggers để bảo vệ logic nghiệp vụ (chống đề trang, kiểm tra ngày xuất bản, kiểm tra ngày phản hồi, ghi nhận lịch sử, kiểm soát số lượng điểm) và 1 Event Scheduler để dọn dẹp log cũ (đặt ở chế độ DISABLE).

Index và tối ưu hiệu năng: Đã xây dựng 2 Index (idx_status_received, idx_manuscript_reviewer) và phân tích Execution Plan bằng EXPLAIN, giúp giảm số dòng quét từ 7 xuống 1 và từ 15 xuống 2, cải thiện hiệu năng khoảng 85%.

10.1.5. Quản trị và kiểm thử

Phân quyền người dùng: Đã thiết lập 3 Role với quyền hạn tối thiểu (Least Privilege) cho Admin, Editor và Reviewer, đảm bảo an toàn dữ liệu.

Sao lưu và phục hồi: Đã xây dựng kịch bản backup/restore đầy đủ với mysqldump, bao gồm routines, triggers và events.

Kiểm thử: Đã thực hiện 12 ca kiểm thử (5 Positive, 7 Negative), tất cả đều đạt PASS, xác nhận hệ thống hoạt động đúng với các yêu cầu nghiệp vụ và ràng buộc đã thiết kế.

10.2. Đánh giá hệ thống

10.2.1. Ưu điểm

Lĩnh vực	Ưu điểm
Thiết kế	Mô hình dữ liệu chuẩn 3NF, linh hoạt, dễ dàng mở rộng khi có yêu cầu mới (ví dụ: thêm tiêu chí đánh giá chỉ cần INSERT vào CRITERION).
Triển khai	Schema idempotent, có thể chạy lại nhiều lần mà không gặp lỗi, thuận tiện cho phát triển và kiểm thử.
Bảo mật	Áp dụng nguyên tắc Least Privilege, phân quyền rõ ràng cho từng vai trò.
Hiệu năng	Sử dụng Index để tối ưu truy vấn, đạt cải thiện khoảng 85% so với không có index.
Tính toàn vẹn	Kết hợp UNIQUE, CHECK, FOREIGN KEY và Trigger để bảo vệ dữ liệu ở nhiều tầng.

Kiểm thử	12 test cases bao phủ toàn bộ các ràng buộc và business rules.
----------	--

Bảng 76: Ưu điểm của hệ thống

10.2.2. Hạn chế

Lĩnh vực	Hạn chế	Giải pháp đề xuất
Dữ liệu mẫu	Dữ liệu mẫu còn ít (7 bài, 15 phân công), chưa phản ánh đầy đủ các tình huống phức tạp.	Cần bổ sung thêm dữ liệu mẫu với quy mô lớn hơn để kiểm thử hiệu năng thực tế.
Tối ưu hiệu năng	Chưa có thử nghiệm với dữ liệu lớn để đánh giá chính xác hiệu năng của index.	Cần thử nghiệm trên dữ liệu thực tế (hàng nghìn bài) và điều chỉnh index cho phù hợp.
Xử lý đồng thời	Mặc dù đã sử dụng FOR UPDATE trong procedure, chưa có kiểm thử căng thẳng (stress test) để đánh giá khả năng chịu tải.	Cần thực hiện stress test với nhiều người dùng cùng lúc.
Tích hợp ứng dụng	Hệ thống chỉ mới có cơ sở dữ liệu, chưa có giao diện người dùng (frontend) hoặc API.	Có thể phát triển thêm ứng dụng web hoặc mobile để tương tác với cơ sở dữ liệu.

Bảng 77: Hạn chế của hệ thống

10.3. Hướng phát triển và mở rộng

Mặc dù hệ thống hiện tại đã đáp ứng tốt các yêu cầu nghiệp vụ của tòa soạn, vẫn còn nhiều tiềm năng để nâng cấp thành một nền tảng xuất bản học thuật toàn diện. Dưới đây là các hướng phát triển trong tương lai:

10.3.1. Mở rộng chức năng nghiệp vụ

Tích hợp lưu trữ và tìm kiếm toàn văn: Hiện tại hệ thống chỉ quản lý siêu dữ liệu (metadata). Trong tương lai, có thể tích hợp với các dịch vụ lưu trữ đám mây (AWS S3, Google Cloud Storage) để lưu trữ file PDF/Word của bài nộp, đồng thời sử dụng công nghệ tìm kiếm full-text (MySQL Full-Text Search hoặc Elasticsearch) để hỗ trợ biên tập viên và độc giả tra cứu nội dung bài viết theo từ khóa.

Hệ thống cảnh báo và nhắc việc tự động: Xây dựng các tính năng gửi email/SMS tự động dựa trên dữ liệu ngày tháng (deadline). Ví dụ: tự động nhắc nhở chuyên gia phản biện khi sắp đến hạn nộp kết quả, hoặc thông báo ngay cho tác giả khi trạng thái bài viết chuyển sang "Được chấp nhận" hoặc "Bị từ chối".

Hỗ trợ đa ngôn ngữ: Mở rộng hệ thống để hỗ trợ nhiều ngôn ngữ khác nhau, phục vụ cộng đồng học giả quốc tế.

10.3.2. Ứng dụng trí tuệ nhân tạo (AI)

Gợi ý chuyên gia phản biện thông minh: Thay vì biên tập viên phải tự tìm kiếm chuyên gia thủ công, hệ thống có thể phát triển module phân tích ngôn ngữ tự nhiên (NLP) để tự động quét phần tóm tắt (Abstract) và từ khóa của bài nộp, từ đó gợi ý các chuyên gia phản biện có lĩnh vực chuyên môn phù hợp nhất dựa trên lịch sử đánh giá và hiệu suất của họ (dữ liệu từ `vw_reviewer_performance`).

Phát hiện đạo văn và đánh giá chất lượng: Tích hợp các công cụ phát hiện đạo văn (như Turnitin) và phân tích chất lượng bài viết bằng AI, hỗ trợ biên tập viên trong quá trình xem xét sơ bộ.

10.3.3. Phân tích và báo cáo nâng cao

Bảng điều khiển thống kê (Dashboard): Xây dựng hệ thống báo cáo trực quan cho ban biên tập với các chỉ số đo lường hiệu quả như: tỷ lệ bài được chấp nhận theo quý, thời gian phản hồi trung bình của từng chuyên gia, số lượng bài viết theo từng lĩnh vực chuyên môn, xu hướng gửi bài theo thời gian.

Phân tích dữ liệu chuyên sâu: Sử dụng các công cụ phân tích dữ liệu (Python, R, Power BI) để khai thác dữ liệu trong cơ sở dữ liệu, phát hiện các xu hướng nghiên cứu mới, đánh giá chất lượng bài viết và hiệu suất của hệ thống phản biện.

10.3.4. Bảo mật và phân quyền nâng cao

Xác thực đa yếu tố (MFA): Tăng cường bảo mật bằng cách bổ sung xác thực đa yếu tố cho các tài khoản quan trọng (Editor, Admin).

Mã hóa dữ liệu nhạy cảm: Áp dụng các biện pháp mã hóa đối với dữ liệu nhạy cảm như email, địa chỉ liên hệ và nhận xét đánh giá, nhằm tăng cường bảo mật thông tin và tuân thủ các quy định về bảo vệ dữ liệu (GDPR).

Phân quyền chi tiết hơn: Mở rộng hệ thống phân quyền đến từng bảng và từng cột cụ thể, đảm bảo mỗi người dùng chỉ có thể truy cập đúng phạm vi dữ liệu cần thiết.

10.3.5. Tối ưu hóa hạ tầng

Replication và Load Balancing: Đối với hệ thống có lượng truy cập lớn, có thể triển khai cơ chế nhân bản (master-slave replication) để phân tán tải, cải thiện hiệu năng đọc và đảm bảo tính sẵn sàng cao (high availability).

Partitioning: Đối với các bảng có kích thước lớn (MANUSCRIPT, REVIEW_SCORE, MANUSCRIPT_AUDIT), có thể áp dụng phân vùng (partitioning) theo thời gian (ví dụ: theo năm hoặc quý) để tăng tốc truy vấn và dễ dàng quản lý dữ liệu.

10.3.6. Tích hợp hệ thống khác

Kết nối với ORCID: Tích hợp hệ thống với ORCID (Open Researcher and Contributor ID) để tự động xác thực danh tính tác giả và reviewer, giảm thiểu sai sót và tăng độ tin cậy.

Tích hợp các tạp chí khác: Mở rộng hệ thống để hỗ trợ nhiều ấn phẩm khác nhau, tạo thành một nền tảng xuất bản học thuật đa tạp chí.

10.4. Kết luận chung

Đề tài "DataFrontiers Review Quarterly" đã đạt được những kết quả quan trọng trong việc xây dựng một hệ thống cơ sở dữ liệu quản lý quy trình xuất bản tạp chí học thuật trực tuyến. Hệ thống được thiết kế và triển khai trên nền tảng MySQL 8+ với 15 bảng, đầy đủ các ràng buộc toàn vẹn (UNIQUE, CHECK, FOREIGN KEY), 7 Triggers, 3 Views, 1 Function và 2 Stored Procedures, đáp ứng đầy đủ 16 yêu cầu nghiệp vụ của đề bài.

Sau quá trình kiểm thử với 12 test cases (5 Positive, 7 Negative), hệ thống đã chứng minh được tính đúng đắn và độ tin cậy. Tất cả các ràng buộc, trigger, procedure, function và view đều hoạt động đúng như thiết kế, đảm bảo tính toàn vẹn và nhất quán của dữ liệu. Việc áp dụng các kỹ thuật tối ưu hóa hiệu năng như Index và phân tích Execution Plan bằng EXPLAIN đã giúp cải thiện đáng kể tốc độ truy vấn, đạt khoảng 85%, khẳng định hệ thống sẵn sàng cho môi trường vận hành thực tế với khối lượng dữ liệu lớn.

Bên cạnh đó, hệ thống cũng đã được trang bị các cơ chế quản trị và bảo mật cơ bản như phân quyền người dùng theo nguyên tắc Least Privilege và kịch bản sao lưu/phục hồi dữ liệu. Đây là nền tảng vững chắc để có thể triển khai hệ thống trong môi trường production và mở rộng thêm các tính năng trong tương lai.

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu cốt lõi, vẫn còn nhiều hướng phát triển tiềm năng như tích hợp lưu trữ và tìm kiếm toàn văn, ứng dụng AI trong đề xuất chuyên gia phản biện, xây dựng bảng điều khiển thống kê và tăng cường bảo mật. Đây sẽ là những mục tiêu quan trọng để nhóm tiếp tục phát triển và hoàn thiện hệ thống trong tương lai.

Tóm lại, dự án đã hoàn thành xuất sắc mục tiêu đặt ra, cung cấp một giải pháp cơ sở dữ liệu toàn diện cho tạp chí DataFrontiers Review Quarterly, không chỉ phục vụ cho mục đích học tập mà còn có tiềm năng ứng dụng thực tế cao.

TÀI LIỆU THAM KHẢO

Date, C. J. (2003). *An introduction to database systems* (8th ed.). Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill Education.

Oracle Corporation. (2024). *MySQL 8.0 reference manual*. <https://dev.mysql.com/doc/>

MySQL Tutorial. (2024). *MySQL FOREIGN KEY constraints*.
<https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>

MySQL Tutorial. (2024). *MySQL stored procedures and functions*.
<https://dev.mysql.com/doc/refman/8.0/en/stored-programs-views.html>

MySQL Tutorial. (2024). *MySQL triggers and events*.
<https://dev.mysql.com/doc/refman/8.0/en/triggers.html>

MySQL Tutorial. (2024). *Optimizing queries with EXPLAIN*.
<https://dev.mysql.com/doc/refman/8.0/en/explain.html>

JGraph Ltd. (2024). *Draw.io (diagrams.net)* - Online diagram tool. <https://app.diagrams.net/>

Oracle Corporation. (2024). *MySQL Workbench - Visual database design tool*.
<https://www.mysql.com/products/workbench/>